Catapult HLS User View Scheduling Rules

Stuart Swan, Platform Architect, Siemens EDA

25 April 2026

Introduction

In a high-level synthesis (HLS) design methodology, much of the benefit is lost if design verification and debug must still be performed at the RTL level. In manual RTL design flows, RTL verification is typically one of the most costly and time-consuming phases of the overall flow. In HLS flows, RTL-level verification is even more challenging, because the RTL is machine-generated rather than hand-crafted.

This document presents a set of HLS scheduling rules, a modeling methodology, and a collection of formal proofs that together support a flow in which almost all design verification and debug for full systems can be carried out on the pre-HLS SystemC model, subject to the explicitly stated assumptions, observation-boundary choices, witness-selection conditions, and liveness/progress premises described in the formal appendices. The methodology and proofs are organized around a latency-insensitive design style, while still fully supporting real-world techniques such as:

- latency-sensitive signal-level protocols
- latency-sensitive portions of systems
- both sequential and combinational logic
- pipelined designs
- shared memories between sequential processes
- reordering of RAM accesses to optimize hardware pipelines
- removal of pipeline registers for stable signal inputs to hardware pipelines

These techniques capture the requirements gathered from hundreds of Catapult HLS customer tape-outs and are needed to meet the demanding quality-of-results (QOR) targets traditionally achieved with manual RTL design.

The methodology presented here builds on established verification practices such as SystemVerilog UVM, constrained-random stimulus generation, and functional and code coverage. The system under test is modeled and verified in SystemC, and formal proofs establish behavioral equivalence for the synthesized RTL implementation.

Although the focus of this document is HLS, the underlying modeling methodology and formal proof framework are not limited to HLS-generated RTL. The key benefit is compositionality: when individual processes and their communication interfaces satisfy the same observable-interface, capacity, synchronization, ordering, and progress obligations stated in the formal appendices, those processes can be composed with predictable behavior between the SystemC model and the RTL model. Thus, the same approach can also be used in manual RTL design methodologies, with RTL verification focused on showing that the hand-written RTL satisfies the same implementation-side obligations that HLS-generated RTL is required to satisfy.

A document abstract is provided in Appendix M.

Document Goals

This document presents HLS tool scheduling rules and a modeling methodology. The goals are:

1. To provide easy-to-understand rules to HLS users.
2. To ensure precise and consistent rules in both SystemC and C++.
3. To offer rules that are effectively compatible with how Catapult currently operates.
4. To enable the best possible *quality of results* (QOR) in Catapult synthesis.
5. To cover all known user requirements and scenarios.
6. To serve as a suitable starting point for a standardization proposal (e.g., in Accellera SWG).

To illustrate the goals of this document more specifically, consider what an engineer writing a testbench for an HLS model needs to understand about how HLS tools operate. This engineer may be using SystemVerilog UVM or may be writing a testbench in C++/SystemC. They likely are not an expert in any specific HLS tool (and may not want to be), but their testbench needs to work for both the pre-HLS model as well as the post-HLS model. Thus, it is crucial for the DV engineer to have a precise understanding of how the HLS tool will transform the design while still enabling it to be fully verified. This document describes what transformations the HLS tool is allowed to perform so that the pre-HLS and post-HLS models can be effectively verified with the same testbench. The overarching philosophy of the scheduling rules is to present "no surprises" to such a DV engineer, while still giving the HLS tool ample freedom to optimize the design.

Background

HLS tools generate RTL from C++ models. Broadly speaking, this conversion takes a sequential C++ model and turns it into concurrent hardware that maintains the same behavior. HLS tools identify concurrent processes within the C++/SystemC model and then independently synthesize each process. Briefly, some of the techniques that HLS tools use to achieve good HW QOR when synthesizing each process include:

- Optimized scheduling based on the selected silicon target technology.
- Automatic HW pipeline construction according to the user's specifications.
- Automatic HW resource sharing.
- Automatic scheduling of memory accesses.

The internal behavior of each process is specified by the control and dataflow behavior of the C++ code within the process. However, the external communication that each process has with other processes and HW blocks is specified via IO operations that are coded within the model. To enable a reliable, scalable, and verifiable HLS flow that generates high quality hardware, the scheduling behavior of these IO operations needs to be precisely handled at all steps of the flow. This document specifies the rules that govern the scheduling behavior for these IO operations within HLS models. These rules are specified with respect to an individual process, but the intent of the rules is to enable reliable and verifiable behavior of large sets of interacting processes operating as a system in real-world designs. (Appendix G provides formal guarantees regarding the equivalence of the pre-HLS and post-HLS systems.)

By default, HLS tools can insert additional clock cycles (or latency) anywhere within a process -- for example, when pipelining a loop or to enable HW resource sharing. The overall approach used in this

document is to make the entire design and testbench system *latency insensitive* to the maximum extent possible, while still fully enabling key HW optimizations. In addition, the overall approach enables the pre-HLS system simulation to be capacity-accurate and throughput-accurate with respect to the post-HLS RTL system.

In some cases, designs or testbenches may use protocols which are latency-sensitive. These situations can be handled by isolating the latency-sensitive portions to small, self-contained parts of the design or testbench, and then keeping the rest of the design and testbench latency insensitive. See Appendix D for more information.

In many cases the overall design will need to satisfy end-to-end latency requirements. For these designs it is still highly advantageous to use a latency-insensitive modeling approach and verify in the post-HLS model that the overall design latency requirements have been satisfied, since this is typically easy to do.

This document focuses on sequential HW processes. Combinational HW processes are supported but mostly not discussed since their synthesis is straightforward. All the examples and discussion in this document are for SystemC processes that are sensitive only to a single rising clock edge. (Appendix O discusses support for multiple clock domains.)

This document distinguishes between the following:

1. The *conceptual model* for the scheduling rules.
2. The simulation behavior of a model using the rules in C++ or SystemC.
3. The synthesis of a C++/SystemC model using the rules in an HLS tool such as Catapult.

The goal is to align each of these three cases as closely as possible, so that the user has easy-to-understand rules, while simulation and synthesis work without surprises. However, as we will see, there are practical considerations which may in certain cases cause small deviations from the conceptual model in either simulation or HLS.

A simple real-world example that illustrates the motivation for the scheduling rules is provided in Appendix A of this document.

The examples referred to in this document are available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master

The most up-to-date version of this document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

An Analogy from RTL Synthesis

To better understand the specific purpose of this document, let's consider how RTL synthesis works. Say you have a sequential block that you are modeling in Verilog RTL, and it has an output port coded like:

```
Out1 <= #some_delay new_val;
```

In Verilog simulation, if some_delay is less than the clock period of the block, then it will probably not affect the overall cycle-level behavior of the system during simulation. However, if some_delay is more than the clock period, it probably will.

During RTL synthesis, all RTL synthesis tools will ignore all delays in the input model, in this case even if some_delay is greater than the clock period. Some RTL synthesis tools might give a warning or error for code like the above such as "Simulation and synthesis results are likely to mismatch because the delay in model is greater than clock period." Some RTL synthesis tools might outright reject a model containing such delays.

One might argue that RTL synthesis tools should always match the Verilog simulation behavior of the input model. But the overall approach works well because RTL is a good and simple *conceptual model* that users and tool vendors can align around. The slight differences between the *simulation model* and the *conceptual model* used by RTL synthesis tools can be easily managed.

We'll return to this example later in this document.

Next, let's clarify some terminology related to signal IO. Say you have a Verilog model like:

```
forever begin
  @(posedge clk);  // wait for 1st rising clock edge
  output1 <= input1 + 10;
  @(posedge clk);  // wait for 2nd rising clock edge
end
```

In this example, input1 is sampled when the process wakes on the first @(posedge clk) and evaluates the RHS. For the purposes of the scheduling rules, we anchor that Read(input1,·) to the most recent synchronization point. The resulting Write(output1,·) is anchored to the next synchronization point (the cycle in which the written value is treated as stable for others to observe).
Cycle-level anchoring convention (used throughout this document). For cycle-level reasoning, each signal IO is assigned an anchor cycle relative to surrounding synchronization calls (e.g., wait() / SyncChannel / @(posedge clk)):
• Read(sig, val) is anchored to the closest preceding synchronization call, i.e. pred_S(Read).
• Write(sig, val) is anchored to the closest succeeding synchronization call, i.e. succ_S(Write).
This convention is only for cycle-level reasoning and does not depend on simulator update regions or delta-cycle ordering.

Catapult HLS Status Concerning Rules in this Document

A separate document provides a list of clarifications related to support within Catapult HLS for the rules described in this document. The items in the list are named *Cat#*, so that each item has a unique number.

This document annotates certain rules with *Cat#* to refer to the items in the separate document.

Terms Used in this Document

Latency-Insensitive: In digital hardware design, *latency-insensitive* refers to a system that operates correctly despite variable communication delays between components. This is achieved by using mechanisms to decouple computation from communication timing. Such designs improve scalability and reliability in complex systems with unpredictable or variable latencies. ARM's AXI4 and APB are examples of latency-insensitive protocols. ARM's AMBA 5 CHI credit-based NOC protocol is also an example of a latency-insensitive protocol.

Process: In Verilog, a process is an *always block* and its equivalent constructs. In SystemC, a process is an instance of an SC_THREAD or SC_METHOD.

Message-passing Interface: A *message-passing interface* reliably delivers messages (or transactions) from one process to another. This document uses this term to denote the type of communication found in Kahn Process Networks. See https://en.wikipedia.org/wiki/Kahn_process_networks
Message-passing read interfaces are always separate from message-passing write interfaces – there are no bidirectional message-passing interfaces. In this document, message-passing channels are assumed to be point-to-point: for each channel c, exactly one producer performs all Push_c operations and exactly one consumer performs all Pop_c operations.

Synchronization Interface: A *synchronization interface* synchronizes one process with another and/or with a global clock. For an example of a synchronization interface, see https://en.wikipedia.org/wiki/Barrier_(computer_science)

Signal IO: In digital HW design, signals are the fundamental communication mechanism. Signals enable communication between two HW blocks/processes, but communication with signals in real HW always incurs at least some delay because communication cannot be faster than the speed of light.
In HDLs and in SystemC, signal delays are modeled with the *delayed update* semantics.

blocking / non-blocking: In this document, a blocking message-passing operation is modeled as issuing a persistent request that remains asserted (and with stable payload, if applicable) until the operation commits (the message is sent or received). A non-blocking message-passing operation returns immediately with a status indicating whether the operation committed in that cycle.

One-way / two-way handshake protocols: A one-way handshake protocol only has one signal between a sender and receiver to synchronize communication. A two-way handshake protocol has two signals (one in each direction) between a sender and receiver to synchronize communication.

shall: This term indicates that a compliant tool or flow is required to follow the indicated rule.

may: This term indicates that a compliant tool or flow is allowed to follow the indicated rule but is not required to do so.

Classes of Operations Involved in Scheduling Rules

There are three classes of operations involved in the scheduling rules:

1. Calls to message-passing interfaces (which are all ac_channel methods, all SystemC MIO calls except calls to SyncChannel)
2. Calls to synchronization interfaces (which are calls to ac_sync and Matchlib SyncChannel, also ac_wait and SystemC wait)
3. Signal IO (which are SystemC signal reads and writes, also C++ model *direct inputs*)

All the operations above are referred to as *IO operations*.

Basic Conceptual Model

The basic conceptual model encompasses processes that have no loop pipelining but may have preserved loops. If a process has a preserved loop, then the user may place a wait statement in the loop. Wait statements explicitly placed in the model are called *explicit wait statements* in this document and are classified as calls to a *synchronization interface*.

The basic conceptual model rules are:

1. Synchronization interface calls within a process always remain in the source code order.

2. Signal read operations occur at the closest preceding call to a synchronization interface. (Cat1)

3. Signal write operations occur at the closest succeeding call to a synchronization interface.

4. Message-passing operations are free to be reordered subject to the following constraints:
- All message-passing operations before a call to a synchronization interface shall be issued before or in the same cycle in which the synchronization interface commits. (Here "the synchronization interface commits" refers to the commit of that synchronization call in the calling process's trace. For blocking message-passing operations (Push/Pop), the process cannot complete the synchronization call until any earlier issued blocking message requests in that interval have committed.)
- All message-passing operations after a call to a synchronization interface shall be issued after the cycle in which the synchronization interface call commits.
- Two message-passing operations on separate interfaces which appear in sequence in the model may be issued either in the same sequence or in parallel in simulation and in synthesis, but they shall not be issued in the reverse sequence. (Cat2)

Some explanation for the very last point: While pure message-passing systems with unbounded FIFOs are immune to reordering concerns, real-world hardware implementations have finite buffer sizes. Reversing the order of message-passing calls in a system with bounded FIFOs can introduce deadlocks that were not present in the original model. The last rule prevents HLS from introducing deadlocks *via this specific mechanism* (reversing the program order of message-passing calls that appear in sequence in the source). However, deadlock behavior can still be affected by other transformations that change the *timing* of request/commit—most notably overlapped execution in pipelined loops and finite-capacity effects.

Pipelined Loops

When a loop is pipelined in HLS, the body of the loop is split into pipeline stages. HLS may start the next iteration of the loop before the current iteration has completed, thus increasing throughput as compared to a non-pipelined implementation.

The user may place wait statements in the body of a pipelined loop to manually separate operations into their respective pipeline stages but is not required to (and in most cases will not do so). We call these wait statements *explicit pipeline stage wait statements*. The scheduling rules for pipelined loops are the same as the rules given in the basic conceptual model, with the addition of these pipeline stage wait statements into the set of calls to synchronization interfaces.

To guarantee equivalent behavior between the pre-HLS and post-HLS systems, pipelined loops must be implemented with automatic flush to prevent deadlocks. See Appendix B for a formal presentation of pipelined loops.

When HLS pipelines a loop, multiple iterations of the loop are overlapped and execute at the same time. During loop pipelining, for all IO operations, HLS shall ensure that an access to a specific message-passing interface, signal, or synchronization interface shall not be reordered relative to same-interface accesses from other iterations.

During pipelining any signal reads and writes and associated synchronization interface calls become embedded in specific pipeline stages. Considering the entire set of signals read or written by the process, if HLS pipelining would cause the order of signal IO to differ between the pre-HLS and post-HLS models, or if any two signal IO operations that occur on the same clock edge in the pre-HLS model would not occur on the same clock edge in the post-HLS model, then the HLS tool shall detect such a situation and require the user to explicitly indicate in the model source (e.g., via a pragma) that HLS pipelining can still be used. (Cat7) In other words, if a process communicates with other processes using only latency-insensitive protocols, then HLS pipelining by default shall not break any of the protocols.

Synchronization boundary for pipelined loops.
If a pipelined loop is separated from later work by an explicit synchronization interface call, then while the process is pending at that synchronization call, any back-annotated message-passing capacity B(c) whose accepted data could otherwise be consumed only by work after that synchronization call shall be treated as unavailable to upstream producers until the synchronization call commits. Equivalently, the producer-facing availability corresponding to such capacity is temporarily withdrawn at the synchronization boundary. Work already accepted before the boundary may drain under the automatic-flush rules, but no new transfer may be accepted across that boundary into the next post-sync interval before the sync commits.

Direct Inputs

Normal SystemC signal read operations occur at the closest preceding synchronization interface call (e.g., wait statement) in both the pre-HLS and post-HLS models. (Cat1). If the HLS tool adds states to the process, or if it pipelines the process, then this implies that the HLS tool must add registers for each such read operation such that the read occurs where specified in the pre-HLS model, and the value is stored until the point where it is consumed in the post-HLS model. The area cost of such registers may be high

if there are a lot of signals, and in some cases, it may be unneeded area since the value of the signals may not actually need to be stored internally to the process.

The simplest case is if such external signals are held stable after the process comes out of reset. In this case, HLS may assume that it is free to physically sample the signal values as late as possible, with no need for register storage, while the logical Σ-visible Read event remains anchored as specified by E2. This case is handled with the following pragma on SystemC signals and ports (Cat6):

#pragma hls_direct_input

To ensure that there are no pre-HLS versus post-HLS simulation mismatches, the environment that drives the signal shall hold it stable after all receiving processes that use this signal with this pragma come out of reset. Note that with this approach it is allowable to have *dynamic resets*, i.e. activation of block resets and associated resetting of direct input signals as part of the normal operation of the HW.

A related but somewhat more complex case is where input signals to the HW block may only be changed at "agreed upon" times, typically while the portion of the HW block that relies on them is temporarily idle. For example, a block may process 2D images. At the start of each new image, it may be desirable for the TB or external environment to update the control signals for how the block will process the next image. Typically HLS designs are pipelined, and the HW pipeline for the current iteration must be fully *ramped down* before the input signals can be updated to affect the next iteration. In the pipelined-loop case, this relies on the general synchronization-boundary rule for pipelined loops: while the process is pending at the designated sync, any back-annotated message-passing capacity whose accepted data would belong to the next post-sync interval is temporarily unavailable to upstream producers until the sync commits. In this case we can use the SystemC *SyncChannel* or C++ *ac_sync* primitives to precisely synchronize the DUT with the TB/environment to enable the input signals to be updated at the correct time. The `#pragma hls_direct_input_sync` directive shown below associates the sync operation with the direct inputs that it controls. The precise synchronization scheme shown here ensures that there are no pre-HLS versus post-HLS simulation mismatches even though we are using direct inputs and also changing their values while the design is executing.

```
  // This is example 61 in Catapult Matchlib examples
  sc_in<bool> SC_NAMED(clk);
  sc_in<bool> SC_NAMED(rst_bar);

  Connections::Out<uint32_t>  SC_NAMED(out1);
  Connections::In<uint32_t>   sample_in[num_samples];
  Connections::SyncIn         SC_NAMED(sync_in);
#pragma hls_direct_input
  sc_in<uint32_t> direct_inputs[num_direct_inputs];


  void main() {
    out1.Reset();
    sync_in.Reset();

#pragma hls_unroll yes
    for (int i=0; i < num_samples; i++) {
      sample_in[i].Reset();
    }

    wait();  // reset state
```

```
    while (1) {
#pragma hls_direct_input_sync all
     sync_in.sync_in();

#pragma hls_pipeline_init_interval 1
#pragma hls_stall_mode flush
     for (uint32_t x=0; x < direct_inputs[0]; x++) {
       for (uint32_t y=0; y < direct_inputs[1]; y++) {
        uint32_t sum = 0;
#pragma hls_unroll yes
        for (uint32_t s=0; s < num_samples; s++) {
           sum += sample_in[s].Pop() * direct_inputs[2 + s];
        }
        ac_int<32, false> ac_sum = sum;
        ac_int<32, false> sqrt = 0;
        ac_math::ac_sqrt(ac_sum, sqrt);  // internal loop unrolled in cat .tcl file
        if (sqrt > direct_inputs[7])
          out1.Push(sqrt);
       }
     }
    }
  }
};
```

From the perspective of the testbench or the environment, the updating of the signals controlled by the `hls_direct_input_sync` directive shall only occur at a precise point. The TB shall first wait for the `rdy` signal for the sync to be asserted by the DUT, and then the TB shall update all the input signals it wishes to change while simultaneously driving the sync `vld` signal high for one cycle.

It is important to note that the only safe operation to use to synchronize the updating of direct inputs is the sync operation as shown above. Other operations such as Push/Pop or ac_channel operations should not be used for this. In terms of the Appendix G trace model, the Σ-visible Read(sig,val) events remain anchored to their synchronization points; any additional internal sampling made by the implementation between anchors is treated as an ε-step and must not change the recorded value label.

In the example above the DUT block that is being synthesized determines when to call sync, and thus it determines when the direct inputs will be updated. In some cases, it may be necessary for the environment around the DUT to determine when the direct inputs should be updated. In this case the same approach as shown above should be used; however a separate input from the environment to the DUT (either using a signal or a message-passing interface) should request that the DUT call sync as soon as feasible. This will ensure that the DUT has properly ramped down its pipeline and is ready to receive the newly updated direct inputs as per the overall synchronization scheme described above.

Additional Options for Scheduling Message-passing Interfaces

The following option may be added during HLS (Cat2):

```
STRICT_IO_SCHEDULING=relaxed
```

When this is specified, the HLS tool is allowed to reorder message-passing interface calls freely on distinct interfaces (still not moving calls across synchronization interface calls). This relaxed mode may violate the default no-reverse discipline and therefore may introduce deadlocks that are not present under the default rules. It is recommended that this relaxed option only be used when the user wants to

see what order the HLS tool prefers to schedule message-passing interface calls (e.g., to achieve best QOR).

Once the user knows the preferred order, it is recommended that the user modify the pre-HLS source code to reflect the preferred order, and then return to the use of the default scheduling modes. This methodology ensures that HLS cannot introduce new deadlocks *via reversal of source order on distinct message interfaces* as described earlier. Deadlock behavior may still depend on bounded-capacity effects and on overlapped execution (e.g., loop pipelining); those cases are addressed separately in this document via the pipelined-loop discussion (automatic flush / WFG-inertness) and the later deadlock reasoning. The formal equivalence and deadlock-preservation results in Appendices G–K assume STRICT_IO_SCHEDULING is not set to relaxed.

Scheduling of Array Accesses

Arrays may appear in HLS models, and they may be preserved through synthesis and mapped to RAMs. Pointers may also appear in HLS models, and pointer dereferences are resolved to array accesses during HLS.

There are two cases to consider for arrays for the purposes of the scheduling rules:

1. Array instantiation in the HW is internal to the process
   - The array accesses are not visible outside the process, and thus their scheduling is also not visible externally.
   - All the scheduling rules described elsewhere in this document remain unaffected in this case.

2. Array instantiation in the HW is external to the process
   - In this case the user model shall indicate how array accesses are mapped onto IO operations that are external to the process.
   - We call this the *array access mapping layer*. The *array access mapping layer* maps array accesses onto IO operations described above (signal IO, message-passing interface calls, and synchronization calls).
   - The user model may indicate that it is allowable for HLS to transform array accesses, for example, to cache, merge, split, or reorder array accesses (e.g., to improve QOR). These transformed operations, if allowed, are an outcome of using the *array access mapping layer*.
   - In all cases the scheduling rules described elsewhere in this document for the core IO operations (signal IO, message-passing calls, synchronization calls) remain unaffected.
   - In all cases array accesses shall remain constrained by any explicit synchronization interface calls present in the process in the source model. Precisely speaking, all array reads or writes before a synchronization call shall commit before or in the same cycle at which the synchronization call commits, and all array reads or writes after a synchronization call shall commit in a cycle after the synchronization call commits. This rule is stated in terms of commit cycles (the cycle the mapped memory storage is accessed); the scheduler enforces it by choosing issue cycles consistent with the fixed access latency defined by the array access mapping layer.
   - Note that if transformed operations occur and array accesses are visible externally in both the pre-HLS and post-HLS model, then comparison of pre-HLS and post-HLS behaviors may need to account for the transformed operations.

When a loop is pipelined, multiple iterations of the loop are overlapped and execute at the same time. During loop pipelining, an access to a memory interface (or array) may be moved over or in parallel with an access to the same memory if the HLS tool can prove the reordering is conflict-free. If the array/memory is external to the process, the *array access mapping layer* shall indicate that such reordering is allowable if such reordering is to occur during loop pipelining.

Definitions: Issue vs. Commit

Informally, we can define commit and issue as follows:

Definition: Commit (message channel) — as observed by an external clocked observer
Commit is the cycle in which an external clocked observer sees a transfer *complete*:

- A message commits in the cycle when the observer sees both valid and ready high at the same time.
- The committed payload is the data value seen in that same cycle.

Definition: Issue (message channel) — as observed by an external clocked observer
Issue is the cycle in which an external clocked observer sees a transfer *start*:

- A message action issues when a Push operation asserts vld, or a Pop operation asserts rdy.

See Appendix C for formal definitions of issue and commit.

For RAM reads/writes as presented above, commit denotes the precise cycle the memory storage is accessed; since the access latency is fixed and known to the HLS scheduler, it can schedule the operation's issue cycle so that the commit cycle satisfies the required ordering relative to Sync. The "Scheduling of Array Accesses" constraints above are therefore written in commit-time.

Additional States Added by HLS Synthesis

By default, HLS synthesis tools may add additional states to processes (e.g. add latency to enable resource sharing), which may introduce latency differences in the interface behavior between the pre-HLS and post-HLS models. These additional states are never included in the set of *synchronization interface calls* as described above.

When the directive IMPLICIT_FSM=true is set on a process, the HLS synthesis tool shall ensure that the cycle-level behavior of the interfaces of the pre-HLS and post-HLS models shall be identical. With this option, the internal state machines of the pre-HLS and post-HLS models will be the same.

When the directive IO_MODE=FIXED is set on a process, the HLS synthesis tool shall ensure that the cycle-level behavior of the interfaces of the pre-HLS and post-HLS models shall be identical. With this option, it is still possible that the state machine internal to the process is different between the pre-HLS and post-HLS models (e.g. the post-HLS model may choose to use a pipelined multiplier where the pre-HLS model did not.)

Avoiding Pre-HLS and Post-HLS Simulation Mismatches

The scheduling rules described in this document are designed to be easy to understand, while providing good QOR via HLS and generally avoiding any mismatches between the pre-HLS and post-HLS simulation behaviors.

Non-blocking message-passing operations (PushNB/PopNB in SystemC, C++ ac_channel nb_read/nb_write) are a potential source of mismatches between pre-HLS and post-HLS simulations since their behavior is inherently dependent on the latency within the model, which often changes during HLS. Because of this, non-blocking message-passing interfaces should only be used when no alternative approach is possible. For example, non-blocking message-passing interfaces are required to model time-based arbitration of multiple message streams which access a shared resource. A full discussion of recommended guidelines on the use and verification of non-blocking message-passing interfaces is provided in Appendix L. Note that the scheduling rules described previously in this document fully specify how HLS tools are required to schedule such operations.

Unidirectional message-passing between two processes should not be relied on to achieve synchronization between the two processes, since in general the message latency and storage capacity between the processes may be variable. Also, depending on buffering/pipelining in the realization, the time between a producer's committed Push and the consumer's committed Pop (and the transient in-flight occupancy) may vary. Two unidirectional message-passing channels in opposite directions can be relied upon for synchronization between two processes. (An example of this is the AXI4 ar and r, and aw and b, channels). Note that such synchronization is weaker than explicit synchronization like SyncChannel or signal IO that uses a two-way handshake.

SystemC signal IO operations are a potential source of pre-HLS versus post-HLS simulation mismatches since timing behaviors may change between the two models. The following section provides guidance and rules to help avoid potential mismatches due to signal IO.

When signal IO operations are synchronized with a wait statement, there generally should be a proper two-way handshake associated with the wait statement so that the signal IO is latency insensitive. (Note that this statement does not apply to the signal synchronization approaches described in the Direct Inputs section.)

For example, here's a simple two-way handshake protocol when writing the signal `out_dat`:

```
out_dat = value;
out_vld = 1;
do {
  wait();
} while (out_rdy != 1);
out_vld = 0;
```

And here's a two-way handshake example when reading signal `in_dat`:

```
in_rdy = 1;
do {
  wait();
} while (in_vld != 1);
value = in_dat;
in_rdy = 0;
```

Some signal-level protocols have different two-way handshaking approaches (e.g. ARM APB), but they are still latency insensitive.

If signal IO operations are associated with a wait statement and that wait statement does not have a proper two-way handshake, then the signal IO is likely to be latency-sensitive and may result in pre-HLS versus post-HLS simulation mismatches. In some systems a one-way signal handshake is sufficient for reliable system operation. See Appendix E for further discussion.

The scheduling rules state that signal IO operations occur at either SystemC wait statements or SyncChannel calls (sync_in and sync_out). In the remainder of this section, we will use *wait* statements to refer to both.

RULE 1: It is always best coding style to group signal write operations just before their corresponding wait statement, and signal read operations just after their corresponding wait statement. (Cat3). An example is below:

```
 sc_in<int> i1;
 sc_in<bool> go;
 sc_out<int> o1;
 void my_thread() {
   int new_val=0;
   while (1) {
     o1.write(new_val);
     do {
       wait();
     } while (!go.read());
     new_val = i1.read();
     new_val = some_function(new_val); // function has no internal IO
   }
}
```

By placing the signal IO operations as close as possible to their corresponding wait statement, the HW intent is very clear. And there is no benefit either in terms of simulation performance or HLS QOR if they are placed further away from their corresponding wait statement.

Let's look at another similar example, which now also uses a Matchlib Connections blocking Pop operation:

```
 sc_in<int> i1;
 sc_in<bool> go;
 sc_out<int> o1;
 Connections::In<int> pop1;
 void my_thread() {
   int new_val=0;
   while (1) {
     o1.write(new_val);
     do {
       wait();
     } while (!go.read());
     int pop_val = pop1.Pop();
     new_val = i1.read();
     new_val = some_function(new_val + pop_val); // function has no internal IO
   }
}
```

Under the anchoring rules, i1.read() remains associated with the same synchronization point regardless of the intervening Pop. However, in raw pre-HLS simulation, a blocking Pop may stall for one or more cycles, so i1 can change before the i1.read() executes—creating behavior that differs from the anchored interpretation (and potentially from post-HLS scheduling). The fix is simply to place i1.read() immediately after its intended wait() anchor; this removes the mismatch risk without affecting QOR or simulation performance.

To automatically avoid all such potential pre-HLS versus post-HLS simulation mismatches, HLS tools may provide error or warning messages in cases where models have the pattern shown above. Precisely speaking: if a blocking message-passing operation separates a signal read or write operation from its corresponding synchronization interface call, then the HLS tool may emit an error or warning indicating that reordering the signal IO operation and the message-passing operation in the source text is advisable.

Another scenario in which RULE 1 applies is shown below:

```
 sc_in<bool> go;
 sc_out<int> o1;
 void my_thread() {

   while (1) {
     wait();              // WAIT 1
     o1.write(some_value);
     if (go.read()) {
       some_value = some_function();
     }
     else {
       wait();            // WAIT 2
     }
     some_other_function();
     wait();              // WAIT 3
   }
}
```

The signal read of `go` is clearly and uniquely associated with WAIT 1. However, the signal write of `o1` associates with WAIT 2 if `go` is false and WAIT 3 if it is not. This is a violation of RULE 1 and should be flagged as an error during HLS. The fix, as before, is to move the signal IO operation as close as possible to its intended wait statement so that the association is unconditional.

Next, let's consider *rolled* (or *preserved*) loops that perform signal IO within the loop body. Consider the following example:

```
 sc_in<int> i1;
 sc_out<int> o1;
 void my_thread() {
     wait(); // reset state
     while (1) {
       wait(); // start of while loop
       #pragma hls_unroll no
       for (int i=0; i < 10; i++) {
         o1.write(i1.read() * i);
       }
   }
}
```

Note that the `i1.read()` operation is located inside the `for` loop, so presumably the user's intent is that it should be read as the loop iterates. *If that is not the user's intent, they simply should move the i1.read() operation before the loop start.*

In the post-HLS simulation, each iteration of the loop will consume at least one clock cycle, and a new value for i1 will be read (and a new value for o1 written) on each iteration. Again, this is the user's intent as per the code. In the pre-HLS simulation, the for loop body will execute in zero time, and only the last write to o1 will have any effect. The solution to avoid this mismatch is to manually place a `wait()` statement within the for loop body so that the signal IO synchronization is explicit in the pre-HLS simulation.

RULE 2: If you have signal IO operations within *rolled* (or *preserved)* loops, manually place a wait statement within the body of the loop to avoid pre-HLS versus post-HLS simulation mismatches, and while doing so also follow RULE 1.

To automatically prevent these types of pre-HLS versus post-HLS simulation mismatches, HLS tools may emit warning or error messages if they encounter a rolled loop with signal IO operations but no wait statement in the loop body. (Cat4)

If a pre-HLS model adheres to RULE 1 and RULE 2, then all signal IO in the post-HLS model shall occur only at clock cycles that correspond to explicit wait statements or explicit synchronization statements. For direct-input signals/ports (e.g., #pragma hls_direct_input and #pragma hls_direct_input_sync), this requirement refers to the logical (Σ-visible) anchor cycle of the Read/Write: the event is still considered to occur at its synchronization point even if HLS physically samples a stable signal later; such late-sampling micro-steps are ε-steps and are not Σ-visible. User designs that do not adhere to both RULE 1 and RULE 2 are *ill-formed.*

Returning to the Analogy from RTL Synthesis

At the beginning of this document, we presented the example of a Verilog sequential block with an output coded like:

```
Out1 <= #some_delay new_val;
```

Recall that in Verilog simulation, if some_delay is less than the clock period of the block, then it will probably not affect the overall cycle-level behavior of the system during simulation. However, if some_delay is more than the clock period, it probably will.

During RTL synthesis, all RTL synthesis tools will ignore all delays in the input model, in this case even if some_delay is greater than the clock period. Some RTL synthesis tools might give a warning for the code above like "Simulation and synthesis results are likely to mismatch because delay in model is greater than clock period."

HLS tools that adhere closely to the *conceptual model* presented in this document should automatically provide errors or warnings for violations of RULE 1 and RULE 2 as described in the section above. This is analogous to the error message that the RTL synthesis tool would provide in the example directly above.

However, HLS synthesis tools that do not follow the rules described in this document might adhere in these cases more closely to the pre-HLS SystemC simulation behavior. In this case such HLS tools might not provide any errors or warnings for violations of RULE 1 and RULE 2. This is analogous to an RTL synthesis tool being very smart (maybe even too smart) about synthesizing matching HW based on the actual value of some_delay in the example directly above.

Summary

At the beginning of this document, we said that the intent was to present "no surprises" to a DV engineer who is using a single testbench to verify both the pre-HLS and post-HLS models. The key aspects of the document which support this are:

- Three groups of IO operations are defined (message-passing, signal IO, and synchronization calls) and each is treated uniformly. These IO operations are easy for verification engineers to understand because they are already using them in their testbenches.

- The document specifically avoids complex constructs such as *protocol regions* used in some HLS tools.

- The document preserves the ability of the pre-HLS SystemC model to be *throughput accurate* by using a library such as Matchlib.

- Synchronization calls can affect the scheduling (anchoring) of signal IO operations, and synchronization calls can affect the scheduling of message-passing calls. In the conceptual anchoring semantics of this document, message-passing calls do not affect the anchor cycle of signal IO operations (and signal IO does not affect the legality/order of committed message transfers) except through explicit synchronization. Practically, in raw pre-HLS SystemC simulation a blocking message operation may introduce additional cycles of waiting, so signal IO statements that are lexically separated from their intended anchor can observe different values; RULE 1 addresses this by recommending placement of signal IO adjacent to its anchor.

- HLS cannot by default reverse the order of message-passing calls, so it cannot introduce new deadlocks *via call-order reversal*. For pipelined loops (overlapped iterations) and other transformations that change the timing of request/commit under bounded capacities, deadlock preservation depends on the additional pipelined-loop discipline described in this document (automatic flush / WFG-inertness), not on the no-reverse rule alone.

- HLS pipelining is largely a *don't care* from the perspective of the verification engineer. If the design and the testbench are insensitive to changes in latency, and if external array accesses are not reordered or rearranged during loop pipelining, then the possible use of HLS pipelining will not affect verification, provided the pipelines flush automatically. Even if the design or testbench are sensitive to changes in latency, or if they are sensitive to reordering or rearranging of external memory accesses due to the use of HLS loop pipelining, then the behavior of the DUT will only change in expected (rather than unexpected) ways. (Appendix G provides formal guarantees regarding the equivalence of the pre-HLS and post-HLS systems.)

- For sequential processes, signal IO operations in the post-HLS model always occur exactly at synchronization points (e.g., wait statements) that are explicit in the pre-HLS source. For direct-input signals/ports, “occur at synchronization points” is meant in the logical/anchoring sense: the Σ-visible Read/Write is timestamped at the synchronization point even if the implementation samples the stable signal later; those internal late samples are ε-steps. For combinational processes, observable signal Read/Write events are instead interpreted at the cycle’s ε-quiescent combinational fixed point, as specified by R2C.

Appendix A – Factory Analogy

The scheduling rules described in this document apply to pre-HLS and post-HLS HW models. Although the rules may seem abstract and perhaps even arbitrary, they are shaped by the need to model systems in the real world that are required to have predictable behavior.

To understand the motivation behind the rules, it might be helpful to draw a simple real-world analogy and its correspondence to the rules described earlier.

Consider a factory that produces various types of wooden furniture:

The factory consists of people (processes) stationed at workbenches with various tools.

Each person is given written instructions about the specific tasks they are to perform (C++ code within a process).

People are instructed to send or receive objects (messages) to or from other people in the factory.

Sending or receiving objects may be blocking or non-blocking from the perspective of a person.

There is a clock with a second hand on the factory wall that everyone can see. (HW clock).

People have colored flags they can raise or lower to communicate with other people (signals).

If someone raises or lowers a flag, this is only seen by others the next time the clock second hand is at the top of the clock. (propagation delay of signals between sequential processes)

A person can choose to pipeline the tasks that they were assigned by hiring subordinates (pipeline stages) and having each one do a subtask. In general, this will improve the throughput for the tasks that the person was assigned.

It is possible for two people to explicitly synchronize their work by communicating via a synchronization protocol such as a barrier (synchronization).

It is possible to restart the work of some or all the people by raising a reset flag (HW resets).

Assume:

1. That the time that people take to complete their various tasks is in general variable, and similarly that the time that objects (messages) take to pass between people is in general variable.
2. That each person in general wants to complete their tasks as quickly and efficiently as possible.
3. That the factory needs to be able to make multiple types of furniture at the same time. For example, it might make chairs that need to be different colors or have different styles.

To reliably produce output, each person in the factory will need to adhere to rules like those described in this document.

Here's a sketch of a specific way the above example relates to the scheduling rules:

Person 1 sends chair seats and chair backs to Person 2. This is done with blocking operations, and there is no storage capacity between the people when the objects are sent. (This means that a Push operation cannot complete until the corresponding Pop operation is performed.) Assume that the fastest an object can be sent between people is 1 minute.

The written instructions that person 1 is given are:

```
while (1) {
 // internal processing code for seats and backs ...
 seats.Push(seat_object);
 backs.Push(back_object);
}
```

The written instructions that person 2 is given are:

```
while (1) {
 seat_object = seats.Pop();
 back_object = backs.Pop();
 // internal processing code for seats and backs ...
}
```

If both person 1 and person 2 choose to perform their tasks sequentially as written, then objects will be passed over time as:

```
            Person 1            Person 2
Minute 1:   seats.Push(seat1);
Minute 2:   backs.Push(back1);  seat1 = seats.Pop();
Minute 3:   seats.Push(seat2);  back1 = backs.Pop();
Minute 4:   backs.Push(back2);  seat2 = seats.Pop();
Minute 5:                       back2 = backs.Pop();
```

If both person 1 and person 2 choose to perform their IO operations in parallel, then objects will be passed over time as:

```
           Person 1                                Person 2
Minute 1:  seats.Push(seat1); backs.Push(back1);
Minute 2:  seats.Push(seat2); backs.Push(back2);   seat1 = seats.Pop(); back1 = backs.Pop();
Minute 3:                                          seat2 = seats.Pop(); back2 = backs.Pop();
```

If person 1 chooses to perform his IO operations in parallel, and person 2 chooses to perform his IO operations sequentially as written, then objects will be passed over time as:

```
           Person 1                                  Person 2
Minute 1:  seats.Push(seat1); backs.Push(back1);
Minute 2:                                            seat1 = seats.Pop();
Minute 3:  seats.Push(seat2); backs.Push(back2);     back1 = backs.Pop();
Minute 4:                                            seat2 = seats.Pop();
Minute 5:                                            back2 = backs.Pop();
```

If person 1 chooses to perform his IO operations sequentially as written, but person 2 chooses to perform his IO operations sequentially in the reverse order as it was written, then objects will be passed over time as:

```
            Person 1            Person 2
Minute 1:   seats.Push(seat1);
Minute 2:   backs.Push(back1);  back1 = backs.Pop();
Minute 3:                       seat1 = seats.Pop();
```

In this case the person 1 seats.Push(seat1) operation at minute 1 will not complete at the start of minute 2. This means that the person 1 backs.Push(back1) operation will never start, and thus the Person 2 back1 backs.Pop() operation will never complete. So, the system will be in deadlock.

This example directly corresponds to the ordering rules related to message-passing operations in the *basic conceptual model* within this document, and in particular the specific rule that disallows reversing the order of message-passing operations.

Appendix B – Formal Presentation of Pipelined Loops

Modeling convention for pipelined-loop overlap.

The post-HLS pipelined-loop implementation may overlap iterations and may internally accept/stage up to the finite bound B(c_in) of input values for younger overlapped iterations before older iterations' outputs are produced. This internal accept/stage behavior is part of the back-annotated channel realization counted in B(c_in) between the Σ-visible endpoints of c_in (producer Push_{c_in} and consumer Pop_{c_in}). These internal accept/stage movements are modeled as ε-steps. **Σ** records only the committed boundary transfers at the **Σ**-visible endpoints. In particular, the consumer-side boundary Pop_{c_in} is the loop-body Pop_{c_in} operation from the pre-HLS source when that Pop is included in the chosen Σ / Σ_DUT projection. The externally relevant effect of pipelining is therefore captured entirely by B(c_in) (via boundary backpressure/acceptance behavior and overlap timing at the Σ-visible boundary), without introducing any new auxiliary or duplicate Σ-visible boundary Pops: the Σ-visible consumer-side Pop_{c_in} commits are still the same loop-body Pop_{c_in} operations from the pre-HLS source, though they may commit at different times (and thus at different prefixes) than in an unpipelined execution. Once the pipeline reaches a quiescent blocked state under the automatic flush discipline below, it does not initiate new blocking request assertions for younger overlapped iterations.

Multi-input Pop points and coupler policies.

When a pipelined loop consumes from multiple input channels (e.g., c1,c2), back-annotation still assigns a separate finite bound B(ci) and cycle-boundary occupancy occ_ci(t) to each outer Σ-visible boundary channel ci. However, in the multi-input / coupler cases of Appendix Q the formal source-side proof

target is no longer merely the outer-boundary Sys_B; it is the elaborated reference model Sys_B^elab of Appendix J, Remark 2. In Sys_B^elab, B(·), occ_·(t), E4, BoundaryReq, ChannelEnabled/ChannelDisabled, and the Appendix K blocking/WFG machinery are interpreted per explicit abstract channel, including any introduced staged/bundle/join channels. Any conjunctive/join dependency (e.g., "join not met") shall therefore be represented only via such explicit staged/bundle/join boundary channels in Sys_B^elab (not as cross-channel disablement at the outer Σ-visible endpoints), so that for Appendix K the blocking point is that explicit boundary and the relevant Front_P(σ) members there are jointly channel-disabled ("ALL disabled ⇒ stall"). For the detailed elaboration patterns and examples, see Appendix Q.

Practical note (Σ_DUT projection).

The loop-body Pop_{c_in} in the pre-HLS source (i.e., the consumer-side Pop_{c_in} operation, conceptually after the in-flight capacity B(c_in)) may be exposed in Σ_DUT if desired, but is typically omitted since it is latency-insensitive, and it is often practically difficult to expose/collect in the post-HLS RTL. The liveness/deadlock reasoning still accounts for this blocking point without requiring it to be Σ_DUT-visible: Appendix K defines WFG edges using the ChannelEnabled/ChannelDisabled status of the corresponding pending boundary operation instance q with kind(q)=Pop_{c_in} (via E4/occ_c/B(c)), so no additional internal interface needs to be treated as a separate Σ-observable channel.

Automatic flush (drain-only blocking discipline).

Potential deadlocks are avoided by having the pipeline automatically flush.
Terminology (pending blocking frontier / Front_P(**σ**)). In an ε-quiescent cycle, consider the set of pending (i.e., issued but not yet committed) source-level blocking message-passing operations (Pop/Push) in the current interval between adjacent synchronization calls. By the Issue/Commit linkage (Appendix C) together with BoundaryReq's definition at the Σ-visible boundary (as given in "Common Formal Definitions for Appendices I-K"), each such pending blocking op has its endpoint-owned boundary request asserted (Push: vld, Pop: rdy) until it commits; purely combinational channel-side structure (including couplers) may affect only the complementary mirror signal (Push: rdy, Pop: vld) and thus whether the op commits in that cycle after ε-settling. Define Front_P(σ) to be the set of such pending blocking operations that are minimal with respect to source program order within that interval (if multiple minimal blocking ops are pending, they form a frontier). We refer to Front_P(σ) as the pending blocking frontier (or "minimal pending blocking frontier"). This Front_P(σ) notion is distinct from NextFront_P (Lemma S2 / Corollary J.1), which denotes the next source-level frontier-item set over Ω_P and may include synchronization-call items, signal-anchored action items, and message-operation items. A message-operation item in NextFront_P may be pending before it contributes any committed Σ-event.

A pipelined loop is said to automatically flush if, whenever the pending blocking frontier Front_P(σ) is non-empty and every op ∈ Front_P(σ) cannot make progress because it is not channel-enabled (evaluated after combinational ready/valid/backpressure has ε-settled for the cycle), then:

1. Block point. The process is blocked on message passing at that pending blocking frontier Front_P(σ), as in the unpipelined source.
2. No new later work. While blocked at that frontier (i.e., while Front_P(σ) remains non-empty and all of its members are channel-disabled), the implementation does not issue any new blocking Pop/Push operations that are later in source program order (including any younger overlapped iterations) and therefore does not begin asserting their BoundaryReq.

3. No withdrawal. While blocked, the implementation does not withdraw any already-asserted blocking Pop/Push request (no "try-and-withdraw"); outstanding blocking requests remain asserted until they commit.
4. Clarification (Frontier-minimality vs. other asserted requests). The Wait-For Graph in Appendix K is intentionally defined from the minimal pending blocking frontier Front_P(σ), i.e., the current blocking causes. Accordingly, an already-asserted blocking request that is not in Front_P(σ) (for example, a request from a younger overlapped iteration that was issued before the blocked condition, or a downstream request draining work already past the frontier) may persist as protocol state under the "no withdrawal" rule, but it is not treated as a wait-for source while an earlier pending blocking op remains frontier-minimal. Such a request contributes WFG blocking only if/when it later becomes a member of Front_P(σ).
5. Drain-only downstream progress. Work that has already passed the blocked frontier point(s) may continue to drain, and any already-asserted resulting output-Push requests may commit when channel-enabled, provided no work advances past the blocked frontier point(s). (Formally: "no work advances past" is meant in the ≤_P / issue sense—while Front_P(σ) is fully disabled, P does not issue any new blocking message operation instance y such that, for some op ∈ Front_P(σ), op ≤_P y and op ≠ y; downstream drain may only complete/commit operations whose BoundaryReq was already asserted prior to reaching this blocked condition.) (Equivalently: this is the "ALL disabled ⇒ stall" policy—P stalls only when none of the operations in Front_P(σ) are channel-enabled; downstream work may drain while Front_P(σ) is fully disabled.)
6. Synchronization-boundary withdrawal of future-epoch acceptance.
   Let s be an explicit synchronization interface call that separates the current source interval from later work. While the process is pending at s, the implementation shall not present producer-facing availability for any back-annotated capacity intended only for suff_S(s); equivalently, any asserted upstream producer request targeting such capacity shall remain channel-disabled until s commits. This does not prevent drain of work already accepted before reaching s.

Example (II=1, latency=4, automatic flush): a loop that performs one blocking Pop_{c_in} and one blocking Push_{c_out} per iteration in the pre-HLS source may, after pipelining in the post-HLS RTL, internally accept/stage up to four input values into pipeline staging (pipeline overlap) before the first output Push_{c_out} commits, subject to the back-annotated bound B(c_in) (E4). Values may then reside for several cycles in internal pipeline storage; such internal staging/hand-off is ε-state within the back-annotated realization and does not create any new auxiliary or duplicate Σ-visible Push_c/Pop_c events beyond the boundary commits at the Σ-visible endpoints (it may change only the timing of those boundary commits under overlapped execution). The drain-only blocking behavior is as defined above under "automatic flush"; in particular, if the blocked operation is in a late pipeline stage (e.g., the final Push), there may be little or no downstream work to drain.

Appendix C – Formal Definitions of Issue and Commit

Scope / shared definitions. The definitions below are with respect to the Σ-visible endpoints of the chosen source reference model Sys_ref (Appendix J, Remark 2)—i.e., the ready/valid interfaces at which Σ records message-transfer events in the raw rendezvous case Sys, the ordinary bounded-FIFO case Sys_B, or the elaborated back-annotated case Sys_B^elab. All shared notions about (i) Σ vs. ε steps, (ii) BoundaryReq / ChannelEnabled / ChannelDisabled, (iii) non-blocking polls as ε on failure, and (iv) the B(c) / occ_c(t) (cycle-boundary, non-fall-through) interpretation of E4 are as defined in "Common Formal

Definitions for Appendices I-K" and are to be interpreted over the explicit abstract channels of Sys_ref. Appendix C defines only Commit, Issue, and the Issue/Commit linkage conditions that make Issue(q) well-defined.

Definition: Commit (message channel) — as observed by an external clocked observer
Commit is the cycle in which an external clocked observer sees a transfer complete at a Σ-visible boundary endpoint: A message commits in the cycle when the observer sees both valid and ready high at the same time. The committed payload is the data value seen in that same cycle.

Definition: Issue (message channel) — as an auxiliary (scheduler/witness) timestamp
Issue(q) is an auxiliary timestamp used to state scheduling constraints (R4/E3/E5). Intuitively, it is the first cycle in which the calling process begins requesting the transfer corresponding to the source-level message operation instance q at the chosen Σ-visible boundary endpoint.
Issue/Commit linkage (well-formedness conditions). Fix a Σ-visible endpoint direction, and let req(t) denote the endpoint-owned boundary request signal in cycle t (valid for Push, ready for Pop), i.e., the process-driven request level captured by BoundaryReq at that boundary (per "Common Formal Definitions for Appendices I-K"). For each source-level message operation instance q on that endpoint direction:
• Boundary-request soundness (blocking requests are non-withdrawable). req(Issue(q)) = 1. If q is a blocking Push/Pop, then once issued it remains the unique outstanding request on that endpoint direction until it commits: req(t)=1 in every cycle t from Issue(q) through the commit cycle of q (inclusive), with stable payload while asserted for Push. (This is the "no deassert-before-commit" discipline used throughout the document.)
• Stable attribution for a single outstanding source-level request. If req(t)=1 and no transfer commits on that endpoint direction in cycle t, then any continued assertion req(t+1)=1 is attributed to the same outstanding source-level message-operation instance q only so long as the requesting source-level operation remains that same outstanding instance. For blocking Push/Pop, this is the mandatory non-withdrawable case above, so attribution cannot change before commit. For non-blocking PushNB/PopNB, continuous assertion of the endpoint request signal, by itself, does not identify later executed non-blocking call instances as the same q; a new executed source-level non-blocking call creates a new dynamic instance $q' \in Q\_exec,P$ for the owning process P, even if req does not deassert between cycles.
• Back-to-back issuance under continuously asserted req. If q_o commits in cycle t and the next source-level operation q_1 on the same endpoint direction is issued back-to-back, then $Issue(q_1)=t+1$ (even if the request signal does not deassert between commits), provided q_o and q_1 lie within the same source interval between adjacent synchronization calls.
• Sync-boundary discipline. If $q_o \in pref\_S(s)$ and $q_1 \in suff\_S(s)$ for a synchronization call s, then $Issue(q_1)$ is strictly after s's commit in the same trace (i.e., $Issue(q_1) > clk\_pre(s)$ in τ_pre and $Issue(q_1) > clk\_post(s)$ in τ_post). Any boundary request assertion that persists while the process is still pending at s is attributed to the outstanding operation(s) in pref_S(s) and does not constitute issuance of any operation in suff_S(s).

For non-blocking message passing (PushNB/PopNB), a failed poll produces no Σ-event and is modeled as an ε-step, while a successful poll contributes the corresponding committed Push_c(v) / Pop_c(v) event; see “Non-blocking message-passing” in “Common Formal Definitions for Appendices I-K” and Appendix I.2. Each executed non-blocking call is a distinct dynamic source-level message-call instance q ∈ Q_exec,P for the owning process P. Thus, if repeated PopNB/PushNB polls occur in later loop iterations or at later source-level call instances, they are distinct q’s even if the endpoint request remains continuously asserted and no intervening commit occurs. Stable attribution therefore applies to a single outstanding source-level request instance; it does not collapse distinct non-blocking dynamic call instances across iterations into one q. Successful non-blocking calls that commit are also represented in the frontier-item universe through Q_Ω,P when their committed transfer is relevant to Ω_P; failed non-blocking polls remain only in Q_exec,P and contribute no Ω_P item.

Appendix D – Modeling Latency-Sensitive Protocols

In some cases, designs or testbenches may use protocols which are latency-sensitive. These situations can be handled by isolating the latency-sensitive portions to small, self-contained parts, and then keeping the rest of the design and testbench latency insensitive.

Consider a case where a signal-level protocol is latency-sensitive. The protocol will have specific timing behavior that it must meet. To handle this, we create a transactor that has two sides: the first side interacts with the signals and handles the detailed timing requirements, and the second side sends and receives messages with the rest of the system. The message-passing side is latency insensitive.

To properly model the detailed timing behavior, the transactor is modeled at the cycle-accurate level in SystemC. There is an example of such a transactor model in the following document and example:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/examples/53_transactor_modeling/transactor_modeling.pdf

Appendix E – One-Way Handshake Protocols

In some systems a one-way signal handshake is sufficient for reliable system operation. When using a one-way handshake, the implicit assumption is that the “missing” handshake isn’t required since it is always true.

For example, in time-domain signal processing hardware designs, signal processing HW blocks may input new samples at a fixed rate. The HW blocks are always ready to receive new samples on each clock, but they need to know if the samples are valid. These types of designs can be modeled with the one-way handshake dat/vld protocol demonstrated in this example:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master/matchlib_examples/examples/32_dat_vld

Appendix F – Scheduling Rules: Modeling Guidelines Summary

1) Prefer to use Connections::In/Out + SyncChannel over signal IO and wait().

2) Prefer to use Pop()/Push() over PopNB()/PushNB().

3) When pipelining a loop, prefer to use `hls_stall_mode flush`.

When using signal IO:

4) If modeling a cycle-accurate process, use `disable_spawn` and follow the example 53_transactor_modeling style and do not use Push/Pop in the process.

5) If modeling a *direct input*, use `hls_direct_input` and possibly `hls_direct_input_sync`, and only change the signal at allowed times.

6) If combining signal IO with Connections::In/Out in the same process, use proper signal IO handshake that does not rely on In/Out ports for process synchronization. Place signal IO operations very close to their wait() statements.

7) Unless you are modeling a cycle-accurate process, you should expect that latency will change between the pre-HLS and post-HLS models.

Appendix G – Equivalence Rules

This document informally states that a primary goal is to present "no surprises" to a DV engineer using the same testbench to verify the pre-HLS and post-HLS models.

Somewhat more formally, we can show how the scheduling rules within this document establish a set of equivalence rules between the pre-HLS and post-HLS models that provide strong guarantees about their behaviors.

The *basic conceptual model* establishes the base behavior for a single process:

1) Synchronization interface calls always remain in source code order.
2) Signal reads occur at closest preceding synchronization interface call.
3) Signal writes occur at closest succeeding synchronization interface call.
4) All message-passing operations before a call to a synchronization interface shall be issued before or in the same cycle in which the synchronization interface commits. (For blocking message-passing operations (Push/Pop), this implies the corresponding commits occur no later than the synchronization call's commit cycle.)
5) All message-passing operations after a call to a synchronization interface shall be issued after the cycle in which the synchronization interface call commits.
6) Two message-passing operations on separate interfaces which appear in sequence in the model may be issued either in the same sequence or in parallel in simulation and in synthesis, but they shall not be issued in the reverse sequence. Operations on the same message-passing interface are issued in source order (they are never reversed).
7) Synchronization calls can affect the scheduling of signal IO operations, and synchronization calls can affect the scheduling of message-passing calls, but message-passing calls cannot affect the scheduling of signal IO operations and vice-versa.
8) The STRICT_IO_SCHEDULING=relaxed option is not used.

9) We distinguish the rendezvous source model Sys (B(c)=0 for all channels) from the buffered source interpretation Sys_B, which uses bounded-FIFO channel capacities B(c) matching the post-HLS RTL (Appendices I–K).

Conceptually, systems are composed of many such processes which follow the basic conceptual model, and which communicate using only synchronization interface calls, latency-insensitive signal-level protocols, and committed message-passing transfers. If a design uses non-blocking message-passing calls (PushNB/PopNB), then unsuccessful polls are treated as internal ε-steps (no Σ-event), and each successful completion is represented in Σ as the corresponding committed Push_c/Pop_c transfer (Σ records transfers, not failed polls). If the resulting latency-sensitive behavior is externally visible at the DUT boundary, the snooping wrapper of Appendix L may be used to align the selected admissible pre-HLS execution with the RTL. Here we call this system of processes the *conceptual system*.

Practically, the modeling approach described above is too restrictive for real-world systems with optimized HW, so the scheduling rules in this document allow for key HW optimizations. But this is done in a carefully controlled manner, such that the HW optimizations do not break equivalent behavior with the *conceptual system*.

Here is a summary of the HW optimizations that the scheduling rules allow, and a brief description of how we maintain equivalent behavior with the *conceptual system*:

1. Pipelined loops: In the post-HLS RTL, the pipeline may overlap iterations to improve throughput. See Appendix B for a formal presentation of pipelined loops.

2. Pipelined loops: HLS by default will not break any latency-insensitive signal IO protocols when pipelining loops.

3. Direct inputs: Signals that do not change after a process comes out of reset can be read as late as possible using *hls_direct_input*, saving register area.

4. Direct inputs: When *hls_direct_input_sync* is used, signals read by a process are updated at the precise point at which the HW pipeline is guaranteed to be ramped down, so the signal values do not need to be saved within pipeline registers, saving register area.

5. Memory accesses: HLS can reorder memory accesses within a process only if it can prove that the reordering is conflict-free.

6. Shared memories: Memories shared between processes are modeled as simple C arrays but always include explicit synchronization between processes in both the pre-HLS and post-HLS models.

7. Non-blocking message-passing interfaces: They are modeled at the level of committed transfers (successful completion), with unsuccessful polls treated as ε-steps and thus not part of Σ. If latency differences in these committed-transfer events are externally visible at the DUT boundary and must be aligned for verification, equivalent behavior between pre-HLS and post-HLS systems can be maintained by snooping the post-HLS system and using this to delay pre-HLS message arrival (Appendix L).

8. Latency-sensitive global signal IO: If latency differences are externally visible to the DUT, equivalent behavior between pre-HLS and post-HLS systems can be maintained by snooping the post-HLS system and using this to delay pre-HLS signals.

9. Latency-sensitive local signal IO: Isolate local latency-sensitive protocols to small, dedicated transactors, and use latency-insensitive signal IO or message-passing to communicate with the rest of the system.

10. One-way signal handshake protocols: Only use in cases where backpressure is not possible and embed assertions into the model to check that this is always true.

11. Combinational processes: If the pre-HLS model contains combinational processes, their observable signal Read/Write behavior is interpreted at each cycle's ε-quiescent combinational fixed point. Under the well-formedness requirement that the combinational network is deterministic and reaches a unique fixed point, HLS combinational synthesis preserves the fixed-point signal relation, so combinational signal IO can be included in Σ without using synchronization anchors.

Taken as a whole, these rules support a verification methodology in which full-system verification is performed primarily on the pre-HLS system model, and post-HLS RTL verification can be reduced to targeted checks that the formal side conditions are met. The precise guarantee is the execution-level guarantee of Appendix J: every RTL_B observable behavior under the fixed stimulus has a matching Sys_ref behavior (Post → B), while the reverse direction applies only to RTL-consistent source-side prefixes or to source-side executions selected by the Appendix L snooping / witness-selection mechanism. Thus a source-side pass justifies the corresponding RTL_B behavior only within that RTL-consistent / witness-selected subset. For liveness/deadlock guarantees (Appendices I–K), the results additionally assume weak fairness and channel-progress obligations for the scheduler/environment (Appendix N); the cycle-by-cycle R/E rules alone do not imply those progress properties.

Formalization of the Equivalence Rules in Appendix G

The following notation is used in this section.

Symbol Meaning

P	A process (thread or method) in the design
Σ	The alphabet of observable IO actions
τ_P	A (finite or infinite) per-cycle trace of observable actions executed by process P: for each global clock cycle t, τ_P[t] is the (possibly empty) set of Σ-actions committed by P at t. Actions committed in the same cycle are treated as simultaneous (unordered); only cross-cycle order (and the explicit E1–E5 constraints) is semantically significant.
S	The subset of Σ that are synchronization calls (explicit wait, SyncChannel, START_P/FINISH_P indicating process start/finish)
R	The subset of Σ that are signal reads
W	The subset of Σ that are signal writes

M	The subset of Σ that are committed message-passing transfers (i.e., committed Push_c / Pop_c events on message-passing channels, including successful non-blocking transfers). Unsuccessful non-blocking polls contribute no Σ-event and are modeled as ε-steps, and therefore are not members of M.
Q_exec	The execution-level set of dynamic source-level message-call instances executed by a process at the Σ-visible endpoints (blocking Push/Pop, and non-blocking PushNB/PopNB calls). Elements of Q_exec may or may not commit. In particular, failed non-blocking polls are elements of Q_exec, have no committed Σ-event, and are modeled as ε-steps. The frontier-item universe Ω_P, defined below, is separate from Q_exec.

A *system trace* is the tuple
τ = (τ_P)_P, one component per process.

For any action a ∈ Σ, let clk_pre(a) / clk_post(a) be the clock cycle at which a is committed.
For any issued message-call instance q ∈ Q_exec, issue_pre(q) / issue_post(q) denotes the auxiliary Issue timestamp for q in the pre-HLS / post-HLS trace, respectively, as defined in Appendix C and required to satisfy the associated Issue/Commit linkage (well-formedness) conditions with respect to the Σ-visible boundary request signal on q's endpoint direction.
If q commits, it contributes a unique committed transfer m(q) ∈ M with commit cycle clk_pre(m(q)) / clk_post(m(q)). If q does not commit (e.g., an unsuccessful non-blocking call), then m(q) is undefined / absent and no Σ-event is added to M.
(Q_exec-matching / witness convention.) Because unsuccessful non-blocking polls are ε-steps and produce no Σ-event, Q_exec is not in general reconstructible from the Σ-projection alone. Whenever E3, E5, or any later proof compares executed message-call instances across Sys_ref and RTL_B, the comparison is made relative to the chosen execution witness. That witness supplies a partial matching μ_Q between dynamic source-level message-call instances in the compared traces, preserving process identity, endpoint/interface identity, dynamic source-order position, and source-level call-instance identity where applicable. If q commits, then μ_Q(q) commits the corresponding matched transfer m(μ_Q(q)) with the same Σ-label/value under the event-matching convention. If q is an unsuccessful non-blocking poll, then q has no m(q) and no Σ-label; its presence and its match are supplied only by the witness-compatibility condition WC, by the sufficient NB-CF condition when applicable, or by the Appendix L snooping / witness-selection construction. Thus E3/E5 quantify over Q_exec only relative to this chosen witness; they do not require failed non-blocking polls to be inferable from Σ alone.

(No deassert-before-commit assumption.) For blocking message requests q ∈ Q_exec, once q is issued, its Σ-visible boundary request is not withdrawable: it remains asserted in every cycle from issue_pre(q) (resp. issue_post(q)) through q's commit cycle (if it commits), and for Push the payload remains stable while the request is outstanding.

Dynamic source-order convention. The order relation used below is over dynamic source-level operation/frontier items in the compared execution / witness, not over all syntactic call sites in the source text.

For a process P, let Q_exec,P denote the execution-level set of dynamic source-level message-call instances of P at the chosen abstract boundary. Q_exec,P includes blocking Push/Pop calls, successful non-blocking PushNB/PopNB calls, and failed non-blocking PushNB/PopNB polls.

Separately, let Ω_P denote the witness-specific dynamic universe of source-level frontier items that are actually reached under the fixed stimulus / selected witness and that are relevant to observable/frontier/deadlock reasoning. Items in Ω_P include synchronization-call instances, anchored signal Read/Write action instances, and message-operation frontier items at the chosen abstract boundaries. A message-operation frontier item is either (i) a blocking Push/Pop instance that has been reached, whether pending or committed, or (ii) a successful non-blocking PushNB/PopNB instance that contributes a committed Push_c(v) / Pop_c(v) event. A failed non-blocking poll q is an element of Q_exec,P, has no committed Σ-event m(q), and is not an element of Ω_P merely because it executed.

Let Q_Ω,P := Q_exec,P ∩ Ω_P denote the message-operation slice of the frontier-item universe. Let Σ_P denote the corresponding observable labels/events contributed by items of Ω_P when they are Σ-visible. For a message-call instance q ∈ Q_exec,P, the committed Σ-event m(q) ∈ M exists only if q commits; if q is a failed non-blocking poll, m(q) is absent and q contributes no Σ-event. For a blocking message-operation item q ∈ Q_Ω,P, q may be pending before it commits; during that pending interval q is in Ω_P and Q_Ω,P, but m(q) is absent.

Operations on untaken branches and unexecuted loop iterations are not members of Ω_P, Q_Ω,P, or Q_exec,P. Distinct loop iterations, unrolled instances, and repeated calls are represented by distinct dynamic instances.

For dynamic items x and y of process P, write x <_src y, or equivalently x <_P y when the process must be explicit, to mean that P's source control flow and the R-rules force x to precede y in that dynamic execution / witness. The relation <_src is used both on Ω_P for frontier reasoning and on Q_exec,P for E3/E5 issue-order reasoning. Same-cycle issue or commit is still allowed where the R/E rules allow equality of issue timestamps or same-cycle buckets; <_src identifies the source-induced precedence constraint, not a total simulator schedule over all syntactic statements. The relation ≤_P used for frontier reasoning is the reflexive closure of the restriction of this source-induced order to Ω_P.

For sequential-process signal IO, <_src / <_P is interpreted through the R2 anchor semantics rather than through the raw lexical statement location of the signal read or write. Thus a signal Read is ordered, for formal frontier and trace-equivalence purposes, at its pred_S anchor, and a signal Write is ordered, for formal frontier and trace-equivalence purposes, at its succ_S anchor. A lexically earlier signal Write whose succ_S anchor is a later synchronization call does not, merely by lexical placement, precede or block intervening message-operation instances in the same source interval; conversely, a lexically later signal Read whose pred_S anchor is an earlier synchronization call does not, merely by lexical placement, wait behind intervening message-operation instances. Signal IO may constrain message IO only through the explicit synchronization anchors and the stated R/E rules, not by raw lexical adjacency inside an interval.

---

1. Conceptual Model for a Single Process
   For every process P the pre-HLS trace τ_P must satisfy the following *partial-order constraints* over these dynamic operation instances:

R1 (Program order of S).
∀ s1, s2 ∈ S : s1 <_src s2 ⇒ clk_pre(s1) < clk_pre(s2)

R2 (Location of R/W for sequential processes).
For a sequential process P, ordinary signal Read/Write events are anchored to synchronization events as follows:

$\forall\, r \in R\_P : clk\_pre(r) = clk\_pre(pred\_S(r))$
$\forall\, w \in W\_P : clk\_pre(w) = clk\_pre(succ\_S(w))$

where pred_S(r) (resp. succ_S(w)) is the closest dynamic synchronization-call instance that precedes (resp. follows) the signal Read/Write on the dynamic source path of the current execution / witness, after applying the anchoring convention above. This anchoring determines the signal event's formal position for $\leq\_P$ / $<\_P$, Done_P, Remain_P, and NextFront_P reasoning. In particular, for a sequential-process signal Write w, the fact that the write statement appears lexically before a message operation q in the same synchronization interval does not imply $w <\_P q$ when w is anchored to the succeeding synchronization call; the write's formal observable event is placed at succ_S(w). Likewise, a signal Read anchored to pred_S(r) is not delayed by intervening message operations merely because the read statement appears lexically after them.

Every observable signal Read/Write of a sequential process must have a real bounding synchronization anchor on the dynamic path on which it occurs. A process-start synchronization event START_P may serve as the initial predecessor anchor when the modeling convention explicitly includes START_P in S. A terminating process event FINISH_P may serve as the terminal successor anchor when the process actually terminates and FINISH_P occurs. If no real predecessor anchor exists for a Read, or no real successor anchor exists for a Write on the dynamic path being modeled, the sequential-process signal IO is ill-formed for this formal model or lies outside the finite observable prefix being compared. No observable Read/Write event is assigned clock time ∞.

R2C (Combinational signal IO at the ε-fixed point).
A combinational process P has no process-owned synchronization events and no message-passing events in this formal clause; it may, however, contribute observable signal Read(sig,val) and Write(sig,val) events to Σ. These events are not anchored by pred_S/succ_S. Instead, for each clock cycle t, the global state first reaches the cycle's ε-quiescent combinational fixed point after all sequential clock-boundary updates and all internal combinational propagation for that cycle have settled. Let μ_t denote this unique fixed-point valuation of the signal network.

If a combinational process P reads sig in cycle t, the corresponding Σ-event is Read(sig, μ_t(sig)) in the observable bucket β_t. If P drives sig in cycle t, the corresponding Σ-event is Write(sig, μ_t(sig)) in β_t, where μ_t(sig) is the final resolved fixed-point value of sig at the chosen observation boundary. Any delta-cycle re-evaluation, intermediate combinational value, or redundant re-drive before the fixed point is an ε-step and is not separately Σ-visible.

Combinational-process signal IO is well formed only when the relevant combinational network is deterministic and reaches a unique ε-fixed point in each cycle being compared. Pure combinational oscillations, ambiguous multiple-driver resolution, or hidden stateful behavior inside a process treated as combinational are outside this R2C model unless represented by explicit state or synchronization elsewhere in the formal model.

R3 (Isolation around S).
Let s be a dynamic synchronization-call instance of process P. Let pref_S(s) be the set of executed

dynamic message-call instances $q \in Q\_exec,P$ in the immediately preceding source interval of P whose source-induced order places q before or at the boundary controlled by s. Let $suff\_S(s)$ be the set of executed dynamic message-call instances $q \in Q\_exec,P$ in the immediately following source interval of P whose source-induced order places q after s. Then
$\forall\, q \in pref\_S(s) : issue\_pre(q) \le clk\_pre(s)$ and $\forall\, q \in suff\_S(s) : issue\_pre(q) > clk\_pre(s)$.
(For blocking message requests, if the synchronization call commits in the process trace, then any earlier blocking message requests in that dynamic source interval must have committed in some cycle ≤ that sync-commit cycle, by blocking-call control-flow semantics. Failed non-blocking polls in Q_exec,P satisfy the same issue-side synchronization-boundary rule, but they contribute no Σ-event and do not become Ω_P frontier items merely because they executed.)

R4 (Safe message issue ordering).
For every pair $q1,q2 \in Q\_exec,P$ within the same process and within the compared dynamic execution / witness,
$q1 <\_src\ q2 \Rightarrow issue\_pre(q1) \le issue\_pre(q2)$.
(For same-channel operations, this states that dynamic calls on a single interface are issued in source-induced program order. For distinct channels, it forbids issuing dynamic message-call instances in the reverse of their source-induced order. This rule does not impose ordering between syntactic operations on untaken branches or between dynamic instances that are not ordered by the source-induced partial order of the selected witness. Failed non-blocking polls are included in this issue-order universe when they are executed, but remain ε-steps with no Σ-event.)

R5 (Channel capacity semantics; Sys vs. Sys_B).
Appendix G defines the raw rendezvous source model Sys, in which every channel has effective capacity 0. Concretely: for every channel c and any clock cycle t, the number of unmatched committed Push_c actions is zero and the number of unmatched committed Pop_c actions is zero; equivalently, no Push_c shall commit unless a matching Pop_c also commits at t, and no Pop_c shall commit unless a matching Push_c also commits at t.

In Appendices I–K we additionally use the back-annotated source interpretation Sys_B: the same source processes as Sys, but interpreted under the case-split channel semantics of E4 with capacities B(c) matching RTL_B. Thus:
• B(c)=0 is interpreted as the rendezvous case of E4: matching Push_c(v) and Pop_c(v) endpoint events commit in the same cycle, and no unmatched committed endpoint event may remain after any cycle boundary.
• B(c)>0 is interpreted as the cycle-boundary, non-fall-through bounded-FIFO case of E4, using the occupancy predicate occ_c(t) and the FIFO availability predicates occ_c(t)<B(c) for Push_c and occ_c(t)>0 for Pop_c.

The source conceptual semantics for a process is thus a labeled partial order $(\Sigma, \le\_P)$ generated by R1–R4 plus the applicable E4 channel semantics: raw rendezvous for Sys, or the case-split back-annotated channel semantics of Sys_B / Sys_ref.

---

2. Equivalence Relation

Let τ_pre and τ_post be the Σ-traces (finite or infinite) of the same process before and after HLS, where each Σ-event is labeled exactly as in "Common Formal Definitions for Appendices I–K" (e.g., Push_c(v), Pop_c(v), Read(sig,val), Write(sig,val), and synchronization events).

Convention (S): all references to S, pred_S/succ_S, pref_S/suff_S, and <_src in E1–E5 use the dynamic source-induced order defined in R1–R4 for the compared execution / witness. Thus τ_pre denotes the Σ-trace of the pre-HLS source over the dynamic operation instances reached under the fixed stimulus / selected witness.

Σ-event matching convention. We use a canonical per-endpoint occurrence matching to make "the same Σ-events" precise. For each channel c, match committed Push_c and Pop_c events by their ordinal occurrence on that channel (e.g., the k-th committed Push on c in τ_pre matches the k-th committed Push on c in τ_post; likewise for Pop_c). For each such matched pair, the endpoint (c) and the payload/value label must agree. For the ordinal index k to be well-defined under the per-cycle "set/bucket" trace representation used in these appendices, we assume that for any fixed channel endpoint c, at most one committed Push_c and at most one committed Pop_c may occur in any single clock cycle.

For each observable signal sig, match Read(sig,·) and Write(sig,·) by their ordinal occurrence on that signal within the process trace, again requiring the value labels to agree.

For sequential processes, we apply the following canonicalization for observable signals: within each anchor interval between consecutive synchronization events in the order S, we record at most one Σ-visible Read(sig,val) and at most one Σ-visible Write(sig,val) for each (process, sig), with the anchor cycle given by E2. Any additional physical sampling of sig by the implementation, or redundant re-driving of sig within the same anchor interval, is treated as an internal ε-step and must not change the recorded value label. If a design intentionally relies on intra-interval changes of sig, or on distinguishing multiple reads/writes of sig within the same anchor interval, then sig must be modeled using an explicit synchronized interface rather than as an ordinary sequential-process observable signal.

For combinational processes, the canonical signal event for each (process, sig, cycle) is taken at the cycle's ε-quiescent combinational fixed point as defined by R2C. Intermediate delta-cycle reads/writes and repeated re-evaluations before the fixed point are ε-steps. The matched pre-HLS and post-HLS events must agree on process, signal, direction Read/Write, cycle bucket, and fixed-point value label.

For synchronization events, match by their per-process occurrence index in the source order (R2). Independent Σ-events on different endpoints (including distinct message-passing interfaces) may commute and/or be grouped differently within a clock cycle as permitted by the R-rules and E3. Here "independent" is meant in the Σ-equivalence sense: the pair is not additionally ordered by any explicit Σ-visible ordering/timestamp constraint in E1–E5 (notably E3/E5); it is not a statement about (in)comparability under the Appendix G happens-before relation (→). Accordingly, τ_pre and τ_post are not required to be identical as a single total order, nor to agree on same-cycle "grouping" of distinct-interface Push/Pop operations, beyond the explicit ordering/timestamp constraints in E1–E5 below.

We say $\tau\_pre \approx \tau\_post$ iff the two traces match on Σ in this sense, and all the following hold:

E1 (Per-process synchronization-order preservation).
For every process P,
$\forall\, s_1, s_2 \in S\_P : s_1 <\_P s_2 \Rightarrow clk\_post(s_1) < clk\_post(s_2)$.

Equivalently, synchronization events are matched and ordered by their per-process source occurrence index. E1 does not preserve incidental global clock ordering between synchronization events of different

processes unless that ordering is induced by an explicit inter-process synchronization relation already represented in Σ.

E2 (Signal-visibility preservation).
Partition signal Read/Write events by process kind:
• R_seq and W_seq are the ordinary signal Read/Write events of sequential processes.
• R_comb and W_comb are the signal Read/Write events of combinational processes, interpreted by R2C.

For sequential-process signal IO:
∀ r ∈ R_seq : clk_post(r) = clk_post(pred_S(r))
∀ w ∈ W_seq : clk_post(w) = clk_post(succ_S(w))

Equivalently, ordinary signal Reads/Writes of sequential processes are logically timestamped at their synchronization anchors, as in R2. The same anchoring convention is used on the pre-HLS side when forming the matched trace.

For combinational-process signal IO, pred_S/succ_S anchoring does not apply. Instead, for each observable cycle bucket β_t, the matched pre-HLS and post-HLS traces contain the corresponding combinational Read/Write events at the cycle's ε-quiescent combinational fixed point, as defined by R2C. Any delta-cycle re-evaluation, intermediate combinational value, or redundant re-drive before the fixed point is an ε-step and is not separately Σ-visible.

E3 (Safe message issue ordering). (Post-HLS preservation of R4).
For every pair of executed message-call instances q1,q2 ∈ Q_exec,P of the same process P:
(i) Same-interface source order: if q1 and q2 access the same message-passing interface and q1 <_src q2, then issue_post(q1) ≤ issue_post(q2).
(ii) Distinct-interface no-reverse: if q1 and q2 access distinct message-passing interfaces and appear in sequence in the source (q1 <_src q2), then issue_post(q1) ≤ issue_post(q2) (i.e., they shall not be issued in the reverse sequence).
(Note: any pipelined-loop internal staging/dequeue discussed in "Pipelined Loops" is ε-microstate within the back-annotated channel realization and is not a Σ-visible boundary issue event in Q_exec,P nor a committed-transfer event in M; therefore it does not constitute a counterexample to (ii).)

E4 (Per-channel channel semantics: rendezvous and bounded FIFO).
For every observable channel c with capacity B(c), the projection of τ_post to events on c is legal for that channel's capacity model. The channel model splits into two cases.

• Rendezvous case, B(c)=0. Channel c is a capacity-zero rendezvous channel. A committed transfer on c consists of a matching same-cycle pair Push_c(v) and Pop_c(v) with the same payload value v. No unmatched committed Push_c or Pop_c may exist after any cycle boundary. Equivalently, the k-th committed Push_c(v) and the k-th committed Pop_c(v) occur in the same clock cycle and carry the same value. A Push_c endpoint can commit in cycle t only when the matching Pop_c endpoint boundary request is also asserted in cycle t, and symmetrically a Pop_c endpoint can commit in cycle t only when the matching Push_c endpoint boundary request is also asserted in cycle t.

• Buffered case, B(c)>0. Channel c is a bounded FIFO with cycle-boundary, non-fall-through semantics. The committed Pop_c(v) events return exactly the FIFO-ordered sequence of values v previously

committed by Push_c(v), with no drops, duplications, or reordering. For every cycle-aligned prefix π_t of the c-projection—i.e., π_t contains exactly the events on c committed in cycles < t—letting push(π_t) and pop(π_t) be the number of Push_c and Pop_c events in π_t, we have

$0 \le \text{push}(\pi\_t) - \text{pop}(\pi\_t) \le B(c)$.

Equivalently, the abstract occupancy after cycle t−1 is occ_c(t) = push(π_t) − pop(π_t). Availability for commits in cycle t is evaluated against occ_c(t) at the start of cycle t: Pop_c may commit in cycle t only if occ_c(t) > 0, Push_c may commit in cycle t only if occ_c(t) < B(c), and simultaneous Push_c and Pop_c commits in the same cycle are permitted iff 0 < occ_c(t) < B(c).

Thus the inequality-based FIFO availability predicates are used only for B(c)>0. The B(c)=0 case is not interpreted by substituting B(c)=0 into the bounded-FIFO inequalities; it is interpreted by the rendezvous rule above.

Note: occ_c(t) is the logical occupancy of channel c at the chosen Σ boundary, induced solely by the Σ-visible boundary-commit history (Push_c/Pop_c) and the annotated capacity B(c) in the buffered case B(c)>0. For B(c)=0, the corresponding channel fact is the same-cycle rendezvous request/commit predicate rather than a nonzero FIFO occupancy predicate.

E5 (Messages cannot cross their immediate synchronization boundary).
For every synchronization call s (in the source order used by R3 / pref_S / suff_S over Q_exec,P):
$\forall\, q \in \text{pref\_S}(s) : \text{issue\_post}(q) \le \text{clk\_post}(s)$
$\forall\, q \in \text{suff\_S}(s) : \text{issue\_post}(q) > \text{clk\_post}(s)$
(For blocking message requests, if the synchronization call commits in the process trace, then any earlier blocking message requests in that source interval must have committed in some cycle ≤ that sync-commit cycle, by blocking-call control-flow semantics. Under the Issue/Commit linkage above, any boundary request assertion that persists while the process is still pending at s reflects only already-issued outstanding operation(s) in pref_S(s); no operation $q \in \text{suff\_S}(s)$ may have Issue(q) ≤ clk_post(s), and any back-to-back issuance on that endpoint direction may occur only in the first cycle strictly after clk_post(s). For non-blocking calls, a later executed source-level PushNB/PopNB in suff_S(s) is a distinct q and cannot be identified with an earlier pref_S(s) call solely because the endpoint request signal remained asserted.)

See *Appendix I – Per Process Trace Equivalence Proof* for formal proof of the following one-way witness statement:

For any single source-level process P that obeys the conceptual R rules and B rules, when interpreted as part of the prescribed source reference model Sys_ref (see Appendix J, Remark 2), every finite observable trace/prefix of the post-HLS RTL_B process has a matching source-level trace/prefix in Sys_ref that satisfies E1–E5 (Post → B). Appendix I does not claim the unrestricted converse for all source-side traces. When a reverse correspondence is needed, it is restricted to RTL-consistent source-side prefixes as stated later in Appendix J / Theorem J.1. The raw rendezvous case is recovered when Sys_ref = Sys_B and all effective capacities are zero.

---

3. Allowed Transformations and Why They Preserve ≈

Transformation permitted by the rules	Formal justification
Loop pipelining (overlaps iterations)	Introduces internal pipeline stages (modeled as ε-steps) and overlaps iterations. R1–R5 (and hence E1–E5) are applied to the explicit S set. Loop pipelining may commute independent Σ-actions across iterations (e.g., across distinct channels)—where "independent" here means "not additionally ordered by any explicit Σ-visible constraint in E1–E5 between the pair (notably E3/E5)," rather than "incomparable under Appendix G's happens-before relation (→)"—but it must still preserve: (i) per-channel FIFO legality (E4), (ii) the required issue discipline (E3) for source-ordered pairs, and (iii) the sync-boundary constraints (E1/E5) with respect to the (explicit + implicit) S-actions.
Overlap of internal dequeue/staging vs. later pipeline work (**ε**)	Permitted because the internal staging/acceptance activity is ε-internal within the back-annotated channel realization (as defined in "Appendix B - Pipelined Loops") and does not constitute a Σ-visible boundary message-issue reorder. Σ-visible boundary message issues must still satisfy E3, and per-channel committed-transfer legality is ensured by E4; automatic flush / Appendix K addresses deadlock preservation for overlap timing.
#pragma hls_direct_input (late sampling of static signals)	This pragma imposes a designer/environment stability contract: the signal's value must remain stable after reset (or after its last permitted update) while the receiving process may consume it. The logical signal Read actions r remain anchored exactly as in E2 (clk_post(r) = clk_post(pred_S(r))). HLS may implement this by sampling the signal later (instead of inserting internal storage), but because the signal is stable the sampled value equals the value at pred_S(r); therefore E2 and ≈ are preserved.
#pragma hls_direct_input_sync (controlled updates)	This pragma imposes an explicit timing contract on the environment: the controlled direct-input signals may change only on the cycle of the designated synchronization call s* (i.e., during the sync handshake). With that contract, the logical anchoring required by E2 is preserved: for the receiving process, the relevant Reads are anchored at the closest preceding synchronization call pred_S(r) = s*, and any environment update Write w_update is aligned with synchronization (succ_S(w_update) = s*). HLS may still implement late sampling internally, but the value observed is the same as the value at the E2 anchor point, so ≈ is preserved without modifying E2. Instrumentation note (logical signal timestamping). In the Σ-traces used for ≈ checking, sequential-process signal Read/Write events are timestamped at their E2 anchor

Transformation permitted by the rules	Formal justification
	points (e.g., clk_post(pred_S(r)) for Reads), not at any later physical RTL sample cycle that HLS may introduce. Any internal late-sampling micro-steps are treated as ε-steps. Combinational-process signal Read/Write events are instead timestamped in the observable cycle bucket β_t corresponding to the cycle's ε-quiescent combinational fixed point, as defined by R2C. Intermediate delta-cycle reads/writes, redundant re-evaluations, and transient combinational values before that fixed point are ε-steps and are not separately Σ-visible. Therefore, equivalence checking must instrument/record the appropriate logical Read/Write event: the anchored event for sequential-process signal IO, or the fixed-point bucket event for combinational-process signal IO. A wrapper/transactor may be used when needed to expose only those logical events rather than naive physical sample or delta-cycle events.
Memory-access reordering proven conflict-free	Let addr(a) be the symbolic address of access a. If the tool proves $addr(a_1) \neq addr(a_2)$ for the overlapped window, then the commutation of a_1 and a_2 is observationally silent; no external signal/message depends on the internal order.
Implicit FSM states / added latency	Extra states introduce *silent* ε-steps between observable actions; the partial order on Σ is unchanged, so E1–E5 are unaffected.
Shared-memory arrays with explicit synchronization	When a memory is accessed by multiple processes, the designer explicitly coordinates access with synchronization operations modeled as S-actions, and the externally relevant memory behavior is defined by the array access mapping layer. E1 preserves the order of the synchronization actions that bracket or authorize the shared-memory accesses. The array access mapping layer then defines the mapped memory operations, their commit cycles, and the required serialization/coherence rule for the shared storage, and the pre-HLS and post-HLS traces must agree under that declared memory semantics. E2 is not used here except for ordinary signal IO. E4 is relevant only if the mapping layer represents memory commands/responses as message-passing channels; otherwise memory legality is discharged by the mapping layer's memory semantics, not by FIFO channel semantics. Under these conditions, the shared-memory transformation preserves the observable behavior required by ≈.

Transformation permitted by the rules	Formal justification
Non-blocking message-passing (PushNB/PopNB) verified by post-HLS snooping	If the latency differences are externally visible to the DUT, a verification wrapper delays the pre-HLS side so that the committed transfer events (successful completions) occur in the same order/timing as observed on the post-HLS channel. This wrapper is itself purely synchronising (S), so it cannot violate R1–R5. Once the wrapper is assumed, τ_pre and τ_post coincide on the committed message-transfer history recorded in Σ (i.e., the Push_c/Pop_c commit events), so E3–E5 are preserved. Unsuccessful non-blocking polls remain ε-steps and are not required to match. This is not a transformation that inherently preserves ≈, but rather a verification technique to enforce equivalence for inherently latency-sensitive operations.
Latency-sensitive *global* signal IO matched by snooping	If the latency differences are externally visible to the DUT, the same "mirror-and-delay" wrapper used for NB channels is applied to any globally-visible signal whose timing matters. Because the wrapper inserts only S-actions, the partial-order constraints are unchanged. This is not a transformation that inherently preserves ≈, but rather a verification technique to enforce equivalence for inherently latency-sensitive operations.
Latency-sensitive *local* signal IO isolated in cycle-accurate transactors	Local timing-exact protocols are confined to dedicated transactor processes that expose only latency-insensitive M or S operations to the rest of the design. Since the transactor boundary is now the observable interface, R1–R5 apply *outside* the timing-sensitive region, so system-level ≈ still holds.
One-way signal-handshake protocols with run-time assertions	For single-direction vld or rdy signals, an assertion proves that back-pressure can *never* occur. That assertion establishes a refinement in which the missing handshake is equivalent to a permanent logical '1', so the protocol is observationally identical to a two-way handshake that trivially satisfies E2.
Combinational processes	Combinational processes have no process-owned S actions and no message-passing M actions in this formal clause, but they may have observable signal Read/Write events in Σ. Their signal IO is interpreted at each cycle's ε-quiescent combinational fixed point as specified by R2C: intermediate delta-cycle propagation is ε, and only the fixed-point Read/Write labels are Σ-visible. Under the well-formedness requirement that the combinational network is deterministic and reaches a unique fixed point in each compared cycle, HLS combinational synthesis preserves these fixed-point signal relations. Thus equivalence is not vacuous; it is fixed-

Transformation permitted by the rules	Formal justification
	point equality of the combinational signal Read/Write events in the corresponding cycle buckets.

For the purposes of the formal analysis, non-blocking message-passing is modeled at the level of committed transfers: a successful PushNB/PopNB contributes the same committed Push_c/Pop_c event to Σ as a blocking Push/Pop, while unsuccessful polls are ε-steps.

If a design contains latency-sensitive global signals or non-blocking transfers whose timing/ordering is externally visible at the DUT boundary and must be reproduced exactly in both simulations, we take as an axiom that a purely synchronizing snooping wrapper can be applied to supply the same latency choices to the pre-HLS simulation as those observed in the post-HLS RTL (Appendix L). Conceptually, this wrapper is a witness-selection device: it does not enlarge the set of admissible pre-HLS behaviors, but restricts the pre-HLS run to one admissible execution whose Σ-events align with the observed RTL execution. Under this wrapper, the committed Σ-traces of the two simulations agree, and the equivalence rules E1–E5 apply to the wrapped runs.

Appendix L provides detailed guidance to enable this perfect alignment to be achieved in practice.

---

4. Proof Structure

The proofs proceed in three stages:
First, in Appendix I we establish the per-process witness correspondence needed by the later system-level proof: under a live environment, every post-HLS RTL_B process trace (equivalently, every reachable finite observable prefix) has a matching source-level trace in the prescribed source reference model Sys_ref (Post → B), satisfying E1–E5. Here Sys_ref is the source-side proof target defined later in Appendix J, Remark 2: in the ordinary bounded-FIFO case, Sys_ref = Sys_B; in the elaborated back-annotated cases, Sys_ref = Sys_B^elab. The converse direction (B → Post) is not claimed for all Sys_ref traces in general; when it is needed, it is restricted to RTL-consistent prefixes (e.g., via Appendix L's snooping / witness-selection technique). Here, live environment does not mean assuming the entire system is deadlock-free; it refers specifically to the standard conditions of weak fairness, finite pipeline depth, and the local progress invariants (e.g., P_push(c),P_pop(c)). These are scheduling and channel-usage side-conditions, not the global liveness property that will be proved later.

Second, in Appendix J we compose these per-process results into a system-level equivalence theorem. This stage relies on channel-ordering and occupancy lemmas but does not assume system-level liveness.

Finally, Appendix K discharges the earlier "live environment" assumption by proving that system-level liveness is preserved through HLS. Specifically, if the prescribed source reference model Sys_ref (with capacities B(c) interpreted at its chosen abstract boundaries and matching RTL_B) is live under weak fairness, then RTL_B is also live. In the ordinary case Sys_ref = Sys_B; in the elaborated back-annotated cases Sys_ref = Sys_B^elab. (When Sys_ref = Sys_B and B(c)=0 for all channels, this specializes to the raw rendezvous model.) This step ensures that the overall proof is non-circular: the liveness property invoked in the first stage is rigorously established only at the end.

---

5. Causal Dependency Between Processes

To prove system-level equivalence, we must distinguish between incidental timing and functionally significant ordering.

Formal Definition of Causal Dependency

To formalize the notion of functionally significant ordering and resolve ambiguity, we define the happens-before relation, denoted by ->, on the set of all observable IO actions (Σ) across all processes in the system. The relation is causal (information-flow) rather than purely temporal: events related by → may still commit in the same clock cycle (e.g., rendezvous), but the direction of → indicates which event can be depended on by the other. This relation establishes a strict partial order of events, where a -> b is read as "a happens before b".

The *happens-before* relation is the smallest relation satisfying the conditions below:

1. Intra-Process Order: If actions a and b occur within the same process P and a precedes b in the program's execution trace, then a -> b. This directly reflects the sequential nature of the code within a single thread. Clarification (non-temporal). Here “precedes” refers to the order of IO action *instances* in the process’s trace/program order (i.e., the order induced by the process’s execution), not the relative order of their commit cycles; distinct-interface message actions may be issued in order while committing in either order depending on backpressure.

2. Inter-Process Communication: The relation is established by the direct exchange of information or synchronization between processes.

    - Message-passing (committed transfer): If a is the commit of a send on channel c in process P1 (a committed Push_c, including a successful PushNB), and b is the commit of the corresponding receive on c in process P2 (a committed Pop_c, including a successful PopNB), for the same logical message instance, then a → b. Here “corresponding receive” is defined by the channel’s FIFO semantics: b is the Pop_c commit that consumes the element produced by a (matching is by FIFO position / transfer instance, not by payload-value equality; duplicate payload values do not introduce ambiguity). Equivalently, if we enumerate committed Pushes on c as Push_c^1, Push_c^2, … in commit order and committed Pops as Pop_c^1, Pop_c^2, … in commit order, then Push_c^k → Pop_c^k. For rendezvous channels (i.e., B(c)=0), the matching Push_c and Pop_c commits occur in the same clock cycle; nevertheless we still include the directed edge a → b to reflect unidirectional message transfer (the Pop observes the value supplied by the Push) and to allow system-level causal chaining via transitivity. This edge does not assert an earlier commit cycle—only causal precedence—and no reverse edge (b → a) is implied unless there is an explicit reverse-direction communication (e.g., another channel or a signal handshake). Unsuccessful non-blocking polls are ε-steps and do not participate in →.

    - Signal Handshake: If a is a signal write action w(sig) in P1 and b is a signal read action r(sig) in P2 that reads the value written by a as part of an explicit handshake protocol, then a -> b. A complete two-way handshake (e.g., vld/rdy) creates a causal chain, such as w(vld)@P1 -> r(vld)@P2 -> w(rdy)@P2 -> r(rdy)@P1.

    - Explicit Synchronization (two-party SyncChannel / ac_sync): Let *sout* be the commit of a SyncChannel sync_out (or equivalent ac_sync call) in process P1, and let *sin* be

the commit of the matching sync_in in process P2 for the same synchronization instance. Then this synchronization acts as a two-process barrier:

- Every observable action in P1 that occurs before *sout* (in P1's trace/program order) satisfies a→b for every observable action b in P2 that occurs after *sin*.
- Symmetrically, every observable action in P2 that occurs before *sin* satisfies a→b for every observable action b in P1 that occurs after *sout*.
- (The two sync commits *sout* and *sin* are treated as simultaneous; we do not impose *sout*→*sin* or *sin*→*sout*.)

3. Transitivity: The relation is transitive. If a -> b and b -> c, then a -> c.

Using this relation, we now provide a formal definition for causal dependency between synchronization events, which are the fundamental ordering anchors in the scheduling model.

Definition: Causal Dependency
A synchronization event s2 in process P2 is causally dependent on a synchronization event s1 in process P1 if and only if s1 -> s2.
If neither s1 -> s2 nor s2 -> s1 holds true, the events s1 and s2 are considered causally independent (or concurrent). Any change in the observed temporal ordering of causally independent events between the pre-HLS and post-HLS simulations is considered an incidental, functionally insignificant variation in latency. A well-formed design, under this methodology, shall not rely on a specific ordering of causally independent events for functional correctness.

To be clear, the shall not rely requirement is a design rule. To adhere to this design rule, designs must use message passing, SyncChannel operations, and signal handshakes to properly order operations across the system. Designs which violate this design rule are outside of the formal guarantees.

---

6. System-Level Theorem

Theorem (Execution-Level / Trace-Set Compositional Equivalence).
Fix an initial state and a fixed external stimulus/environment behavior for both the prescribed source reference model Sys_ref and RTL_B, with Sys_ref as defined in Appendix J, Remark 2. (The raw rendezvous model Sys is the special case obtained when Sys_ref = Sys_B and all effective capacities are zero.)

Interpret "fixed stimulus" per Appendix I.2, "Fixed-stimulus interpretation (reactive environments)." Let Exec_post denote the set of finite execution prefixes of RTL_B that are reachable under this stimulus and that are prefixes of at least one infinite execution under the same stimulus satisfying the stated fairness/progress premises; let Exec_B denote the corresponding set of finite execution prefixes of Sys_ref.

Assume the Appendix I / Appendix L witness machinery is available as follows: the per-process Post → B witness construction of Appendix I, including the witness-compatibility premise WC for ε-modeled non-blocking polls (and any other ε-only internal choices that can affect the next Σ-visible control flow/frontier), with NB-CF as a sufficient condition that makes the witness trivial; together with the capacities/occupancies/enablement semantics of Sys_ref, finite capacities on every explicit abstract channel of Sys_ref, the channel progress invariants P_push(c) and P_pop(c), and weak fairness (WF). Assume also B1 (Deterministic Trace Property for RTL_B under the fixed stimulus). Use B4/B5 when ε-normalization or uniqueness up to finite ε-stutter is invoked.

Then the main system-level result is:

(Post → B existence) For every $\tau\_post \in Exec\_post$, there exists $\tau\_B \in Exec\_B$ such that $\tau\_B,system \approx \tau\_post,system$ under E1–E5.

(B → Post existence, RTL-consistent prefixes only) For every $\tau\_B \in Exec\_B$ whose Σ-projection matches a prefix of the unique RTL_B Σ-trace under the same fixed stimulus, there exists $\tau\_post \in Exec\_post$ such that $\tau\_B,system \approx \tau\_post,system$.

Moreover, because B1 holds, the matching Σ-label sequence is unique (up to insertion/removal of finite ε-steps permitted by B4/B5), although the corresponding source-side witness prefix/state in Sys_ref need not be unique as a full micro-state.

Proof sketch. Appendix J first proves a pairwise composition lemma: any chosen compatible pair of system traces whose per-process projections satisfy ≈ lifts to system-level ≈. Appendix J then instantiates that lemma at the execution-set level using Appendix I's witness construction together with the Exec_post / Exec_B scope conditions. Thus the main result is explicitly about sets of executions under fixed stimulus, not merely about a single pre-chosen trace pair.

---

7. Practical Implication ("No Surprises" Guarantee)

If a verification environment exercises only the alphabet Σ (signals, message ports, synchronization calls) and does not test internal latency, then—under the assumptions of Theorem J.1 in Appendix J (fixed stimulus, the stated fairness/progress premises, and the Appendix I / Appendix L witness conditions where needed)—the prescribed source reference model Sys_ref and the post-HLS RTL agree at the chosen observable boundary in the following qualified execution-level sense:

- for every $\tau\_post \in Exec\_post$, there exists a matching $\tau\_ref \in Exec\_B$; and
- in the reverse direction, only those source-side $\tau\_ref \in Exec\_B$ that are RTL-consistent (i.e. whose Σ-projection matches a prefix/execution admitted by the unique RTL_B Σ-trace under the same fixed stimulus, when B1 holds) are guaranteed to have a matching $\tau\_post \in Exec\_post$.

Accordingly, the precise "no surprises" reading is one-sided for arbitrary RTL behavior and restricted on the source side: no Σ-observable behavior can appear in RTL_B under the fixed stimulus without a matching behavior in Sys_ref; but a source-side pass claim transfers to RTL_B only for the RTL-consistent subset of source-side behaviors covered by Theorem J.1 (or after the Appendix L snooping / witness-selection restriction has been applied where needed).

Thus, for a Σ-only testbench under the same fixed stimulus, any mismatch found on RTL_B corresponds to a matching mismatch on Sys_ref; however, a source-side pass justifies RTL_B only through the theorem's restricted reverse direction. Here Sys_ref means Sys_B in the ordinary bounded-FIFO case and Sys_B^elab in the elaborated back-annotated cases of Appendix B / Appendix Q. See Appendix J / Theorem J.1 for the full execution-level statement.

Appendix G states the equivalence rules and the high-level implication. Appendix I provides the per-process Post → B witness construction, and Appendix J makes the main system theorem explicit at the execution-set level for the prescribed source reference model. Taken together, these appendices establish the stated witness-based execution-level correspondence at the chosen observable interfaces under fixed stimulus, the stated fairness/progress premises, and the Appendix I / Appendix L witness

conditions where needed. In particular, the unrestricted guarantee is Post → B; the reverse direction ranges only over RTL-consistent source-side behaviors covered by Theorem J.1.

Appendix H – Possible Criticisms

This section highlights several potential challenges in the verification approach of Appendix G.

1. Designer-Managed Shared-Memory Synchronization

Appendix G requires that any memory shared between processes use explicit synchronization primitives inserted by the designer. The HLS tool does not perform static verification of those primitives. In practice, we mitigate this risk by encapsulating shared memories within parameterized library classes (for example, see examples 12_ping_pong_mem and 15_native_ram_fifo) that are pre-verified for correct memory access coordination in both the pre-HLS and post-HLS models. Typically such classes will be parameterized for aspects such as element type and memory size.

2. Latency-Sensitive Signal Alignment ("Snooping")

See Appendix L for detailed discussion on snooping.

3. By Default, Matchlib Connections Model a Skidbuffer inside In<> Ports

(Note that this overall document and specifically Appendix G are not intended to be strictly tied to Matchlib. Instead, this document is intended to be applicable to any pre-HLS model written in SystemC using signals, message-passing channels, and synchronization calls.)

Matchlib Connections supports throughput accurate modeling in the pre-HLS simulation. Currently the throughput accurate modeling mechanism relies on the Pre() and the Post() methods using a 1-place buffer to store transactions that are in flight on Connections::In<> ports. This means that by default In<> ports function like a skidbuffer. Connections::Out<> ports do not introduce any additional storage capacity into the pre-HLS simulation.

By default, Catapult adds skidbuffers on input ports into the post-HLS design, so the formal equivalence relationship described in this document still holds. In other words, the default pre-HLS and post-HLS designs both still model rendezvous semantics, since both explicitly include a structural instantiation of a skidbuffer on input ports. In effect, we define the rendezvous boundary *before* the skidbuffer.

It is possible to remove some or all the skidbuffers during Catapult HLS. Because the abstract rendezvous boundary is defined before the skidbuffer, this choice is orthogonal to the equivalence rules (R1–R5/E1–E5) as long as skidbuffer-internal behavior is not included in Σ. However, to keep the pre-HLS simulation's *port micro-architecture* aligned with the RTL configuration, the following compile flag must then be used in this case:

```
-DFORCE_AUTO_PORT=Connections::SYN_PORT
```

This will remove all the skidbuffers from the pre-HLS model. In this case, if some of the skidbuffers are still inserted during HLS, the capacity effects of those specific skidbuffers should be added into Sys_B so that the formal equivalence relationship still holds.

In this case the recommended methodology is to use both approaches:

- Use the default throughput accurate mode in Matchlib for most analysis and debug, and for pre-HLS performance verification.

- To keep the pre-HLS simulation's *port micro-architecture* strictly aligned with the RTL configuration, then use -DFORCE_AUTO_PORT=Connections::SYN_PORT, and rerun final verification tests.

Common Formal Definitions for Appendices I-K

Throughout Appendices I–K, the formal comparison is between RTL_B and Sys_ref (Appendix J, Remark 2), i.e., the same source processes interpreted under the channel semantics of E4 with capacities B(c) matching the RTL. The E4 channel semantics are case-split: B(c)=0 is interpreted as a capacity-zero rendezvous channel, and B(c)>0 is interpreted as a cycle-boundary, non-fall-through bounded FIFO. The rendezvous case is therefore recovered as the B(c)=0 case of the channel model, not by applying the B(c)>0 FIFO-availability inequalities with B(c)=0.

Symbol / Rule	Definition — concise but complete
BoundaryReq(q, **σ**)	For any dynamic source-level message-operation instance q ∈ Q_exec,P on an explicit abstract channel c of Sys_ref, with kind(q) ∈ {Push_c, Pop_c}, BoundaryReq(q,σ) means: at the chosen abstract boundary and ε-quiescent cutpoint σ, q's endpoint-owned boundary request is asserted. Operationally (ready/valid): · for kind(q)=Push_c: the producer endpoint asserts vld_c, with stable payload as required; and · for kind(q)=Pop_c: the consumer endpoint asserts rdy_c. BoundaryReq is therefore a predicate about the endpoint's boundary-visible request assertion for a specific dynamic operation instance q, not about an already-committed Σ-event label. Purely combinational realization structure on the channel side may affect only the complementary mirror signal at that boundary (Push: rdy; Pop: vld) and thus the commit condition after ε-settling, but it does not change the endpoint request assertion captured by BoundaryReq(q,σ). When a cycle-index notation is needed, BoundaryReq(q,t) is shorthand for BoundaryReq(q,σ_t), where σ_t is the ε-quiescent cutpoint for cycle t.
ChannelEnabled(q, **σ**)	ChannelEnabled(q,σ) means: BoundaryReq(q,σ) holds, and in the ε-quiescent ready/valid fixed point at σ, the channel-side commit-enabling condition for q holds at the chosen boundary. Concretely, for bounded-FIFO channels B(c)>0, the channel-side condition is exactly the E4 availability predicate at that boundary:

Symbol / Rule	Definition — concise but complete
	· for kind(q)=Push_c: occ_c(σ) < B(c), i.e. space is available; and · for kind(q)=Pop_c: occ_c(σ) > 0, i.e. data is available. For rendezvous channels B(c)=0, the channel-side condition is the matching complementary endpoint participation in the same ε-quiescent cycle. If q commits, it contributes the corresponding committed Σ-event m(q) ∈ M; before q commits, m(q) is absent, but BoundaryReq(q,σ) and ChannelEnabled(q,σ) are still well-defined.
ChannelDisabled(q, **σ**)	ChannelDisabled(q,σ) means: BoundaryReq(q,σ) holds but ChannelEnabled(q,σ) does not. Intuitively: the endpoint is requesting the transfer for dynamic operation instance q, but the transfer is blocked by channel-side conditions, such as empty/full status or the rendezvous partner not simultaneously requesting, as evaluated at the ε-quiescent handshake fixed point used throughout the document's enablement tests. Consequently, at the chosen boundary the channel-side enablement test remains exactly E4. Let q ∈ Q_exec,P be a dynamic source-level message-operation instance of its owning process P on an explicit abstract channel c of Sys_ref, and let kind(q) ∈ {Push_c, Pop_c} denote its endpoint direction. BoundaryReq(q,σ) means that q's endpoint-owned boundary request is asserted at the ε-quiescent cutpoint σ, with stable payload if q is a Push. For a message-operation frontier item q ∈ Q_Ω,P on channel c, ChannelEnabled(q,σ) means that BoundaryReq(q,σ) holds and c's E4 availability predicate holds at σ after combinational settling: space/data availability for B(c)>0, or complementary endpoint participation for B(c)=0. ChannelDisabled(q,σ) means that BoundaryReq(q,σ) holds but ChannelEnabled(q,σ) does not. If q commits, it contributes the corresponding committed Σ-event m(q) ∈ M, labeled Push_c(v) or Pop_c(v); if q has not yet committed, m(q) is absent, but BoundaryReq(q,σ), ChannelEnabled(q,σ), and ChannelDisabled(q,σ) are still well-defined. Failed non-blocking polls in Q_exec,P \ Ω_P do not have BoundaryReq, ChannelEnabled, or ChannelDisabled obligations merely because they executed. Any cross-channel dependency introduced by combinational couplers must be reflected only at explicit internal staged/bundle/join boundaries introduced by the elaboration (i.e., via BoundaryReq/ChannelEnabled/ChannelDisabled on message-operation frontier items q ∈ Q_Ω,P at those explicit staged/bundle channels). At the chosen Σ-visible channel endpoints, BoundaryReq(q,σ) and ChannelEnabled(q,σ)/ChannelDisabled(q,σ) for q ∈ Q_Ω,P on channel c may not depend on other channels' state beyond the per-channel E4 predicate for c, and no extra hidden predicate may block an otherwise E4-enabled boundary transfer.
Σ	Set of observable actions for the equivalence check (committed IO events) at the chosen observable boundary (often the DUT IO boundary for functional equivalence, but optionally expanded—e.g., for Appendix K deadlock reasoning):

Symbol / Rule	Definition — concise but complete
	• channel commit events Push_c(v) and Pop_c(v) on channels designated as observable, labeled with the transferred message value (or abstract message identity) v • synchronization events wait(), Sync, START_P, FINISH_P • single-cycle signal actions Write(sig, val) and Read(sig, val) on signals designated as observable, labeled with the value written/read. Internal-only channels/signals may be excluded from Σ (treated as ε-microstate) when they are not part of the chosen observable boundary. If such channels/signals are later brought into the observable boundary (e.g., by snooping for debug or analysis), then they become Σ-events and must match under ≈. Deadlock-analysis requirement (Appendix K): for Appendix K, Σ is instantiated to include every message-passing channel whose endpoint transfers at the chosen Sys_ref boundary can contribute a dependency edge in the Appendix K Wait-For Graph, either because the channel is on the declared observable boundary or because it is snooped. The WFG itself is defined over pending dynamic blocking message-operation frontier items $q \in Q_{\Omega,P}$ for the relevant owning process P, not over already-committed Σ-event labels and not over failed non-blocking polls in $Q_{exec,P}$. Thus, when Appendix K refers to a blocked Push or Pop at a boundary, the relevant object is the pending source-level operation instance q, with kind(q) ∈ {Push_c, Pop_c}, and the edge is determined by BoundaryReq(q,σ) together with ChannelEnabled(q,σ)/ChannelDisabled(q,σ) at the ε-quiescent cutpoint σ. Internal micro-interfaces that lie within the back-annotated realization counted in B(c) (e.g., post-HLS RTL pipeline stage implementation) are not treated as separate channels for this requirement: they may be exposed/snooped for debug, but Appendix K's WFG edges are defined using the ChannelEnabled/ChannelDisabled status of the corresponding boundary operation instance q via E4/occ_c/B(c), so such internal points can participate logically in the deadlock argument without being explicitly Σ-observable. Non-blocking message-passing (PushNB/PopNB): A non-blocking call that returns "not completed" produces no Σ-event and is modeled as an internal ε-step. When a non-blocking call succeeds (returns "completed"), that success is represented in Σ as the corresponding committed Push_c(v) or Pop_c(v) event on that channel, labeled with the same transferred value/message v (i.e., Σ records transfers and their payloads, not failed polls). A non-blocking call is considered issued in the cycle in which it executes (its Issue cycle). A call that returns "completed" commits in that same cycle and therefore yields a committed transfer m(q) ∈ M; a call that returns "not completed" produces no committed transfer event (m(q) undefined / absent) and may be treated as an ε-step with respect to Σ-visible message-transfer events. Each executed non-blocking call is still a distinct dynamic source-level message-call instance $q \in Q_{exec,P}$ for the owning process P. Since failed non-blocking

Symbol / Rule	Definition — concise but complete
	q's have no Σ-label, their cross-trace matching is witness-relative: it is supplied by the chosen μ_Q / WC witness, by NB-CF when failed polls cannot affect the next Σ-visible control flow or frontier, or by Appendix L's snooping / witness-selection mechanism. Continuous assertion of an endpoint request does not by itself identify multiple executed non-blocking calls as one q, and the Σ trace alone is not required to determine how failed non-blocking q's are matched. A successful non-blocking call may also be a member of Q_Ω,P when it contributes a committed Push_c(v) / Pop_c(v) event; a failed non-blocking poll remains in Q_exec,P \ Ω_P. A successful non-blocking call may also be a member of Q_Ω,P when it contributes a committed Push_c(v) / Pop_c(v) event; a failed non-blocking poll remains in Q_exec,P \ Ω_P.
START_P	First observable action emitted by process P after reset. Marks the moment P begins executing user code. A new START_P is emitted each time P exits reset.
FINISH_P	Last observable action of process P. If P executes an unbounded loop, FINISH_P never occurs, and the trace is treated as infinite.
ε-step	An *internal*, unobservable RTL state transition (pipeline advance, FSM micro-state, etc.).
B(c)	Back-annotated effective capacity of channel c at the chosen Σ-observable Sys_B boundary (the same boundary used by E4 and Appendix K). Finite, B(c) ≥ 0. It reflects the total bounded "composite buffer" storage/credit in the RTL realization between the producer's and consumer's Σ-visible endpoints (e.g., FIFO storage, skid/elastic buffering, and any HLS-inserted pipeline staging that can hold in-flight messages). • B(c)=0 ⇒ rendezvous (capacity-zero) channel. • B(c)>0 ⇒ bounded FIFO (of that effective capacity). Point-to-point endpoint assumption (single-producer/single-consumer): For every message-passing channel c (including both buffered channels with B(c)>0 and rendezvous channels with B(c)=0), there is exactly one producer process that may execute Push_c on c, and exactly one consumer process that may execute Pop_c on c. Hence the complementary endpoint relevant to any blocked Push_c/Pop_c on c is unique. Modeling note (shared resources / arbitration): If an implementation conceptually has multiple producers and/or multiple consumers for a logical resource, that sharing must be modeled explicitly (e.g., an arbiter process plus per-client point-to-point channels), rather than as a single multi-endpoint "channel." This keeps WFG wait dependencies well-defined and unique per edge. Single-transfer-per-cycle endpoint constraint (scalar channel interface): For every message-passing channel c, in any clock cycle t, at most one Push_c may commit and at most one Pop_c may commit. Equivalently, the set of commits on a fixed channel in a single cycle contains ≤ 1 Push and ≤ 1 Pop (it may contain one of each in the same cycle). For rendezvous channels (B(c)=0), a committed transfer occurs only as a matching same-cycle Push_c(v) and Pop_c(v) pair. For buffered channels (B(c)>0), a same-cycle Push_c and

Symbol / Rule	Definition — concise but complete
	Pop_c do not imply fall-through; the channel semantics are cycle-boundary, non-fall-through as stated in E4. Reset-empty FIFO convention: after each reset boundary at which the compared execution begins or restarts, every buffered message-passing channel with B(c)>0 has logical FIFO occupancy 0 at the chosen Σ boundary. Equivalently, the reset state contains no preloaded in-flight messages in the abstract channel realization used by Sys_ref / RTL_B. If a design requires preloaded channel contents, those contents must be modeled explicitly as post-reset producer actions or by a separate initialized-state extension outside the present empty-FIFO formal model. Modeling note (multi-lane / burst transfers): If an implementation can transfer k>1 messages per cycle on a logical resource, model it as k parallel point-to-point channels (or as a single widened message transferred by one Push_c/Pop_c per cycle) so that the Σ-level event model continues to satisfy the scalar constraint above.
occ_c(t)	Abstract FIFO occupancy at the Sys_B channel boundary: number of items committed by Push_c but not yet committed by Pop_c at that boundary, measured from the most recent reset boundary under the reset-empty FIFO convention. Equivalently (E4), occ_c(t)=push(π_t)−pop(π_t), where π_t is the Σ-visible boundary-commit prefix strictly before cycle t (i.e., excluding any Push_c/Pop_c commits that occur in cycle t), so occ_c(t) is the abstract occupancy at the beginning of cycle t. For Pure-FIFO channels with B(c)>0, the FIFO availability predicates (not-full/not-empty) are evaluated against this begin-of-cycle occupancy; hence a Pop_c cannot commit in a cycle that begins empty solely because a Push_c also commits in that same cycle (no fall-through), and likewise a Push_c cannot commit in a cycle that begins full solely because a Pop_c also commits in that same cycle. Clarification (no “second Push/Pop”): movements of an item within the RTL channel realization (e.g., FIFO→staging/skid buffering, staging/skid buffering→pipeline-stage registers, pipeline-stage→pipeline-stage handoff) are internal ε-steps; they do not constitute additional Σ-visible committed Push_c/Pop_c events beyond the boundary events counted by push(π_t)/pop(π_t). Derived cutpoint shorthand (used in Appendices K and R): if σ is an ε-quiescent cutpoint occurring in cycle t_σ, write occ_c(σ) ≜ occ_c(t_σ). Likewise, when channel enablement is evaluated at an ε-quiescent cutpoint, write ChannelEnabled(q,σ) ≜ ChannelEnabled(q,t_σ) and ChannelDisabled(q,σ) ≜ ChannelDisabled(q,t_σ). This is only notational shorthand for cutpoint reasoning; the primary semantic definition of occupancy remains the cycle-boundary quantity occ_c(t). Note: occ_c(t) is an abstract analysis quantity derived from Σ-visible boundary commits; in both Sys_B and RTL_B, processes do not directly read occ_c(t) and

Symbol / Rule	Definition — concise but complete
	instead proceed solely based on the per-channel ready/valid outcome of their own Push/Pop attempts.
P_push(c)	*Finite-progress invariant (producer):* interpret "continuously enabled" as a persistent (non-withdrawable; no deassert-before-commit) ready/valid request. • Buffered case (B(c) > 0): If the producer's Push_c request remains asserted continuously from some cycle t_o onward while the channel is full (occ_c = B(c))—with stable payload while asserted—and the complementary endpoint continuously requests the matching Pop_c (persistent request asserted; Pop_c boundary request remains true), then some cycle $t' \geq t_o$ commits a complementary Pop_c (i.e., the full condition is eventually discharged). • Rendezvous case (B(c) = 0): If the producer's Push_c request remains asserted continuously from some cycle t_o onward (with stable payload while asserted) and the complementary endpoint continuously requests the matching Pop_c (persistent request asserted; Pop_c boundary request remains true), then some cycle $t' \geq t_o$ commits the rendezvous transfer—i.e., a matching Push_c(v) and Pop_c(v) commit in the same cycle.
P_pop(c)	*Finite-progress invariant (consumer):* interpret "continuously enabled" as a persistent (non-withdrawable; no deassert-before-commit) ready/valid request. • Buffered case (B(c) > 0): If the consumer's Pop_c request remains asserted continuously from some cycle t_o onward while the channel is empty (occ_c = 0)—and the complementary endpoint continuously requests the matching Push_c (persistent request asserted with stable payload; Push_c boundary request remains true)—then some cycle $t' \geq t_o$ commits a complementary Push_c (i.e., the empty condition is eventually discharged). • Rendezvous case (B(c) = 0): If the consumer's Pop_c request remains asserted continuously from some cycle t_o onward and the complementary endpoint continuously requests the matching Push_c (persistent request asserted with stable payload; Push_c boundary request remains true), then some cycle $t' \geq t_o$ commits the rendezvous transfer—i.e., a matching Push_c(v) and Pop_c(v) commit in the same cycle.
Equivalence rules (E1–E5)	*E1 Synchronization-order preservation*: For each process P, the sequence of synchronization events executed by P (wait(), SyncChannel, START_P, FINISH_P) appears in the same source order in the pre-HLS and post-HLS traces. *E2 Signal visibility:* For each sequential process P, each signal Read in P occurs at the closest preceding synchronization event in P, and each signal Write in P occurs at the closest succeeding synchronization event in P, in both the pre-HLS and post-HLS traces (per R2/E2). For each combinational process P, signal Read/Write events in P are observed at the cycle's ε-quiescent combinational fixed point, as defined by R2C. These

Symbol / Rule	Definition — concise but complete
	combinational Read/Write events are members of the same observable cycle bucket β_t as the fixed-point signal values they read or drive; they are not required to have pred_S/succ_S anchors. Interpretation (logical timestamping). For sequential processes, the equations in E2 define the logical timestamps of Read/Write Σ-events (the anchor points at which equivalence is checked). An implementation may physically sample a Read later than pred_S(r) (or physically apply a Write earlier than succ_S(w)) provided the value is stable across the anchor interval and the trace emitted for equivalence records the Σ-event at the anchor point (see the Instrumentation note in Appendix G). For combinational processes, the logical timestamp is the fixed-point observation bucket β_t; internal delta-cycle propagation before the fixed point is ε and is not separately Σ-visible. Direct-input pragmas (e.g., #pragma hls_direct_input and #pragma hls_direct_input_sync) do not change E2 and are not exceptions to E2. For sequential processes, they impose stability/timing contracts that allow later physical sampling, or controlled updates at a synchronization boundary, while the logical Read/Write Σ-events used for ≈ checking remain timestamped at the E2 synchronization anchor points and retain the anchored value labels. For combinational processes, ordinary direct-input pragmas do not introduce a separate anchoring rule; any observable combinational signal Read/Write event remains timestamped at the ε-fixed-point bucket defined by R2C. Thus the pragma affects only implementation sampling freedom under its stated contract, not the logical Σ timestamp used for equivalence. *E3 (Safe message issue ordering). (Post-HLS preservation of R4).* For any two executed message-call instances q1,q2 ∈ Q_exec,P of the same process P: (i) Same-interface order is always preserved: if q1 and q2 access the same message-passing interface/channel and q1 <_src q2, then issue_post(q1) ≤ issue_post(q2). (ii) Distinct-interface no-reverse: if q1 and q2 access distinct message-passing interfaces and appear in sequence in the source (q1 <_src q2), then issue_post(q1) ≤ issue_post(q2) (i.e., they shall not be issued in the reverse sequence). (Note: any pipelined-loop internal staging/dequeue discussed in “Pipelined Loops” is ε-microstate within the back-annotated channel realization and is not a Σ-visible boundary issue event associated with any q ∈ Q_exec,P nor a committed-transfer event in M; therefore it does not constitute a counterexample to (ii).) *E4 FIFO legality*: for each channel c, the post-HLS committed (Push_c(v), Pop_c(v)) history is a legal bounded-capacity execution consistent with Sys_B for that channel (capacity B(c)), and reduces to rendezvous consistency when B(c)=0. In particular, on each channel, the committed Pop_c(v) events return

Symbol / Rule	Definition — concise but complete
	exactly the FIFO-ordered sequence of values v previously committed by Push_c(v), with no drops, duplications, or reordering. *E5 (Messages cannot cross their immediate synchronization boundary).* For every synchronization call s (in the source order used by R3 / pref_S / suff_S): ∀ q ∈ pref_S(s) : issue_post(q) ≤ clk_post(s) ∀ q ∈ suff_S(s) : issue_post(q) > clk_post(s)

Additional Semantic Assumptions (B-rules)

The following B rules are process assumptions used within the formal proofs. These B rules are in addition to the R rules presented in Appendix G.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
B1 Deterministic Trace Property	For any fixed test stimulus and initial state, the sequence of observable actions emitted by the post-HLS design is unique, including the channel identity and payload/value of each committed Push_c(v)/Pop_c(v) and the value of each Write(sig,val)/Read(sig,val). Internal ε-steps may differ between runs, but the externally visible Σ-trace (with labels) cannot. Clarification: B1 is assumed of RTL_B (post-HLS) only. The source-level models Sys/Sys_B may admit multiple ε-/latency-different executions with the same Σ-projection. When a single concrete witness execution of Sys_B is required for simulation/debug, Appendix L's snooping wrapper may be used to select one admissible witness whose latency-sensitive events align with the unique RTL Σ-trace.	Ensures the per-process and system-level equivalence proofs (App. I & J) can match *one* post-HLS trace to *one* pre-HLS trace without branching on scheduler nondeterminism.
B2 Weak Fairness of the Scheduler (WF)	If an action's enabling predicate remains continuously true from cycle *t* onward, the scheduler must eventually select that action (within a finite, but unspecified, number of cycles). Applies to Push, Pop, and Sync operations.	Required for all progress arguments, especially the liveness lemmas in Appendix K and the channel-progress corollaries.
B3 Channel Progress Invariants	For every channel *c* with capacity B(c): • P_push(c): If a process P is stalled at a blocking $Push_{(}c_{)}$ with its boundary request true and a persistent request asserted (payload stable, if applicable) while the channel is full ($occ_{(}c_{)}$ = B(c)), and the	Explicitly assumed in Appendix J to rule out permanent back-pressure loops in which both endpoints continuously request a transfer but it never completes, which are logically

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	complementary endpoint process continuously requests the matching $Pop_{(c)}$ (persistent request asserted; $Pop_{(c)}$ boundary request remains true), then the transfer on *c* must eventually complete (in particular: for B(c)>0, some $Pop_{(c)}$ eventually commits, freeing space; for B(c)=0, the rendezvous completion eventually occurs). • P_pop(c): If a process P is stalled at a blocking $Pop_{(c)}$ with its boundary request true and a persistent request asserted while the channel is empty ($occ_{(c)} = 0$), and the complementary endpoint process continuously requests the matching $Push_{(c)}$ (persistent request asserted with stable payload; $Push_{(c)}$ boundary request remains true), then the transfer on *c* must eventually complete (in particular: for B(c)>0, some $Push_{(c)}$ eventually commits, providing data; for B(c)=0, the rendezvous completion eventually occurs).	independent of the cycle-by-cycle R-rules.
B4 Finite Stutter Bound	Every process P has a finite constant depth D_P such that whenever P is not externally stalled (i.e., P is advancing internal micro-state toward its next observable Σ-action, or its next Σ-action is locally enabled), P can execute at most D_P consecutive ε-steps before either (i) executing a Σ-action, or (ii) reaching a stable waiting state in which P has no further ε-step available until some external/peer/environment condition changes the enabling predicates.	This guarantees the ε-stutter closure needed to construct witness mappings in Appendix I and to bound internal micro-latency (pipeline/FSM bookkeeping) by ≤ D_P. Unbounded waiting due to back-pressure or missing peer stimulus is governed by WF/B2 and the progress obligations B3, not by B4.
B5 System Quiescence Closure	If, at some cycle t_o, no observable actions are enabled in any process, then within at most max_P D_P further cycles the system reaches a fixed point and executes no additional ε-steps.	Needed to finish the starvation-vs-deadlock analysis in Appendix K: ensures an all-disabled state cannot hide behind an infinite tail of unobservable activity.
B6 Time-Divergence / Idle-Stutter Convention	Even after the fixed point of B5 is reached (no further internal ε micro-steps are possible), the global clock continues to advance. We model each subsequent idle clock tick as a stutter step that produces no Σ-event and leaves the global state unchanged.	For prefix/trace matching in E1–E5 and for the Exec_post / Exec_B conventions in Appendix J, these idle ticks are treated as ε-stutter. Consequently, any reachable finite execution prefix/state (including a deadlock cutpoint) is extendable to an

ID	Basic-process property (informal statement)	Why it is needed / how it is used
		infinite execution under the same stimulus by appending idle-stutter ticks.

Appendix I – Per Process Trace Equivalence Proof

I.1 Overview

This appendix establishes only the per-process witness property in the direction needed by Appendix J's execution-level correspondence: under a fixed initial state and fixed external stimulus/environment behavior, every reachable finite observable prefix of the post-HLS model RTL_B has a matching finite observable prefix in the prescribed source reference model Sys_ref (Appendix J, Remark 2), satisfying E1–E5 (Post → B).

Downstream citation discipline. Later appendices may cite Appendix I for this Post → B witness construction over Sys_ref, and for the local per-process ordering / anchoring facts proved along the way (e.g., Lemma I.0). Appendix I does not, by itself, justify an unrestricted statement that every Sys_ref trace/prefix has a matching RTL_B trace/prefix.

Importantly, the converse direction ("every Sys_ref trace has a matching RTL_B trace") is NOT claimed in general when latency-sensitive / non-blocking effects are made Σ-visible (e.g., by observing non-blocking poll outcomes or other timing-sensitive events via an expanded observation boundary). In such cases, Sys_ref may admit additional Σ-traces (for example, extra unsuccessful poll observations, or in elaborated cases additional source-side micro-realizations with the same outer intent) that do not match the RTL_B Σ-trace under the same stimulus. When a reverse correspondence is required, Appendix L's snooping / witness-selection technique is used to restrict attention to RTL-consistent Sys_ref executions (see Theorem J.1). The rendezvous specialization is recovered when Sys_ref = Sys_B and B(c)=0.

I.2 Formal Preconditions

- Observable alphabet Σ is as defined in *Common Formal Definitions for Appendices I–K*: it consists of the event schemas {Push_c, Pop_c, Sync, wait, START_P, FINISH_P, Write, Read} instantiated only for channels/signals designated observable; additionally, all message-passing channels that can participate in Appendix K deadlock reasoning are designated observable (possibly via snooping) and hence included in Σ.
- Non-blocking message-passing interpretation. In both RTL_B and Sys_ref, a non-blocking call (PushNB / PopNB) is treated as a poll. A successful poll contributes the same observable event as the corresponding committed Push_c / Pop_c on that channel. An unsuccessful non-blocking call (returns "false" / "not completed") contributes no Σ-event and is modeled as an ε-step. A non-blocking poll may still assert a Σ-visible endpoint request in the cycle of that call. However, each executed non-blocking call is a distinct dynamic source-level instance $q \in Q_{exec,P}$ for its owning process P. Accordingly, continuous assertion of an endpoint request with no intervening commit does not by itself collapse later executed non-blocking call instances into an earlier q.

Only a poll that succeeds contributes the corresponding committed transfer m(q) ∈ M for that call instance. If a request assertion persists without a new executed source-level call instance being entered, its attribution remains with that same currently outstanding q; but repeated non-blocking polls in later loop iterations or later source-level call instances create additional q ∈ Q_exec,P even if req does not deassert.

- Silent step Any internal RTL microstate transition is written **ε**. In particular, unsuccessful non-blocking polls, arbitration bookkeeping, and pipeline advances are ε-steps.
- Finite-pipeline-depth premise (FPD) There exists a finite constant pipe_depth(P) = D_P < ∞ such that once P begins internal micro-progress toward its next observable Σ-action (or that next Σ-action is locally enabled), P executes at most D_P cycles of ε-steps before either committing a Σ-action or reaching a stable waiting state with no further ε-step available until external/peer conditions change. In particular, a process cannot perform an unbounded number of ε-only failed non-blocking polls while making no progress toward any Σ-action; designs that can spin indefinitely without reaching either a Σ-action or a stable wait state violate FPD/B4 and are outside the model.
- Weak-fairness premise (WF) If a micro-operation remains continuously enabled, the scheduler eventually issues it. This is an assumption on the execution/scheduling environment (See Appendix N), not an automatic guarantee of "clock-synchronous RTL." In practice it requires that any arbiters/back-pressure logic that can affect whether an enabled operation is selected must be fair in the sense of B2/B3.
- Finite-progress invariants (B3) For every channel c, assume producer and consumer obligations P_push(c) and P_pop(c) hold, guaranteeing that data (or space) eventually becomes available. This is an assumption on the execution/scheduling environment (e.g., fairness of arbiters/back-pressure; see Appendix N), not a consequence of the cycle-by-cycle R-rules.

- Channel enablement semantics. Let q ∈ Q_exec,P be a dynamic source-level message-operation instance of its owning process P on message-passing channel c, and let kind(q) ∈ {Push_c, Pop_c} denote its endpoint direction. If q has issued a persistent blocking request to transfer on c, then BoundaryReq(q,σ) holds at the ε-quiescent cutpoint σ: the endpoint-owned boundary request remains asserted, and the payload is stable if q is a Push. The channel-side commit-enabling predicate for q is determined as follows.

  Rendezvous case, B(c)=0. A Push_c operation instance q is channel-enabled exactly when BoundaryReq(q,σ) holds and the matching Pop_c endpoint boundary request is asserted in the same ε-quiescent cycle. A Pop_c operation instance q is channel-enabled exactly when BoundaryReq(q,σ) holds and the matching Push_c endpoint boundary request is asserted in the same ε-quiescent cycle. When both endpoint requests are asserted in that cycle, the matched Push_c(v) and Pop_c(v) operation instances commit as a same-cycle rendezvous pair and contribute the corresponding Σ-events.

  Buffered case, B(c)>0. The channel-side commit-enabling predicate for q is exactly the FIFO availability predicate conjoined with BoundaryReq(q,σ):
  for kind(q)=Push_c: ChannelEnabled(q,σ) ⇔ BoundaryReq(q,σ) ∧ occ_c(σ) < B(c) ("not full"), and
  for kind(q)=Pop_c: ChannelEnabled(q,σ) ⇔ BoundaryReq(q,σ) ∧ occ_c(σ) > 0 ("not empty").

  Modeling note (fall-through / simultaneous empty/full cases). The buffered abstraction for B(c)>0 is cycle-boundary, non-fall-through: it permits a Push and a Pop to both commit in the

same cycle when the FIFO begins the cycle neither empty nor full (0 < occ_c(t) < B(c)); in that case both commits are legal and the occupancy is unchanged. It does not permit a Pop to commit in a cycle that begins empty solely because a Push also commits in that same cycle, nor does it permit a Push to commit in a cycle that begins full solely because a Pop also commits in that same cycle. If an implementation uses a fall-through FIFO (allowing same-cycle pass-through when empty), model that behavior explicitly (e.g., as an explicit bypass/rendezvous path composed with the bounded FIFO), so that the Σ-level committed-transfer trace matches the intended cycle semantics.

Operational interpretation (ready/valid): occ_c(t) is an abstract quantity derived from Σ-visible boundary commits for B(c)>0 buffered channels; processes do not read occ_c(t) directly. For B(c)=0 rendezvous channels, the corresponding channel-side fact is the matching endpoint request predicate in the same ε-quiescent cycle. In both Sys_ref and RTL_B, whether a pending blocking Push/Pop can proceed is realized operationally by the settled per-cycle ready/valid handshake outcome. Here Sys_ref denotes the prescribed source reference model at the chosen abstract boundary: Sys_B in the ordinary case, or Sys_B^elab in the elaborated back-annotated cases. A concrete realization is correct iff, for a persistent request with BoundaryReq true, the ready/valid outcome agrees with the corresponding rendezvous or buffered availability predicate at that chosen boundary.

No additional arbitration/grant/back-pressure gating is part of the channel semantics; any cross-channel coordination in the realization must be expressed only through the settled ready/valid handshake behavior that determines ChannelEnabled/ChannelDisabled, without adding enablement conditions beyond E4. Process-local control (BoundaryReq) determines when an endpoint requests; realization logic determines whether that request is enabled (ChannelEnabled) in that cycle.
Therefore, when the boundary request is true and the relevant FIFO availability predicate holds continuously, the transfer is continuously enabled; by weak fairness (B2) the corresponding commit occurs after a finite (though not necessarily bounded) delay.

- No ill-formed signal IO. Every observable signal Read/Write has a unique real bounding synchronization operation on each dynamic path on which it occurs; otherwise, the design is ill-formed. START_P may be used as an initial anchor only when it is included as a real synchronization event in S, and FINISH_P may be used as a terminal anchor only when the process actually terminates. The formal model does not use a virtual synchronization event at ∞. (From RULE 1 and RULE 2.)
- Fixed-stimulus interpretation (reactive environments). If the “same stimulus” includes a reactive testbench and/or peer processes, it is interpreted as the same global Σ-causal behavior (same externally supplied inputs, and the same peer/environment responses to the same prior Σ-visible history), not as a function of P-local history alone. In Appendix I this interpretation is used only as a witness-replay condition: when a next RTL_B event e is already fixed in the chosen post trace prefix, any peer/environment enabling needed for e is part of that same fixed global Σ-causal behavior and is therefore replayed consistently for the matching Sys_ref construction. This is not a standalone assumption that arbitrary Sys_ref continuations are globally live.
- WC (Witness-compatible non-blocking outcomes). Because failed polls are modeled as ε (unobservable), all correspondence results in Appendices I–K are stated for witness-compatible comparisons: any ε-modeled non-blocking poll outcome (and any other ε-modeled internal choice that can affect the next Σ-visible control flow / next Σ-visible blocking frontier) is

fixed/selected to be consistent with the compared RTL_B execution prefix τ_post (e.g., via the Appendix L witness-selection/snooping wrapper). WC also supplies the μ_Q matching for ε-modeled failed non-blocking call instances when E3/E5 or the proof construction refers to q ∈ Q_exec,P; those failed q's are not inferred from Σ alone.

- Remark (NB-CF as a sufficient condition). If a design satisfies the following non-blocking control-flow property—failed PopNB/PushNB polls do not influence the next Σ-visible control flow/frontier—then the witness is trivial and WC holds automatically. Equivalently: between two consecutive Σ-events of a process, inserting/removing any finite sequence of failed PopNB/PushNB polls may change only internal ε-behavior/timing, not which Σ-visible action (Push/Pop/synchronization anchor) is next in program order.

I.3 Auxiliary Lemmas

Lemma I.0 (Message-issue discipline in RTL).
The RTL satisfies E3: (i) per-interface issue order is preserved for all message operations; and (ii) for distinct interfaces, the no-reverse rule holds.
Lemma I.1 (Bounded **ε**-chain to **Σ**-action or stable wait). Starting from any state of P, within at most D_P clock cycles P either (i) commits its next observable Σ-action, or (ii) reaches a stable waiting state with no further ε-step available until some external/peer condition changes the enabling predicates.
Proof. Direct from FPD/B4. ■
Lemma I.2 (Eventual space / data). Assume P is blocked on • Push_c with occ_c = B(c), and the complementary endpoint continuously requests the matching Pop_c (persistent request asserted; Pop_c boundary request remains true), or • Pop_c with occ_c = 0, and the complementary endpoint continuously requests the matching Push_c (persistent request asserted with stable payload; Push_c boundary request remains true). Then a complementary Pop_c (respectively Push_c) commits within finite time.
Proof. (By B3 / Appendix N.) In either case, P is stalled at a blocking channel call while its persistent request for that call remains asserted (payload stable, if applicable). By the additional premise, the complementary endpoint also continuously requests the matching operation with its boundary request remaining true. Therefore the premises of P_push/P_pop (Appendix N) hold, and the corresponding transfer must eventually complete: for Push_c at full, some complementary Pop_c eventually commits (freeing space); for Pop_c at empty, some complementary Push_c eventually commits (providing data). ■
Lemma I.DR (Deterministic replay / source-state determinacy under fixed stimulus + witness). Fix a process P, a fixed external stimulus, and a fixed witness (as in WC). Consider two executions of the same P source code that start from the same initial source-level state and are run with that same witness. If they produce the same P-local Σ-label sequence, then for every k, the source-level state of P at the ε-quiescent cutpoint after the first k P-local Σ-events is identical in the two executions.
Proof. With fixed stimulus and a fixed witness (WC), the source-level transition from one ε-quiescent cutpoint to the next is deterministic as a function of the current source-level state and the next P-local Σ-label (including any Pop return value or anchored Read value). Induction on k.

I.4 Inductive Construction for Finite Traces (Post → B)

Let
τ_post = τ_post[0..n–1] ∘ e (where |τ_post| = n + 1)
be the next postHLS prefix of RTL_B for process P (under the fixed initial state and fixed stimulus).
Assume by induction that we already have a matching Sys_ref prefix τ_B[0..n–1] satisfying E1–E5

with τ_post[0..n–1]. We extend τ_B by a finite ε-chain (written ε*) followed by an observable action e′ so that
τ_B ∘ ε* ∘ e′ matches τ_post.
Here Sys_ref is the prescribed source reference model for this process/boundary, as fixed in Appendix J, Remark 2.

Non-blocking note. For a non-blocking PopNB/PushNB call, unsuccessful polls are treated as ε-steps. This is sound for Post→B construction under the witness-compatible (WC) premise (I.2): ε-modeled poll outcomes are either (a) irrelevant to the next Σ-visible control flow/frontier (NB-CF holds), or (b) fixed/selected consistently with τ_post (e.g., via Appendix L), so they do not introduce an unmatched Σ-visible control-flow/frontier divergence. Each executed PopNB/PushNB call still constitutes its own dynamic source-level message-call instance q ∈ Q_exec,P for the owning process P; WC/NB-CF fixes the ε-modeled outcomes of those calls, but it does not identify multiple executed non-blocking call instances as one q merely because the endpoint request stays asserted across cycles. The matching of failed non-blocking q instances, when needed by E3/E5 or by the witness construction, is part of the chosen μ_Q witness and is not reconstructed from Σ. A successful non-blocking call may also be represented in Q_Ω,P when its committed transfer contributes a frontier/observable item; a failed non-blocking poll remains in Q_exec,P \ Ω_P.
Lemma I.3 (Finite **ε***-extension step for I.4).
Let τ_post = τ_post[0..n–1] ∘ e (where |τ_post| = n + 1) be the next postHLS prefix of RTL_B for process P (under the fixed initial state and fixed stimulus). Assume by induction that we already have a matching Sys_ref prefix τ_B[0..n–1] satisfying E1–E5 with τ_post[0..n–1]. Then there exists a Sys_ref extension by a finite ε-chain ε* followed by an observable action e′ such that:
(i) τ_B ∘ ε* ∘ e′ matches τ_post under the document's Σ-event matching convention; and
(ii) the extended prefix continues to satisfy E1–E5 (in particular, E3's message-operation issue-order clause holds via Lemma I.0).
Proof. By cases on the kind of Σ-event e. Lemma I.1 bounds P-local internal progress to the relevant call site by ≤ D_P ε-steps (or reaches a stable wait with no further ε-step available until external enabling changes). Once the matching operation is continuously enabled, weak fairness (B2) guarantees it is selected after a further finite (though not necessarily bounded) delay. For blocking message transfers, if temporary disablement is due only to bounded-capacity/full-or-empty conditions (occ_c = B(c) for Push_c, occ_c = 0 for Pop_c) or B(c)=0 rendezvous-style enablement, Lemma I.2 guarantees eventual space/data (given persistent complementary requests), after which the transfer is continuously enabled and the B2 argument applies. Hence ε* is finite in all cases. ■

Case note (what differs across event kinds). For Sync/wait and anchored signal I/O events, the finite ε* step is a witness-replay argument, not an independent global-liveness claim. The lemma conditions on the already chosen next RTL_B event e and, via the fixed-stimulus interpretation above, replays the same peer/environment enabling from the same global Σ-causal history in the Sys_ref witness construction. Thus Lemma I.3 does not assume that peer/environment progress follows from P-local history alone, and it does not preempt the system-level liveness/deadlock arguments of Appendices J and K. For message-passing events, the only internal source of delay is local channel availability (eventual space/data, or rendezvous counterpart), which is exactly Lemma I.2.

I.5 Extension to Infinite Traces (Post → B)

Fix a process P and a fixed external stimulus/initial state. (For reactive environments, interpret "fixed stimulus" per Appendix I.2, "Fixed-stimulus interpretation (reactive environments).")

Let ρ_post be any RTL_B execution under this stimulus that is within the intended Appendix J / Theorem J.1 scope (i.e., an execution whose finite prefixes lie in Exec_post; equivalently: an execution that is a fair/progress-satisfying infinite run, using the idle-stutter/time-divergence convention B6 when no further Σ-progress is enabled). Let τ_post,P be the Σ-observable trace of P induced by ρ_post.
Finite-**Σ**-event case. If τ_post,P contains only finitely many Σ-events $e_1 \dots e_n$ (i.e., after e_n no further Σ-event of P occurs), apply the finite-prefix construction in I.4 to obtain a Sys_ref prefix τ_B,P[0..n) whose Σ-projection matches $e_1 \dots e_n$ and whose correspondence satisfies E1–E5. Then, by B6, extend τ_B,P[0..n) to a countably infinite Sys_ref trace by appending idle clock ticks that produce no further Σ-events. This yields an infinite τ_B,P that remains Σ-equivalent to τ_post,P (viewed under the same B6 idle-stutter extension), since both sides have exactly the same finite Σ-prefix and no further Σ-events thereafter.
Countably-infinite local-**Σ**-event case. Otherwise, τ_post,P has countably many P-local Σ-events. We construct an infinite Sys_ref trace τ_B,P by an explicit inductive limit of the finite-prefix construction in I.4.
This paragraph is a per-process construction only. Its event index n ranges over the P-local Σ-events of a single process trace, after applying the document's per-process Σ-event matching convention. It is not the system-level prefix index used by Theorem J.1, where prefixes advance by causal-prefix units rather than by requiring preservation of the RTL_B cycle-bucket decomposition.
Let τ_B,P[0..0) be the empty local prefix. For each $n \geq 0$, assume we have constructed a finite Sys_ref local prefix τ_B,P[0..n) such that its P-local Σ-projection matches the first n observable P-local Σ-events of τ_post,P and the correspondence satisfies the applicable per-process parts of E1–E5. Let e_n be the (n+1)-st P-local Σ-event of τ_post,P. Apply the I.4 construction step to extend τ_B,P[0..n) by a finite ε^* segment followed by a matching observable event e′_n, obtaining τ_B,P[0..n+1) while preserving the per-process E1–E5 obligations.
Because each extension step adds a finite segment, the increasing chain of local prefixes defines a countably infinite Sys_ref process trace τ_B,P whose every finite local event prefix matches the corresponding local event prefix of τ_post,P. Therefore, $\tau_{B,P} \approx \tau_{post,P}$ at the per-process trace level.
When these per-process witnesses are composed in Appendix J, the system-level proof schedules the matched local events into some legal Sys_ref execution satisfying E1–E5, the chosen channel/synchronization semantics, and the explicitly atomic same-cycle obligations. The resulting Sys_ref execution need not use the same observable cycle-bucket decomposition as the RTL_B execution: independent events may be placed in different cycles or grouped differently whenever no E1–E5 rule, channel rule, signal-anchor rule, or explicit synchronization transaction fixes their relative timing.
Scope clarification: this inductive-limit step establishes the per-process Σ-trace witness only (local prefix/trace matching under E1–E5); it does not by itself discharge global fairness/progress obligations (B2/B3) or prove that an arbitrary global Sys_ref continuation is fair/progress-satisfying. Those execution-level existence/scope obligations are handled in Appendix J / Theorem J.1 via the Exec_post / Exec_B definitions and qualifiers. ■

Appendix J – System-Level Trace Equivalence Proof

Synchronization-pair well-formedness for ≈.
For SyncChannel/ac_sync operations with paired endpoints, the compared Sys_ref and RTL_B executions use the same dynamic synchronization-transaction pairing. That is, the two endpoint events of a matched sync_out/sync_in transaction are matched by dynamic sync-instance identity, commit as one two-party barrier transaction, and are treated as simultaneous events in the same observable cycle

bucket. This synchronization transaction induces the same cross-process before/after ordering as described in the causal-dependency definition.
Equivalently, E1 preserves the per-process source order of synchronization endpoint events, but E1 does not weaken or replace the paired-barrier semantics of SyncChannel/ac_sync. The system-level equivalence relation τ_ref,system ≈ τ_post,system is defined only over synchronization-well-formed traces in this sense: if one endpoint of a dynamic SyncChannel/ac_sync transaction appears in the trace, then the matching endpoint appears in the same observable synchronization transaction, with the same dynamic pairing, in both Sys_ref and RTL_B.

Theorem J.1 (Execution-Level / Trace-Set Compositional Equivalence)

Fix an initial state and a fixed external stimulus/environment behavior (e.g., the same testbench-driven inputs) for both the prescribed source reference model Sys_ref and RTL_B, with Sys_ref as defined in Remark 2. The raw rendezvous model Sys is the special case obtained when Sys_ref = Sys_B and all effective capacities are zero.

Clarification (reactive environments). Interpret "fixed stimulus" per Appendix I.2, "Fixed-stimulus interpretation (reactive environments)."

Let Exec_post denote the set of finite execution prefixes of RTL_B that are reachable under this stimulus and that are prefixes of at least one (infinite) execution under the same stimulus that satisfies the Appendix J fairness/progress premises; and let Exec_B denote the corresponding set of finite execution prefixes of Sys_ref (defined analogously).

Cycle-bucket and causal-prefix convention for this theorem. A system execution may be represented as a sequence of observable cycle buckets β_1 β_2 …, where β_i is the finite set of Σ-events committed in one observable clock cycle of that particular execution. Same-cycle events inside a bucket are unordered except for the explicit constraints imposed by E1–E5, the channel semantics, signal-anchor semantics, and explicit synchronization constructs.

The bucket decomposition records the clock timing of one execution; it is not, by itself, part of the cross-model equivalence obligation. The relation τ_ref,system ≈ τ_post,system does not require Sys_ref and RTL_B to use the same observable cycle-bucket decomposition. Causally independent events may be placed in different cycles or grouped into different buckets in the two executions, provided the matched executions preserve all ordering and atomicity constraints required by E1–E5 and by the explicitly modeled channel, signal, and synchronization semantics.

The prefix order used by Theorem J.1 is causal-prefix order over matched observable units, not RTL_B bucket-prefix order. An observable unit is either:
• a singleton Σ-event; or
• an explicitly atomic same-cycle transaction that the semantics requires not to be split, including a rendezvous Push_c(v)/Pop_c(v) transfer, the two endpoints of a paired SyncChannel/ac_sync transaction, and any other same-cycle transaction explicitly identified by the signal/channel semantics.

Let $\prec_J$ be the partial order generated by E1–E5, per-process source order, E4 channel/FIFO order and legality, signal-anchor constraints, paired synchronization/barrier semantics, and the causal-dependency links stated in this appendix. A theorem prefix is a finite down-closed set of observable units under $\prec_J$. Any topological enumeration of such units is only a proof device: it must not be read as adding an

observable order among independent events. There is no semantic cutpoint after only one endpoint of an explicitly atomic multi-endpoint unit has been taken; such units are split neither in Sys_ref nor in RTL_B.

Scope note (intentional). Appendix J is a correspondence result about fair/progress-satisfying infinite behaviors. We adopt the time-divergence / idle-stutter convention B6: from any reachable ε-quiescent state, the system can always be extended to an infinite execution under the same stimulus by appending idle clock ticks that produce no Σ-events. We also note the following stutter-closure/vacuity fact: the Appendix J fairness/progress premises (weak fairness B2 and channel progress invariants B3) constrain only intervals in which some relevant Σ-action/transfer is continuously enabled. Once the ε-quiescent fixed point of B5 is reached (no further internal ε micro-steps are possible and no observable action is enabled), appending idle-stutter ticks does not create any newly enabled observable actions or transfers; therefore B2/B3 remain satisfied (vacuously) on the idle suffix. Consequently, if an ε-quiescent cutpoint is a B5 fixed point, the idle-stutter extension is itself fair/progress-satisfying and the prefix lies in Exec_post / Exec_B. For ε-quiescent cutpoints that are not B5 fixed points (i.e., some observable Σ-action/transfer is enabled), membership in Exec_post / Exec_B depends on the existence of some infinite extension that satisfies B2/B3; idle-stutter alone need not suffice. The qualifier exists to exclude only prefixes that are inconsistent with the stated fairness/progress premises (i.e., prefixes that arise only under unfair scheduler/environment behavior).

Assume the Appendix I / Appendix L witness machinery is available as follows—namely: the per-process Post → B witness construction from Appendix I, including the witness-compatibility premise WC for ε-modeled non-blocking polls (and any other ε-only internal choices that can affect the next Σ-visible control flow/frontier) as stated in Appendix I.2 and enforceable via Appendix L's snooping / witness-selection technique; with NB-CF as a sufficient condition that makes the witness trivial—together with the capacities/occupancies/enablement semantics of Sys_ref, finite capacities on every explicit abstract channel of Sys_ref, channel progress invariants P_push / P_pop (B3), and weak fairness (B2), plus B1/B4/B5 when ε-normalization is needed or uniqueness is invoked. Then:

(Post → B existence) For every $\tau\_post \in Exec\_post$, there exists $\tau\_B \in Exec\_B$ such that $\tau\_B,system \approx \tau\_post,system$ under E1–E5.

(B → Post existence, RTL-consistent prefixes only) For every $\tau\_B \in Exec\_B$ whose Σ-projection matches a prefix of the RTL_B Σ-trace under the same fixed stimulus (in particular, any τ_B produced by applying Appendix L's snooping wrapper when B1 holds for RTL_B), there exists $\tau\_post \in Exec\_post$ such that $\tau\_B,system \approx \tau\_post,system$.

Moreover, when B1 holds (deterministic Σ-projected trace under fixed stimulus), the matching Σ-label sequence is unique (up to insertion/removal of finite ε-steps permitted by B4/B5). The corresponding execution prefix witness τ_B need not be unique as a stateful prefix, since Sys_ref may admit multiple ε-different realizations with the same Σ-projection. Sys_ref may also admit executions whose Σ-projection does not match RTL_B's Σ-trace when latency-sensitive / non-blocking effects are Σ-visible; such executions are outside the scope of (B → Post) unless constrained by the snooping/witness-selection technique (Appendix L).

Proof strategy. Theorem J.1 is not obtained from pairwise composition alone. Proposition J.0 below is only the conditional lifting step for a chosen compatible pair of system traces. The actual execution-level theorem is obtained by combining that conditional lifting step with the Appendix I / Appendix L witness

machinery and the Exec_post / Exec_B scope conditions, as proved later in this appendix under “Proof of Theorem J.1 (expanded execution-level witness construction).”

Proposition J.0 (Pairwise Composition Lemma)

Fix any test stimulus (environment inputs) and initial state. Let τ_ref,system be an observable trace of the prescribed source reference model Sys_ref under that stimulus, and let τ_post,system be an observable trace of RTL_B under the same stimulus. (If B1 holds, RTL_B is Σ-deterministic for the given stimulus/initial state.) Assume that for every process P, the projections τ_ref,P and τ_post,P satisfy τ_ref,P ≈ τ_post,P, where τ_ref,P is the projection of τ_ref,system onto P and τ_post,P is the projection of τ_post,system onto P.

Thus Proposition J.0 is only a conditional composition result for a chosen compatible trace pair. By itself it is not yet an execution-set theorem and does not provide unrestricted source ↔ RTL correspondence. Theorem J.1 is the later execution-level result obtained by instantiating Proposition J.0 with the witness construction and scope restrictions stated in Appendix I / Appendix L and in the Exec_post / Exec_B definitions.

Assume every explicit abstract channel c in the chosen Sys_ref / RTL_B comparison has finite capacity B(c) ≥ 0. Also assume, as a per-channel Sys_ref / RTL_B well-formedness condition, that both chosen traces satisfy the applicable E4 channel semantics at the chosen abstract boundaries. Thus, for B(c)=0 channels, committed Push_c / Pop_c events obey the same-cycle rendezvous semantics; for B(c)>0 channels, the committed Push_c / Pop_c history obeys the reset-empty, cycle-boundary, non-fall-through bounded-FIFO semantics with capacity B(c), including FIFO order, no underflow, no overflow, no duplication, and no loss. Proposition J.0 does not derive these per-channel E4 facts; it assumes them as part of the definition of the compared well-formed Sys_ref / RTL_B traces.

No weak-fairness or channel-progress premise is needed for this conditional pairwise lifting lemma: Proposition J.0 does not construct a matching trace, but only lifts already-compatible per-process trace projections and already well-formed per-channel communication histories to the aggregate system trace. The weak-fairness, progress, and witness-construction premises are used later in Theorem J.1 when proving the execution-level existence result. Then the aggregate traces are equivalent: τ_B,system ≈ τ_post,system under rules E1–E5 (given the R rules and B rules).

Notation: The order relation <_src used below is the dynamic source-induced order from Appendix G / R1–R4: it ranges over dynamic operation instances in the compared execution / witness, not over all syntactic call sites in the source text. Equivalently, when the process must be explicit, write <_P. Untaken branches and unexecuted loop iterations are not members of the witness-specific dynamic universe.

Proof

We prove each equivalence property. The per-channel E4 facts are not derived inside this proposition; they are part of the per-channel Sys_ref / RTL_B well-formedness hypothesis for the chosen traces.

E1 (Synchronization-order preservation):

Fix any process P. By the proposition hypothesis, τ_B,P ≈ τ_post,P, so the synchronization events of P occur in the same source order in both traces. Since this holds for every P, E1 holds for the system traces τ_B,system and τ_post,system.

E2 (Signal-visibility preservation):

Fix any process P. If P is a sequential process, then by the proposition hypothesis (using the E2 anchoring relation embodied in τ_B,P ≈ τ_post,P), every signal Read in P is anchored to P’s closest preceding synchronization event, and every signal Write in P is anchored to P’s closest succeeding synchronization event, in both τ_B,P and τ_post,P.

If P is a combinational process, then R2C rather than pred_S/succ_S anchoring applies: P’s observable

signal Read/Write events are the fixed-point Read/Write events in each observable cycle bucket β_t. By the combinational well-formedness condition and the equivalence hypothesis for the fixed stimulus / selected witness, the pre-HLS and post-HLS combinational networks reach corresponding unique ε-fixed-point valuations in each compared cycle bucket, so the matched Read/Write labels agree at that bucket. Since the appropriate sequential or combinational signal-visibility condition holds for every process P, E2 holds for τ_B,system and τ_post,system.

E3 (Safe message issue ordering):

Fix any process P. We must show that P's post-HLS execution satisfies E3 as stated in Appendix G: (i) for same-interface pairs q1,q2 ∈ Q_exec,P with q1 <_src q2, we have issue_post(q1) ≤ issue_post(q2); and (ii) for distinct interfaces that appear in sequence in the source (q1 <_src q2), the operations are not issued in reverse order. This is exactly the local per-process property packaged inside τ_B,P ≈ τ_post,P (and, in the execution-level use of Proposition J.0 / Theorem J.1, supplied by Appendix I / Lemma I.0). Since it holds for every P, E3 holds for the system traces τ_B,system and τ_post,system.

E4 (Per-channel channel semantics):

E4 holds by the per-channel Sys_ref / RTL_B well-formedness hypothesis of Proposition J.0. That hypothesis states that, for every explicit abstract channel c in the chosen comparison, the Sys_ref and RTL_B traces already satisfy the applicable E4 channel semantics at the chosen abstract boundary: same-cycle rendezvous semantics when B(c)=0, and reset-empty, cycle-boundary, non-fall-through bounded-FIFO semantics when B(c)>0. Thus Proposition J.0 uses E4 as an assumed channel-realization condition for the chosen traces; it does not attempt to derive E4 from the per-process compatibility relation.

E5 (Messages cannot cross syncs):

For every synchronization call s (in the source order used by R3 / pref_S / suff_S over Q_exec,P):

∀ q ∈ pref_S(s) : issue_post(q) ≤ clk_post(s)

∀ q ∈ suff_S(s) : issue_post(q) > clk_post(s)

(Note: For blocking message transfers, the "no-withdraw"/persistence semantics imply that if a blocking operation is issued in the pre-side interval before s, then it must also commit no later than s (and similarly on the post side), because the process cannot complete s until all earlier-issued blocking requests in that interval have committed. Failed non-blocking polls in Q_exec,P are constrained at the issue level by this rule but contribute no committed Σ-event when they fail.)

This property is guaranteed per-process by the proposition hypothesis on the chosen compatible pair (with Appendix I supplying the witnesses in the execution-level Post → B use case) and requires no inter-process reasoning, so it lifts directly to the system level.

Conclusion:

All five equivalence properties hold at the system level for this chosen compatible trace pair. The composition is valid because:

· Intra-process properties are preserved by the per-process compatibility hypothesis.

· Inter-process communication respects the applicable E4 channel semantics by the per-channel Sys_ref / RTL_B well-formedness hypothesis for the chosen traces.

· No additional progress or fairness argument is needed inside Proposition J.0, because this lemma does not construct or extend executions; it only composes already-given compatible projections and already well-formed per-channel communication histories. Progress, fairness, and witness existence are used later in Theorem J.1 when this pairwise lemma is instantiated at the execution-set level.

Therefore, τ_B,system ≈ τ_post,system. □

Remark 1. When Sys_ref = Sys_B and all B(c)=0, Sys_ref coincides with the rendezvous interpretation of Sys, so Proposition J.0 and Theorem J.1 both specialize to the rendezvous case. In the elaborated cases, the source-side proof target is Sys_ref = Sys_B^elab.

Remark 2 (Reference-model ladder: raw Sys, abstract Sys_B, and elaborated Sys_B^elab).
The Matchlib library provides a capacity back-annotation feature that extracts finite capacities B(c) from the RTL realization of each message-passing channel and makes them available to the source-level model. To avoid ambiguity, this document distinguishes three source-side models:

• Sys: the raw source-level rendezvous interpretation of the user-written processes, i.e., the model obtained when every effective channel capacity is zero and no additional abstract realization structure is introduced.

• Sys_B: the abstract bounded-FIFO source-level interpretation of the same user-written processes under E4, with finite capacities B(c) at the chosen abstract channel boundaries. When all B(c)=0 and no additional abstract realization structure is needed, Sys_B coincides with Sys.

• Sys_B^elab: the elaborated back-annotated source-level model used in the more complex cases discussed in Appendix B / Appendix Q, in which additional explicit abstract staged/bundle/join channels are introduced so that conjunctive/join dependencies, occupancies, and enablement are represented at explicit abstract boundaries rather than as hidden cross-channel predicates at the outer Σ-visible endpoints.

Formal proof target. Unless a statement explicitly says otherwise, the formal results of Appendices J–K target the prescribed source reference model Sys_ref, defined as follows:
(i) Sys_ref = Sys_B in the ordinary case where finite capacities B(c) at the chosen abstract boundaries suffice to capture the RTL realization; and
(ii) Sys_ref = Sys_B^elab in the elaborated cases where additional explicit abstract staged/bundle/join channels are required.
The proofs depend only on the capacities/occupancies/enablement semantics of Sys_ref at its chosen abstract boundaries, not on the concrete RTL micro-implementation inside those boundaries.

For a simple pipelined loop with a single message-passing input and output, Sys_ref is ordinarily just Sys_B: the RTL pipeline's internal staging capacity is accounted for entirely by the back-annotated effective capacity B(c) at the chosen Sys_B channel boundary. Concretely, Sys_B still has exactly one Σ-visible Push_c(v) at the producer endpoint and one Σ-visible Pop_c(v) at the consumer endpoint. With overlapped iterations, the post-HLS RTL may accept/read-ahead up to B(c) values into internal pipeline staging before older iterations produce their corresponding outputs; these accept/read-ahead transfers are modeled as internal microstate (ε) within the concrete realization contributing to B(c). The RTL may realize this staging on either side of a particular module-local "Pop interface" handshake point; that handshake point need not coincide with the chosen Σ-visible Pop endpoint. By construction, the chosen Sys_B boundary encloses all staging counted in B(c) between the Σ-visible endpoints, so movements within that enclosed realization (including stage-to-stage handoff) do not create additional Σ-visible committed Push/Pop events beyond the single boundary Push/Pop events.

In more complex cases such as HW pipelines with multiple message-passing inputs, the back-annotation mechanism may elaborate the source-side simulation into Sys_B^elab by introducing additional explicit abstract realization structure. In those cases the formal target is Sys_ref = Sys_B^elab, and B(·), occ_·(t), E4, BoundaryReq, ChannelEnabled/ChannelDisabled, and the Appendix K blocking/WFG machinery are interpreted over the explicit abstract channels of that elaborated model. See Appendix B and Appendix Q.

Remark 3 (Practical rendezvous liveness screening under determinism).
When B1 holds and the design's control flow does not branch on empty/full observations of non-blocking interfaces (i.e., it depends only on the ordered history of observable Push/Pop/synchronization actions, not on "probe" outcomes (NB-CF, Appendix I.2)), the rendezvous specialization B(c)=0 is often an effective practical screen for design-level deadlocks: it exercises the most constrained communication discipline and can expose true cyclic data-dependency deadlocks early. Because rendezvous is strictly more constrained than buffered/elaborated operation, it can also yield false positives: a deadlock observed under B(c)=0 may disappear once finite buffering or explicit elaborated staged/bundle/join structure is present. However, bounded buffering can still introduce capacity-induced deadlocks (FIFO-depth artifacts) when FIFO depths are underestimated; these deadlocks are real for that modeled capacity configuration but may disappear once capacities are increased to the intended design point. Increasing buffer capacity (or applying dynamic buffer-growth strategies) is a standard way to distinguish depth-limited deadlocks from true cyclic dependency deadlocks.
Accordingly, the formal results of Appendices J–K deliberately target the prescribed source reference model Sys_ref of Remark 2, with the actual back-annotated capacities and explicit abstract boundaries required by the chosen realization, rather than relying on rendezvous alone.

Remark 4 (When raw Sys may be used instead of the prescribed source reference model for safety-only checks).
If verification is concerned only with safety properties over the chosen DUT-boundary projection Σ (denote it Σ_DUT) (and not with deadlock/liveness), the prescribed source reference model Sys_ref of Remark 2 remains the default reference model. This is because Σ_DUT includes committed channel-transfer events, and bounded buffering / explicit staged-bundle-join structure generally changes the set/timing of those committed Push_c/Pop_c events relative to the raw rendezvous case.
Practical note (early bring-up): Before accurate capacities B(c) are available (or before back-annotation/elaboration can be used), running the rendezvous specialization Sys against the intended testbench is still useful to validate functional intent and catch gross protocol/ordering bugs early; however, it remains a screening check, not a proof about the buffered/elaborated RTL-correspondent model.
Important (soundness): When any channel or explicit abstract staged/bundle/join boundary that can influence Σ_DUT-observable behavior has positive effective capacity or nontrivial enablement structure, the raw rendezvous model Sys is often a restriction of Sys_ref / RTL_B at Σ_DUT (it can admit fewer Σ_DUT behaviors). Therefore, using Sys in place of Sys_ref is generally suitable only as a debug/screening check: a failure can be useful diagnostically, but a pass does not by itself prove Σ_DUT-safety of the buffered/elaborated implementation.
Sys may be used as a proof-equivalent substitute for Sys_ref only when Sys and Sys_ref are Σ_DUT-equivalent for the chosen Σ_DUT (i.e., their Σ_DUT-projected trace sets coincide under the intended stimulus). A simple sufficient condition is that every Σ_DUT-visible explicit abstract channel/boundary in Sys_ref has effective capacity zero and that buffering/elaboration elsewhere is Σ_DUT-opaque (it cannot enable/disable/reorder Σ_DUT-events or change when Σ_DUT-events occur). In all other cases—i.e., if any Σ_DUT-visible explicit abstract boundary has positive effective capacity, or if buffered/elaborated internal channels can affect when Σ_DUT-events occur—use the prescribed Sys_ref of Remark 2.
Formal soundness condition. Let Traces_Σ_DUT(M) denote the set of Σ_DUT-projected traces of model M under the intended fixed stimulus. Sys may replace Sys_ref for proving a universal Σ_DUT-safety property ϕ only if Traces_Σ_DUT(Sys) = Traces_Σ_DUT(Sys_ref); under that condition, Sys ⊨ ϕ iff Sys_ref ⊨ ϕ (and otherwise Sys is only a diagnostic/screening check as stated above).

Proof of Theorem J.1 (expanded execution-level witness construction)

We now prove the execution-level clauses of Theorem J.1 by explicit witness construction. We first prove (Post → B). The (B → Post) clause is the same witness-construction argument, but applied only to RTL-consistent prefixes τ_ref ∈ Exec_B of the prescribed source reference model Sys_ref, exactly as stated in Theorem J.1.

Fix τ_post ∈ Exec_post and consider a finite observable prefix of τ_post, followed, when needed, by maximal finite ε-normalization (B4/B5) to an ε-quiescent RTL_B cutpoint σ_post. The RTL_B execution may be displayed as observable cycle buckets $\beta_1\ \beta_2\ \ldots$, but those buckets are used here only to identify the timing of that RTL_B execution and to identify explicitly atomic same-cycle units. They are not the prefix index for the cross-model equivalence proof.

Let U(τ_post) be the finite set of observable units in the chosen RTL_B prefix, where a unit is either a singleton Σ-event or an explicitly atomic same-cycle transaction as defined in the cycle-bucket and causal-prefix convention above. Let $\prec_J$ be the required-order relation generated by E1–E5, E4 channel semantics, signal-anchor semantics, paired synchronization/barrier semantics, and the causal-dependency links of this appendix. Choose any topological enumeration

$\gamma_1\ \gamma_2\ \ldots\ \gamma_n$

of U(τ_post) that is consistent with $\prec_J$. This enumeration is only a proof device; it does not add an observable order among causally independent units and does not require Sys_ref to reproduce the RTL_B cycle-bucket decomposition.

For i = 0..n, let $H_i = \{\gamma_1,\ldots,\gamma_i\}$. Since the enumeration is topological, each H_i is down-closed under $\prec_J$. We construct, by induction on i, a Sys_ref prefix τ_ref[i] ending in an ε-quiescent Sys_ref state σ_ref[i], such that τ_ref[n] is the desired witness τ_ref. Here Sys_ref is as fixed in Remark 2. After each construction step, τ_ref[i] may be extended by a maximal finite ε*-normalization suffix (B4/B5) to reach σ_ref[i]; this does not change its Σ-projection.

Invariant Inv(i).

(I1) Causal-prefix matching:
τ_ref[i],system has produced exactly the matched observable units in H_i, with the same Σ-labels/values and with all $\prec_J$ constraints among those units preserved. The Sys_ref bucket decomposition used to realize H_i may differ from the RTL_B bucket decomposition, except that explicitly atomic same-cycle units must remain atomic.

(I2) Per-process replay-state agreement:
For every process P, the source-level state of P at σ_ref[i] agrees with the deterministic source-level replay state determined by the P-local projection of H_i under the fixed-stimulus and WC witness assumptions. In particular, this agreement covers every source-level variable, control predicate, payload/value expression, and request-attribution fact that can influence future Σ-visible behavior of P or any Appendix K pending-request / drain-only fact of P.

(I3) Channel-history agreement:
For every explicit abstract channel c of Sys_ref, the committed Push_c/Pop_c history induced by H_i is the same on the Sys_ref witness side and on the RTL_B witness side under the event matching

convention. Hence, for B(c)>0 channels, the FIFO occupancy/head facts computed from that history agree; for B(c)=0 channels, each committed transfer is represented only by an explicitly atomic rendezvous Push_c(v)/Pop_c(v) unit.

In particular, this agreement covers:
• any source-level variable read by BoundaryReq(·,·);
• any source-level control predicate Pred_x(·) that decides reachability/selection of the next observable action(s);
• any payload/value expression of such next action(s); and
• for blocking Push/Pop operations, the source-level Issue/Commit attribution facts needed to determine whether a dynamic operation instance q is already issued, not yet committed, and still the operation to which the asserted endpoint-owned request is attributed under Appendix C's Issue/Commit linkage.

Channel-side enablement facts are then determined at the chosen abstract boundary from this request-attribution state together with the channel-history facts preserved by Inv(i).(I3): for B(c)>0 channels, the logical occupancy/head state induced by H_i; for B(c)=0 rendezvous channels, the endpoint-request predicates determined by the replay states of the two endpoint processes.

For each process P, the RTL_B-side comparison state used in Inv(i).(I2) is the deterministic source-level replay state determined by the P-local projection of H_i under the fixed-stimulus and WC witness assumptions. This is the object governed by Lemma I.DR. The corresponding Sys_ref state σ_ref[i] is taken at an ε-quiescent cutpoint after realizing the matched causal prefix H_i, followed if needed by finite ε-normalization.

Base case (i = 0).
Let τ_ref[0] be the empty Sys_ref prefix under the fixed stimulus and σ_ref[0] its initial state. The empty set H_0 contains no observable units. By the fixed-stimulus comparison setup, the initial source-level valuations, request-attribution facts, and reset-empty channel histories agree. Hence (I1), (I2), and (I3) hold.

Inductive step on causal-prefix units (i → i+1).
Assume Inv(i). Let γ_{i+1} be the next observable unit in the chosen topological enumeration of U(τ_post). Because H_i is down-closed under $\prec_J$, every required predecessor of γ_{i+1} has already been matched in H_i. Because the enumeration is only a proof device, γ_{i+1} need not be the next event in the RTL_B cycle-bucket order, and Sys_ref is not required to place γ_{i+1} in the same clock bucket as RTL_B placed it.

Existence of a matching Sys_ref extension.
We show there exists a finite Sys_ref extension from σ_ref[i] that commits a matching unit γ′_{i+1}, followed, if needed, by finite ε-normalization to an ε-quiescent state σ_ref[i+1]:

$$\sigma\_ref[i] \xrightarrow{\varepsilon^*} \tilde{\sigma}\_ref \xrightarrow{\gamma'_{i+1}} \hat{\sigma}\_ref \xrightarrow{\varepsilon^*} \sigma\_ref[i+1].$$

Here γ′_{i+1} has the same Σ-label(s), value label(s), dynamic message-operation matching, and synchronization-transaction identity as γ_{i+1}. If γ_{i+1} is a singleton event, γ′_{i+1} is a singleton event. If γ_{i+1} is an explicitly atomic unit, γ′_{i+1} is the corresponding explicitly atomic Sys_ref unit.

For a singleton process-local non-message Σ-event in γ_{i+1}—for example an anchored Read/Write event as in E2, or a one-sided synchronization endpoint that is not part of a paired atomic transaction—the corresponding source-level operation of the relevant process P is selected from the next-frontier information determined by the P-local replay state in Inv(i).(I2). Since σ_ref[i] agrees with that replay state on the control predicates Pred_x used for next-action selection and on the value expressions, the same local predicate/value facts hold in Sys_ref. For Write events, the value label is determined by the same source-level expression evaluation; for anchored Reads, the value label is determined by the fixed stimulus plus E2 anchoring. Thus the matching Sys_ref event has the same Σ-label.

For a message-transfer event in γ_{i+1}, the matched dynamic source-level message-operation instance q is handled by a two-case classification.

Case 1: q is a not-yet-issued next-frontier message operation. Then q is selected from the message-operation slice of NextFront_P determined by the P-local replay state in Inv(i).(I2). Since σ_ref[i] agrees with that replay state on the source-level variables read by BoundaryReq(q,·), on the control predicates Pred_q used for next-action selection, and on the payload/value expressions, Sys_ref can issue the same boundary request for q with the same payload/value facts, subject to the same side-of-synchronization and same-interface ordering constraints.

Case 2: q is an already-issued pending message operation. Then q need not be a member of NextFront_P(σ_ref[i]); in particular, Appendix B's drain-only discipline allows an already-asserted non-frontier request to persist as protocol state while an earlier pending blocker remains frontier-minimal. In this case the relevant fact is not NextFront membership, but the request-attribution component of Inv(i).(I2): q has already issued, has not yet committed, remains the operation to which the asserted endpoint-owned boundary request is attributed under Appendix C's Issue/Commit linkage, and has the same stable payload facts when q is a Push. Therefore Sys_ref contains the same pending request instance q at the compared cutpoint.

For buffered channels with B(c)>0, the channel-side legality of the matching Push_c/Pop_c commit is checked against the FIFO history induced by H_i. By Inv(i).(I3), the corresponding FIFO occupancy/head facts agree in Sys_ref and in the RTL_B witness history for the same causal prefix. Therefore the matching buffered Push_c/Pop_c event has the same channel-side legality in Sys_ref as in RTL_B, subject to the same reset-empty, FIFO-order, no-underflow/no-overflow, and non-fall-through E4 rules. If a Push_c and a Pop_c occurred in the same RTL_B clock bucket but are not an explicitly atomic rendezvous unit, they may be realized in different Sys_ref cycles or in a different Sys_ref bucket grouping, provided the E4 history and all $\prec_J$ constraints are preserved.

For a rendezvous channel with B(c)=0, a committed transfer is an explicitly atomic observable unit containing the two endpoint events Push_c(v) and Pop_c(v). It is not matched by independently scheduling the two endpoints. If γ_{i+1} is such a rendezvous transfer, then the endpoint processes P and Q have the corresponding source-level boundary requests in the replay states determined by H_i. Each endpoint request may be present either because its operation is the not-yet-issued next-frontier operation just selected, or because it is an already-issued pending request preserved by the request-attribution state described above. By Inv(i).(I2), the same endpoint request predicates and payload/value facts hold in Sys_ref. Therefore the matching Push_c(v)/Pop_c(v) endpoint pair is jointly enabled in Sys_ref and commits as one atomic rendezvous unit.

For a paired SyncChannel/ac_sync transaction, γ_{i+1} is likewise an explicitly atomic unit containing the

two matched endpoint events with the same dynamic synchronization-transaction identity. By the synchronization-pair well-formedness condition for ≈ and by Inv(i).(I2), both endpoint processes reach the corresponding synchronization frontier with the same before/after source-state facts. Therefore Sys_ref commits the matching paired synchronization transaction as one atomic synchronization unit, preserving the induced cross-process ordering.

By the Appendix I.4 finite-prefix extension reasoning, together with Theorem J.1's progress premises and weak fairness, Sys_ref cannot diverge forever by taking only ε-steps while the matched next unit remains continuously enabled. Hence a finite ε* extension exists that reaches a state in which γ′_{i+1} commits. Let τ_ref[i+1] be τ_ref[i] extended by this finite ε* segment, the matching unit γ′_{i+1}, and then, if needed, a maximal finite ε*-normalization suffix to reach σ_ref[i+1] (B4/B5). Then (I1) holds for H_{i+1} by construction; (I2) follows from deterministic replay on each process's P-local projection of H_{i+1}; and (I3) follows from applying the same E4 channel-history update for the matched channel event(s). Thus Inv(i+1) is established.

Under the fixed stimulus and the WC premise, apply Lemma I.DR (Deterministic Replay) to: (a) P's execution within the chosen Sys_ref witness prefix τ_ref[i+1] reaching the ε-quiescent cutpoint σ_ref[i+1], and (b) the source-level replay determined by the P-local projection of H_{i+1} in the RTL_B witness. These two executions start from the same initial source-level state and have the same P-local Σ-label sequence under the document's event-matching convention. Therefore Lemma I.DR implies that the replay states agree for every source-level variable, control predicate, payload/value expression, and request-attribution fact relevant to future Σ-visible behavior or Appendix K pending-request / drain-only reasoning. Thus Inv(i+1).(I2) holds. Because P was arbitrary, the per-process replay-state agreement holds for all processes; and because all channel-history updates are matched under E4, Inv(i+1).(I3) holds as well.

Conclusion.
By induction on causal-prefix units, Inv(n) holds for the complete finite set U(τ_post) of observable units in the chosen RTL_B prefix. Taking τ_ref = τ_ref[n] yields the required Sys_ref witness prefix for (Post → B). The constructed τ_ref,system is equivalent to τ_post,system under E1–E5 and the explicit channel, signal-anchor, and synchronization semantics, but it need not have the same observable cycle-bucket decomposition as τ_post,system. Bucket equality is required only inside explicitly atomic units, such as rendezvous Push/Pop transfers and paired SyncChannel/ac_sync transactions.

At the terminal ε-quiescent prefix, the P-local projection of H_n is exactly the P-local Σ-label sequence of the chosen RTL_B prefix. Therefore the replay-state agreement in Inv(n).(I2) gives the state-level bridge used by Corollary J.1 when it interprets σ_post via projection and then derives frontier/guard/value correspondence at ε-quiescent cutpoints. The reverse clause (B → Post) uses the same causal-prefix construction schema, but only for RTL-consistent prefixes τ_ref ∈ Exec_B, exactly as stated in Theorem J.1. □

Corollary J.1 (State Correspondence at the End of a Matched Observable Prefix)

Purpose.
This corollary serves as the bridge between trace equivalence and state correspondence needed in Appendix K. The mapping target is the prescribed source reference model Sys_ref of Remark 2, not an unrelated raw rendezvous (B(c)=0) execution state.

Statement. Assume the hypotheses of Theorem J.1 for Sys_ref vs RTL_B under the fixed stimulus, and additionally assume B1 holds for RTL_B (unique Σ-projected trace under the fixed stimulus), plus B4 and B5 for ε-normalization. The Σ-label sequence is therefore unique (up to insertion/removal of finite ε-steps), although the corresponding source-side witness prefix/state in Sys_ref need not be unique as a full micro-state.

Let τ_post be any τ_post ∈ Exec_post (as defined in Theorem J.1). Choose one ε-normalization $\bar{\tau}$_post obtained by extending τ_post by a (possibly empty) finite suffix ε* to a terminal state $\bar{\sigma}$_post that is ε-quiescent, meaning: no further ε-step is enabled from $\bar{\sigma}$_post (so any further progress, once enabled, must proceed via an observable Σ-action rather than additional internal ε-steps). Take ε* to be a maximal finite ε-extension of τ_post; such a maximal finite extension exists under B4 (finite stutter bound). For the remainder of this corollary, interpret σ_post as this chosen ε-normalized terminal state $\bar{\sigma}$_post. No uniqueness claim about maximal ε-extensions is needed here; all conclusions below are asserted relative to the chosen ε-normalized representative together with the matched Sys_ref witness supplied by Theorem J.1.

(When referring below to "source-level variables," the "next source-level frontier-item set," and the "logical FIFO state" in σ_post, interpret those notions via the standard source-level projection/abstraction from RTL_B microstate to the corresponding source-level state as used in Appendix I; micro-architectural registers and bookkeeping are ignored by this projection.)

Let Sys_ref denote the prescribed source reference model of Remark 2. Then there exists a finite execution prefix τ_B of Sys_ref ending in some reachable state σ_B such that:
(1) The observable prefixes match: τ_B,system ≈ τ_post,system (equivalence under E1–E5).
(2) The terminal states correspond at the source level: σ_B ≡ σ_post, where "≡" means:
(Note: "≡" is intentionally a frontier/enable/value/request-attribution correspondence (clauses (a)(b)(c)(d)(e)), not full micro-state equality; internal pipeline/buffering/elaboration microstate may differ between σ_B and σ_post provided such differences are ε-steps that are WFG-inert as assumed in Appendix K. Clause (e) is included only to make explicit the pending-request correspondence needed by Appendix K's drain-only / drain-closure reasoning; it does not require equality of arbitrary RTL microstate.)
(a) For every process P, NextFront_P(σ_B) = NextFront_P(σ_post), where NextFront_P(σ) is as defined in Lemma S2 (the set of minimal remaining source-level frontier items in Ω_P with respect to P's source partial order ≤_P). Concretely, this means either:
• a (possibly multi-element) set of candidate message-operation frontier items q ∈ Q_Ω,P, with kind(q) ∈ {Push_c, Pop_c}, on distinct interfaces that are currently minimal candidates and may be issued and, if channel-enabled, committed in any order or together in one cycle, or
• an explicit synchronization-call item s, together with the same set of signal Read/Write action items that are logically anchored at s per E2.
• This frontier equality does not require the pre-HLS and post-HLS models to commit identical subsets of that message set in the same cycle; it only identifies the set of currently-minimal candidate items.
Note. NextFront_P is a candidate frontier relative to the fixed witness universe Ω_P of Lemma S2 and does not by itself assert enablement or "pending blocking" status. Appendix K reasons about pending blocking transfers using Front_P(σ); see Lemma K.0 (K.2.2) for the Front/NextFront bridge at ε-quiescent cutpoints.
(b) By Lemma S1, for every process P, and for every item x ∈ NextFront_P(σ_B) (equivalently x ∈ NextFront_P(σ_post) by clause (a)):

(i) For every frontier item x, any source-level control predicate used to decide that x is the next source-level frontier item is evaluated the same way in σ_B and σ_post. If x ∈ Q_Ω,P is a message-operation frontier item, then the evaluation of BoundaryReq(x,·) is also the same in σ_B and σ_post. If x ∉ Q_Ω,P, no BoundaryReq(x,·) predicate is defined or required. Equivalently, σ_B and σ_post agree on all source-level state variables read when evaluating the relevant guard/control predicate, and, when x ∈ Q_Ω,P, the variables read when evaluating BoundaryReq(x,·). Channel-side enablement of Push/Pop is handled separately by clauses (c) and (d). Failed non-blocking polls may be present in Q_exec,P for witness-matching purposes, but they are not frontier items and are not covered by this NextFront_P clause.
(ii) If x produces a value—i.e., x ∈ Q_Ω,P with kind(x)=Push_c for some channel c, or x is a signal Write action item (or set of such action items) anchored at the next synchronization call—then the value produced by x (payload/value expression) is the same in σ_B and σ_post (hence σ_B and σ_post agree on all source-level variables read by that expression). If x observes a value via signal Read action items anchored at the next synchronization call, those observed signal values are the same in σ_B and σ_post because the stimulus is the same and E2 anchors the reads to that same synchronization call.
(iii) If x ∈ Q_Ω,P with kind(x)=Pop_c for some channel c, then the value returned by that Pop_c operation instance is determined by the channel's logical FIFO contents after the matched observable prefix; by E4 and the matched Push/Pop history, that logical head element is the same for σ_B and σ_post.
(c) For every channel c with B(c) > 0, the abstract FIFO state in σ_B (empty/full predicate, and the logical head element when non-empty) agrees with the logical FIFO state induced by the matched Push_c/Pop_c history of (1). Physical micro-architectural realization—pipeline registers, arbitration bookkeeping, and the implementation of buffering—is abstracted away by "≡", but the logical occupancy/ordering facts induced by the matched trace are not. Equivalently, letting occ_c(t) denote the logical Σ-boundary occupancy used throughout Appendix K (and letting "empty/full" refer to this same logical occupancy), σ_B ≡ σ_post implies σ_B and σ_post agree on occ_c(t) and on the logical head element when non-empty.
In particular, internal transfers within the concrete realization (e.g., movement between pipeline/buffering stages) that do not commit a boundary transfer are treated as ε and do not change this abstract FIFO state.
(d) For each point-to-point rendezvous channel c with B(c)=0 that is included in the Appendix K internal message-passing boundary / WFG analysis, the raw endpoint-request predicates agree:
Req_prod(c,σ_B) = Req_prod(c,σ_post)
and
Req_cons(c,σ_B) = Req_cons(c,σ_post),
where Req_prod(c,σ) means that the producer-side boundary request on c is asserted at the ε-quiescent cutpoint σ, and Req_cons(c,σ) means that the consumer-side boundary request on c is asserted at σ. This is a raw boundary-request fact only; it does not assert that an asserted rendezvous request is frontier-minimal.
(e) For Appendix K drain-only reasoning, already-issued blocking request state agrees at the chosen abstract boundaries. More precisely, for every process P and every dynamic blocking Push/Pop operation instance q in P's current interval between adjacent synchronization calls:
q ∈ Pending_P(σ_B) iff q ∈ Pending_P(σ_post),
where Pending_P(σ) means that Issue(q) has occurred and Commit(q) has not yet occurred at σ, in the Appendix C source-level attribution sense. For every such common pending q:
• the endpoint-owned boundary request is attributed to q on one side iff it is attributed to q on the other side;
• BoundaryReq(q,σ_B) = BoundaryReq(q,σ_post); and
• ChannelEnabled(q,σ_B) = ChannelEnabled(q,σ_post), equivalently ChannelDisabled(q,σ_B) =

ChannelDisabled(q,σ_post).
For B(c)>0 channels, this enablement agreement follows from the common B(c) and the logical occupancy/head-state agreement of clause (c). For B(c)=0 rendezvous channels, it follows from the raw producer/consumer endpoint-request agreement of clause (d). In the elaborated case Sys_ref = Sys_B^elab, all of these Pending, BoundaryReq, ChannelEnabled, and ChannelDisabled facts are interpreted at the explicit abstract channels of Sys_B^elab.

Lemma S1 (Relevant source-level state agreement for next-frontier guards/payloads)

Purpose.
This lemma is the minimal per-process "state agreement" fact needed to justify Corollary J.1, clause (2b): matched observable prefixes determine enough of the source-level state so that, for the common next source-level frontier-item set, source-level control predicates agree, produced/observed values agree, and BoundaryReq agrees only for message-operation items $x \in Q\_\Omega,P$ in the message-operation slice of that frontier. This is the fact Appendix K needs when reasoning about BoundaryReq/blockedness on the Sys_ref side; no BoundaryReq predicate is defined or required for non-message frontier items such as synchronization-call items or signal-anchored action items.

Statement.
Fix a process P. Let τ_post,P be the projection of some reachable post-HLS (RTL_B) finite prefix onto P, and let τ_ref,P be a Sys_ref prefix such that $\tau\_ref,P \approx \tau\_post,P$ under the document's Σ-matching/E1–E5 conventions. Let σ_post be the ε-normalized (ε-quiescent) terminal state of τ_post,P, and let σ_ref be the terminal state of τ_ref,P.

Terminology (Witness-compatible; WC).
Assume this comparison $\tau\_ref,P \approx \tau\_post,P$ is witness-compatible in the sense of WC (Appendix I): any ε-modeled non-blocking poll outcome (and any other ε-modeled internal choice that can affect P's next Σ-visible control flow / next Σ-visible frontier) is fixed/selected consistently with the compared RTL_B prefix τ_post,P (e.g., via Appendix L's witness-selection/snooping wrapper).

Remark (NB-CF as a sufficient condition). If the design satisfies NB-CF (Appendix I)—failed PopNB/PushNB polls do not influence the next Σ-visible control flow/frontier—then the witness is trivial and WC holds automatically.

Let NextFront_P(σ) denote, relative to this fixed compared witness $\tau\_ref,P \approx \tau\_post,P$ (equivalently, relative to the associated Ω_P / $\leq\_P$ universe of the compared execution), P's next source-level frontier-item set: the minimal candidate set of source-level frontier items with respect to P's source partial order, as used in Corollary J.1. Let x be any item in that next frontier (so $x \in$ NextFront_P(σ_ref), and equivalently $x \in$ NextFront_P(σ_post) by the frontier-alignment lemma (S2) / Corollary J.1(2a)).

Then:

(1) Control-predicate agreement, plus boundary-request agreement for message items only.
For any source-level control predicate Pred_x that is evaluated (in the source semantics) to decide that x is the next source-level frontier item at this point (e.g., branch/loop condition determining control reachability of x), we have:
Pred_x(σ_ref) = Pred_x(σ_post).

If $x \in Q\_\Omega,P$ is a message-operation frontier item, then additionally:

BoundaryReq(x, σ_ref) = BoundaryReq(x, σ_post).

If x is not a message-operation frontier item, no BoundaryReq(x,·) predicate is defined or required. Equivalently: σ_ref and σ_post agree on the valuation of every source-level state variable read when evaluating Pred_x(·), and, when x ∈ Q_Ω,P, every source-level state variable read when evaluating BoundaryReq(x,·). Channel-side enablement of Push/Pop is not part of this local guard clause; it is handled by the FIFO/rendezvous enablement clauses in the corresponding-cutpoint relation. Failed non-blocking polls may be present in Q_exec,P for witness-matching purposes, but they are not frontier items and are not covered by this NextFront_P / BoundaryReq clause.

(2) Produced-value agreement (payload/value expressions).
If x produces a value—i.e., x ∈ Q_Ω,P with kind(x)=Push_c for some channel c, or x is a signal Write(sig,val) action item (or set of such write items) anchored at the next synchronization call—then the value produced by x is the same in σ_ref and σ_post. Equivalently: σ_ref and σ_post agree on the valuation of every source-level state variable read by x's payload/value expression.

(3) Pop return-value agreement.
If x ∈ Q_Ω,P with kind(x)=Pop_c for some channel c, then the value returned by that Pop_c operation instance is the same in σ_ref and σ_post, and is equal to the logical head element of c's abstract FIFO after the matched observable prefix (hence uniquely determined by the matched Push/Pop history under the bounded-FIFO semantics).

(4) Signal-read agreement at the next sync anchor (when applicable).
If x observes values via signal Read(sig,·) action items anchored at the next synchronization call, then those observed signal values are the same in σ_ref and σ_post under the "fixed stimulus" interpretation used by Appendix I/J together with the E2 anchoring discipline.

Proof.
We prove (1)–(4) by reducing them to a single per-process invariant: "matched Σ-prefixes fix the valuation of the source-level variables relevant to the next frontier," plus FIFO legality for Pop values.

Define the sequence of P-local observable events in the matched prefixes:
τ_post,P = e1 e2 … en
τ_ref,P = e1 e2 … en
(where equality here is equality of Σ-labels under the document's event matching convention: same channel/signal identity, and same value label for Push/Pop/Read/Write events).

For k = 0..n, let:
• σ_post[k] be the ε-quiescent post-HLS state after the first k P-local observable events.
• σ_ref[k] be the Sys_ref state after the first k P-local observable events.

(Existence of these ε-quiescent representatives is exactly the ε-normalization discipline used throughout Appendix I/J.)

Key invariant I(k).
For every source-level state variable v of process P that can influence any future Ω_P frontier item that may later contribute Σ-visible behavior of P (in particular: any v that may be read by (i) BoundaryReq(q,·) for a message-operation frontier item q ∈ Q_Ω,P, (ii) a control predicate Pred_x deciding

reachability/selection of the next frontier item(s), or (iii) a payload/value expression for the next frontier item(s)), we have:
$v(\sigma\_ref[k]) = v(\sigma\_post[k])$.

(Here $\sigma\_post[k]$ is interpreted via the "standard source-level projection/abstraction" from RTL_B microstate to source-level state used in Appendix I. In particular, this projected source-level cutpoint state is the one used for deterministic replay in Lemma I.DR: under fixed stimulus and the WC premise, it is uniquely determined by the preceding P-local Σ-label sequence.)

Deterministic replay (Lemma I.DR).
Under the fixed stimulus and the WC premise, Lemma I.DR gives source-state determinacy at ε-quiescent cutpoints: for a fixed witness, the source-level state after the first k P-local Σ-events is uniquely determined by the initial source-level state together with the P-local Σ-label prefix (including any Pop return value or anchored Read value).
Apply Lemma I.DR to the two executions induced by the compared prefixes: (a) the Sys_ref execution of P producing τ_ref,P, and (b) the source-level replay corresponding to τ_post,P under the fixed witness (WC), whose ε-quiescent cutpoints are exactly the projected states $\sigma\_post[k]$. These two executions start from the same initial source-level state (fixed-stimulus setup) and produce the same P-local Σ-label sequence e1 e2 … en (by Σ-label matching). Therefore, for every k, the source-level cutpoint states agree; in particular, every source-level state variable v covered by I(k) satisfies $v(\sigma\_ref[k]) = v(\sigma\_post[k])$. Thus I(k) holds for all k, and in particular I(n) holds.

Conclusion of the invariant.
By induction, I(n) holds at the terminal matched cutpoints $\sigma\_ref = \sigma\_ref[n]$ and $\sigma\_post = \sigma\_post[n]$.

Now discharge the lemma's four conclusions for any $x \in NextFront\_P(\sigma\_ref)$:

(1) $Pred\_x(\cdot)$, when present, is a source-level control predicate evaluated from source-level variables of P at the cutpoint. Since those variables agree by I(n), the control-predicate evaluations agree:
$Pred\_x(\sigma\_ref) = Pred\_x(\sigma\_post)$.
If $x \in Q\_\Omega,P$ is a message-operation frontier item, then $BoundaryReq(x,\cdot)$ is also a source-level predicate evaluated from source-level variables of P at the cutpoint. Since those variables agree by I(n), the boundary-request evaluations agree:
$BoundaryReq(x, \sigma\_ref) = BoundaryReq(x, \sigma\_post)$.
If $x \notin Q\_\Omega,P$, no $BoundaryReq(x,\cdot)$ predicate is defined or required, so there is no BoundaryReq conclusion to prove for that frontier item. Failed non-blocking polls may be present in Q_exec,P for witness-matching purposes, but they are not members of NextFront_P and are not considered in this BoundaryReq frontier conclusion.

(2) Any payload/value expression for a Push or anchored Write reads some set of source-level variables of P. Those variables agree by I(n), so the produced value agrees.

(3) For Pop_c, the returned value is the FIFO head after the matched history; this is identical on both sides by the bounded-FIFO legality semantics and the matched Push/Pop history.

(4) For anchored signal reads at the next synchronization call, the observed values agree under the fixed-stimulus interpretation plus anchoring; this is the same mechanism used to justify matching Read(sig,val) labels in the first place.

Therefore, (1)–(4) hold, proving Lemma S1. ■

Lemma S2 (Prefix-to-frontier determinacy / "matched Σ-prefix ⇒ same next source-level frontier-item set")

Fix a process P. Let Sys_ref denote the prescribed source reference model of Appendix J, Remark 2: Sys in the raw rendezvous case, Sys_B in the ordinary bounded-FIFO case, and Sys_B^elab in the elaborated back-annotated case. Let τ_ref,system be a finite Sys_ref execution prefix ending in state σ_ref, and let τ_post,system be a finite RTL_B execution prefix ending in state σ_post.

Assume:
(A1) τ_ref,system ≈ τ_post,system under E1–E5.
(A2) σ_post is the ε-normalized (ε-quiescent) endpoint associated with τ_post,system (as used in Corollary J.1).
(A3) ≤_P is P's source-induced partial order on Ω_P generated by the document's R-rules (R1–R4) and the chosen channel/synchronization semantics; the same Ω_P and ≤_P are used on both the Sys_ref and RTL_B sides for the fixed compared witness. The observable alphabet Σ_P is the label/event projection of the items in Ω_P, not the universe over which pending frontier membership is defined.

Definition (Next source-level frontier-item set).
For any terminal state σ, define NextFront_P(σ) as follows.

1. Let τ_P(σ) be the projection of the system prefix leading to σ onto P's Σ-events. Clarification (dynamic execution-call universe and frontier-item universe; fixed for this comparison). In this lemma, Q_exec,P denotes the set of P's executed dynamic source-level message-call instances in the per-process execution / witness determined by the fixed external stimulus for the comparison τ_ref,system ≈ τ_post,system, including, when applicable, any witness-selected failed non-blocking polls required by Appendix I.2. Separately, Ω_P denotes the set of P's dynamic source-level frontier items in that same per-process execution / witness. Thus Ω_P is a single shared frontier-item universe for both prefixes, Sys_ref and RTL_B: it does not depend on how a finite prefix is extended, nor on ε-interleavings between Σ-events. In particular, Ω_P excludes items on untaken control-flow paths, e.g. the "else" branch when the "if" branch was taken, excludes items from loop iterations that are not executed in the fixed-stimulus- and witness-determined per-process execution used for this comparison, and excludes failed non-blocking polls merely because they executed.

Let Q_Ω,P := Q_exec,P ∩ Ω_P denote the message-operation slice of Ω_P. The observable alphabet/projection Σ_P is derived from Ω_P: synchronization and signal items contribute their corresponding Σ labels when observed, and a message-operation frontier item q ∈ Q_Ω,P contributes the committed Σ-event m(q) only if q commits. A pending blocking message-operation instance may therefore be in Ω_P, in Q_Ω,P, and in NextFront_P(σ) even though its committed Σ-event m(q) is not yet present. A failed non-blocking poll q ∈ Q_exec,P \ Ω_P is an ε-step and is matched only through WC / μ_Q when relevant; it is not an element of Ω_P, Q_Ω,P, Done_P, Remain_P, or NextFront_P. Note: even if one of the compared prefixes is deadlocked and therefore cannot realize further committed Σ-events, Ω_P remains this fixed witness universe and is not reduced by deadlock.

2. Let Done_P(σ) ⊆ Ω_P be the set of source-level frontier items of P that have been realized by σ. For synchronization and signal items, this means that the corresponding Σ-event has occurred in τ_P(σ). For

a message-operation frontier item $q \in Q_{\Omega,P}$, this means that q has committed and its Σ-event m(q) occurs in $\tau_P(\sigma)$. A blocking message-operation instance that has issued but not yet committed is not in $Done_P(\sigma)$, even though its BoundaryReq(q,σ), ChannelEnabled(q,σ), and ChannelDisabled(q,σ) predicates may be defined at σ. A failed non-blocking poll is not in $Done_P(\sigma)$ because it is not in Ω_P. Canonical matching is by source-level dynamic item identity / occurrence index, with Σ labels matched as before, e.g. "k-th Push_c commit matches k-th Push_c commit", "k-th Sync matches k-th Sync", etc.

3. Let $Remain_P(\sigma) := \Omega_P \setminus Done_P(\sigma)$.

4. Define:
$NextFront_P(\sigma) := \min_{\leq_P}(Remain_P(\sigma))$,
i.e. the set of $\leq_P$-minimal remaining source-level frontier items for P.
This is exactly Corollary J.1's "set of minimal (w.r.t. P's source partial order) remaining observable items," but typed as source-level frontier items in Ω_P rather than already-committed Σ-events. It should be read as a candidate next source-level frontier-item set relative to the fixed witness universe Ω_P. In particular, because Ω_P is not reduced by deadlock, membership in $NextFront_P(\sigma)$ does not assert that the item is enabled or realizable as an immediate successor of σ when σ is deadlocked; it only asserts $\leq_P$-minimality among the not-yet-realized items in Ω_P.

Claim.
Under (A1)–(A3),
$NextFront_P(\sigma_{ref}) = NextFront_P(\sigma_{post})$.

Proof.

Step 0 (Reduce to the per-process view).
It suffices to show that $Done_P(\sigma_{ref}) = Done_P(\sigma_{post})$; frontier equality then follows immediately from the shared Ω_P, by the definition above, and the shared $\leq_P$, by (A3).

Step 1 (Synchronization events completed are identical).
Under the document's canonical matching, synchronization events for P are matched by source order / occurrence index: the k-th sync in the source corresponds to the k-th sync in the observable history. Under E1, and the sync-order constraints embedded in ≈, $\tau_{ref,P}$ and $\tau_{post,P}$ contain the same set, equivalently the same prefix, of P's sync events. Hence the "current sync interval" is the same on both sides.

Step 2 (Anchored signal I/O events completed are identical).
Signal reads/writes are matched canonically by signal and occurrence index, with their anchoring to sync points enforced by E2 for sequential processes, and with R2C fixed-point bucket matching for combinational processes. Since Step 1 shows the same sync indices have been traversed for sequential-process signal IO, and E2 preserves the anchoring structure, $\tau_{ref,P}$ and $\tau_{post,P}$ contain the same set of P's Σ-visible sequential signal read/write events. For combinational-process signal IO, R2C gives the corresponding equality of fixed-point Read/Write events in the matched observable cycle buckets. Thus signal events contribute the same elements to $Done_P(\sigma_{ref})$ and $Done_P(\sigma_{post})$.

Step 3 (Committed message-operation items completed are identical).
By the Σ-event matching convention built into the definition of ≈, committed channel events are paired canonically by channel endpoint and ordinal occurrence: for each channel c, the k-th committed Push_c

in τ_ref matches the k-th committed Push_c in τ_post, and likewise for Pop_c, with identical payload/value labels. The commutations/regroupings permitted by ≈ only affect the interleaving and same-cycle grouping of Σ-events on distinct independent endpoints; they do not change the per-endpoint projected sequence of committed events. That is, for any fixed endpoint e, proj_e(τ_ref,P) and proj_e(τ_post,P) are identical as sequences of Σ-labels up to the common prefix length. Therefore, for every endpoint-indexed source-level message-operation item q whose commit would be the endpoint event (e,k), q has committed in τ_ref,P iff the matched q has committed in τ_post,P, with the same committed Σ-label. Hence message-operation items contribute the same elements to Done_P(σ_ref) and Done_P(σ_post). Pending issued-but-uncommitted blocking operations remain outside Done_P on both sides; their request/enable correspondence is handled separately by the cutpoint correspondence clauses used by Appendix K.

Step 4 (Conclude Done equality, then frontier equality).
From Steps 1–3:

Done_P(σ_ref) = Done_P(σ_post).

Therefore:

Remain_P(σ_ref)
= Ω_P \ Done_P(σ_ref)
= Ω_P \ Done_P(σ_post)
= Remain_P(σ_post).

Since ≤_P is the same relation on both sides by (A3), taking ≤_P-minimal elements preserves equality:

min_{≤_P}(Remain_P(σ_ref)) = min_{≤_P}(Remain_P(σ_post)).

By definition of NextFront_P(·), this is:

NextFront_P(σ_ref) = NextFront_P(σ_post).
□

Proof of Corollary J.1

Step 0 (By definition, work with a chosen **ε**-normalized terminal state) By the ε-normalization defined in the Statement, choose one maximal finite ε-extension of the given prefix if needed, and continue to denote the resulting ε-normalized prefix by τ_post; let σ_post be its ε-quiescent terminal state. This replacement preserves the observable prefix and is finite by B4. The remainder of the proof is carried out relative to this chosen ε-normalized representative; no claim about uniqueness or projection-equivalence of all maximal ε-extensions is needed here.
Step 1 (Get a matching source-level observable prefix) Fix the external test stimulus that produced the given post-HLS prefix τ_post. By the system-level prefix-matching property in Theorem J.1, applied to Sys_ref and RTL_B, there exists a finite execution prefix τ_B of Sys_ref under that same stimulus such that τ_B,system ≈ τ_post,system under E1–E5. Let σ_B be the terminal state of this witness prefix τ_B.
Step 2 (Align per-process control points) By Lemma S2, because τ_B,system ≈ τ_post,system and E1/E2/E3/E4/E5 preserve per-process same-interface order, no-reverse constraints, side-of-sync constraints, and signal anchoring at synchronization points, each process P has executed the same per-endpoint observable history in both prefixes (up to commutation/regrouping of independent distinct-

interface message commits). Here NextFront_P is understood exactly as in Lemma S2: it is the candidate next source-level frontier-item set relative to the fixed matched witness pair and its shared per-process universe Ω_P, not a stronger claim that the frontier is determined by raw microstate alone independent of the witness. Therefore, at the end of the prefixes,
NextFront_P(σ_B) = NextFront_P(σ_post),
including the same logically anchored signal Read/Write action items at the next synchronization call when applicable.
Step 3 (Align the relevant source-level state) By Lemma S1, matched prefixes preserve the source-level state needed for next-action enablement and payloads exactly as in Appendix I, giving clause (2b). Clause (2c) holds for buffered channels because, under E4, the abstract FIFO empty/full status and logical head element are uniquely determined by the matched Push/Pop history of (1), and thus coincide in σ_B and σ_post. Clause (2d) holds for point-to-point rendezvous channels in the Appendix K internal message-passing boundary / WFG analysis because, under the same source-side state-correspondence / witness hypotheses used to obtain σ_B, the preserved cutpoint state includes the raw producer-side and consumer-side endpoint-request predicates at those compared ε-quiescent cutpoints. This clause is therefore a property of the chosen corresponding cutpoint state slice, not a consequence of matched per-endpoint observable history alone.
Combining Steps 1–3 gives a terminal source-level state **σ**_B with **σ**_B ≡ **σ**_post. □

J.3 Causal Dependency ("Happens-Before") and What System-Level Equivalence Guarantees (Informative)

The system-level result of Appendix J establishes equivalence of the *aggregate* observable behavior, i.e. τ_B,system≈τ_post,system over the alphabet Σ of channel operations, synchronization events, and signal Read/Write actions.
This equivalence is intentionally *observational*: it constrains what an external environment can distinguish at Σ, not internal RTL micro-timing.
As a consequence, Appendix J should be read as preserving functionally significant ordering, not incidental latency. Functionally significant ordering is exactly the ordering induced by the happens-before relation (→) defined in the "Causal Dependency Between Processes" section: a designer must express any required cross-process ordering using message-passing edges, explicit synchronization (e.g., barriers), or explicit signal handshakes; designs "shall not rely" on the relative order of causally independent events.
Accordingly, the system-level theorem implies the following *causality preservation* principle:

- If two synchronization anchors s1 and s2 are causally related (s1→s2), then the post-HLS execution preserves their order. This is because each elementary happens-before link is preserved: intra-process order is preserved by the per-process result; message-passing links are preserved by E4 channel semantics together with the matched channel-history invariant used in Theorem J.1 and Corollary J.1. In particular, for each explicit abstract channel c, the matched executions preserve the committed Push_c/Pop_c history: for B(c)=0, a committed transfer is an explicitly atomic rendezvous Push_c(v)/Pop_c(v) unit; for B(c)>0, the k-th committed Pop_c observes the k-th earlier unmatched Push_c according to reset-empty, FIFO-order, no-underflow/no-overflow, and non-fall-through E4 semantics. Explicit synchronization/handshake links propagate only at clock edges under the same signal and synchronization semantics. By transitivity, the entire causal chain preserves order.
- If s1 and s2 are causally independent (neither s1→s2 nor s2→s1), then their relative order is not constrained by the methodology, and differences are treated as incidental latency variation; relying on such ordering is outside the formal guarantees. In particular, causally independent

events may be grouped into observable cycle buckets differently in Sys_ref and RTL_B; such regrouping is not semantically significant unless an E1–E5 rule, E4 channel rule, signal-anchor rule, or explicit synchronization transaction makes the grouping observable.

This perspective is what makes the "single testbench / no surprises" implication precise: for the fixed-stimulus execution sets covered by Theorem J.1, any testbench that observes only Σ and encodes its expectations through causal dependencies (channels, barriers, or explicit handshakes) cannot distinguish the matched Sys_ref / RTL_B execution pairs provided by the theorem.

Equivalently, RTL_B introduces no new Σ-observable behavior relative to Sys_ref under that fixed stimulus, and source-to-RTL transfer is asserted only for the RTL-consistent source prefixes covered by Theorem J.1. (Here Sys_ref is as fixed in Remark 2. The rendezvous specialization Sys with B(c)=0 is covered only under the explicit conditions of Remark 4.)

Appendix K – Liveness Preservation for the Prescribed Source Reference Model

K.1 Scope, Notation, and Definition of Internal Message-Passing Deadlock

Terminology alignment. Throughout Appendix K, when evaluating predicates at an ε-quiescent cutpoint σ, let t_σ denote the cycle of that cutpoint and use the cutpoint shorthand occ_c(σ) ≜ occ_c(t_σ). For any dynamic source-level message-operation instance q ∈ Q_exec,P on an explicit abstract channel c of Sys_ref, use the shorthands BoundaryReq(q,σ), ChannelEnabled(q,σ), and ChannelDisabled(q,σ) as defined in "Common Formal Definitions for Appendices I-K"; equivalently, when a cycle-index notation is needed, these mean BoundaryReq(q,σ_t), ChannelEnabled(q,σ_t), and ChannelDisabled(q,σ_t) at the ε-quiescent cutpoint for cycle t. Thus 'boundary request is asserted' means BoundaryReq(q,σ); 'channel-enabled' means ChannelEnabled(q,σ); and 'channel-disabled' means ChannelDisabled(q,σ), all evaluated for the specific operation instance q at the ε-quiescent ready/valid fixed point of the cutpoint cycle. These predicates are not applied to already-committed Σ-event labels except through the operation instance q that contributes m(q) when it commits.

We work with a fixed design:

System Sys_ref: The collection of pre-HLS processes obeying the R-rules and B-rules, interpreted under the prescribed source reference model of Appendix J, Remark 2. In this appendix, capacities/occupancies/enablement predicates are interpreted at the chosen abstract boundaries of Sys_ref; in elaborated cases, this means the explicit abstract staged/bundle/join channels of that model. Each explicit abstract channel c of Sys_ref has finite capacity B(c) ≥ 0, chosen to match the corresponding RTL realization at that abstract boundary.

Post-HLS implementation RTL_B: The hardware implementation of the same processes in which each chosen abstract channel c is realized with finite capacity B(c) ≥ 0. In this appendix, we compare Sys_ref (the prescribed source-side proof target) to RTL_B (the micro-architectural realization). The proof does not require mapping buffered RTL states to a distinct raw rendezvous (B(c)=0) state.

Witness-selection note (debug/simulation): Appendix K's liveness argument uses Corollary J.1 only to assert existence of a corresponding Sys_ref state σ_ref for a given ε-normalized RTL state σ_post (via the source-level correspondence ≡). If, for practical debug, one wishes to construct a deterministic Sys_ref execution that tracks the unique RTL Σ-trace (B1) so that σ_ref / WFG_ref are reproducible, then the

Appendix L snooping wrapper may be applied to select a witness Sys_ref execution aligned to RTL latency-sensitive events.

A global state σ of either system consists of:

1. For each process P: a control location (program point) and local state.
2. For each explicit abstract channel c of Sys_ref: its abstract FIFO state at the chosen Sys_ref boundary—its current occupancy occ_c(t) and (for buffered channels) its ordered contents. Here occ_c(t) counts all items that have been committed by Push_c but have not yet been committed by Pop_c at that chosen abstract boundary. In RTL_B, this count includes items residing anywhere in the concrete sub-realization that contributes to that particular abstract channel's back-annotated capacity B(c). Thus internal movement within the concrete realization of one abstract channel c is ε and does not create additional committed Push_c/Pop_c Σ-events beyond the boundary events counted by occ_c; however, transfers between distinct explicit abstract channels introduced in Sys_B^elab are represented in Sys_ref by those channels themselves and are not collapsed back into one outer Sys_B boundary.
3. Any additional architectural state not representing buffered messages on any channel (e.g., computation pipeline registers, arbitration state, bookkeeping, etc.).

We adopt the usual Wait-For Graph (WFG) abstraction, restricted to internal message-passing channels. Internal means: the producer and consumer of channel c are both processes of Sys_ref / RTL_B (i.e., both endpoints are in the modeled system). Channels whose complementary endpoint is the external environment/testbench (DUT boundary) are excluded from the WFG and from the deadlock definition in Appendix K. We write WFG_ref for the wait-for graph of Sys_ref and WFG_post for that of RTL_B.

- Vertices: Processes.
- Edges: There is a directed edge P→Q via channel c in state σ if there exists some operation op ∈ Front_P(σ) on channel c such that op is channel-disabled (i.e., ChannelDisabled(op,t) holds), and Q is the unique process that is the complementary endpoint for op (producer/consumer partner for c). This edges definition applies uniformly for both buffered channels (B(c)>0, enable is space/data availability via occ_c) and rendezvous channels (B(c)=0, enable requires the complementary endpoint's boundary request to be true in the same cycle), hence WFGs may contain a mix of edges via both kinds of channels. Here Front_P(σ) is the set of minimal (w.r.t. P's program-order / partial source order) pending blocking channel operations op∈{Push_c,Pop_c} (issued by P but not yet committed); Front_P(σ) may contain multiple operations on distinct interfaces (reflecting the R/E freedom to issue/commit multiple Push/Pop in one cycle). P is "blocked on message passing" in σ as defined in Definition (Blocked on message passing) (K.1). Note: WFG edges may exist even when P is not blocked (e.g., one frontier op is channel-disabled while another is channel-enabled); however, internal message-passing deadlock (K.1) considers only sets D in which every P∈D is blocked, so all WFG edges relevant to deadlock reasoning originate from blocked processes with no channel-enabled frontier operation. When Front_P(σ) has multiple members, P may have multiple outgoing WFG edges (one per blocked frontier op).
  (No deassert-before-commit assumption.) Blocking Push_c/Pop_c requests are non-withdrawable: once the request corresponding to a frontier operation is asserted, it is not deasserted before that operation commits (and Push_c payload remains stable while asserted). This rules out "try-and-withdraw" behavior for blocking transfers within the WFG abstraction.
  All WFG reasoning is over **ε**-quiescent (**ε**-normalized) global states: internal pipeline/arbitration micro-steps are treated as ε and are not represented as separate WFG blocking points. Here, "ε-quiescent" includes the fixed point of purely combinational ready/valid propagation, so K.1 enablement is evaluated only on those settled ready/valid values.

(WFG-inert **ε** premise.) We assume these ε-steps are observationally silent with respect to the WFG abstraction: for any $\sigma \xrightarrow{\varepsilon^*} \sigma'$, they do not change (i) the potentially-blocking frontier Front_P(σ) for any process P (i.e., which guarded blocking Push/Pop operations are minimal and pending), nor (ii) the truth of any boundary requests relevant to those frontier operations, nor (iii) the channel-side enabling status (as defined in K.1) of any op ∈ Front_P(σ). Equivalently, ε-steps do not change the channel state relevant to K.1 enabling (e.g., occ_c(t) for B(c)>0 buffered channels), and they do not create/remove rendezvous enablement for a frontier transfer except via Σ-visible committed Push_c/Pop_c events.

In particular (pipelined loops / overlapped iterations): when HLS introduces overlap that may initiate later-iteration message reads "early," we assume the design obeys the Automatic flush (drain-only blocking discipline) defined in Appendix B. Under that discipline, overlapped execution does not introduce any later-iteration blocking Push/Pop into Front_P(σ), so transient internal pipeline activity remains ε and does not contribute wait-for edges. Overlapped execution in pipelined loops is modeled only as a potential retiming of ordinary Σ-visible boundary commits (Push_c/Pop_c) relative to the unpipelined source, and is accounted for by the back-annotated semantics at the chosen abstract boundary of Sys_ref (E4). In particular, the channel facts relevant to K.1 enabling/WFG edges—e.g., occ_c(t) for B(c)>0, and rendezvous enablement for B(c)=0—change only on Σ-visible committed Push_c/Pop_c events at that boundary; internal movement within the concrete realization of a single abstract channel is ε and does not affect K.1 enablement or introduce/remove WFG edges.

Precondition (Kε: WFG-inert ε / no hidden micro-timing control decisions). All results in Appendix K are stated under the precondition Kε that the WFG-inert ε premise above holds for the design. If an implementation intentionally uses internal micro-timing to change such control decisions, then Kε is violated unless that behavior is made Σ-observable (e.g., via explicit modeling/snooping) or is witness-selected under the Appendix I.2 framework.

A common instance is branching on the success/failure of a non-blocking poll in a way that changes the next Σ-visible candidate frontier NextFront_P(**σ**) (Corollary J.1 / Lemma S2); in that case, either NB-CF must hold (Appendix I.2), or the poll outcomes must be witness-selected as stated there.

Non-blocking polling attempts and other purely internal actions do not contribute edges to the WFG; they are modeled as internal ε-steps. When a non-blocking call succeeds, that success is already represented at the Σ level as the corresponding committed Push_c/Pop_c event, but it does not itself add a wait-for edge because it is not a blocking operation.

SyncChannels and barrier well-formedness.

SyncChannel (barrier) operations are treated as pure synchronization events and are omitted from the Wait-For Graph, which tracks only blocking Push/Pop dependencies. We therefore interpret "deadlock" in this appendix as internal message-passing deadlock (i.e., a deadlocked set in which all processes are blocked on some internal Push/Pop dependency captured by the WFG). Any global stall at a barrier caused by mismatched or conditional participation (e.g., a process can permanently bypass the barrier or reach it a different number of times) is a specification-level error already present in the pre-HLS model; such stalls are not created by the HLS transformation and are excluded by an explicit barrier well-formedness assumption (A8 below), not by the internal message-passing deadlock premise. Under this assumption, omitting SyncChannels from the WFG is sound: a post-HLS deadlock implies the existence of a Push/Pop wait-cycle as characterized below.

Definition (Blocked on message passing). Using Front_P(σ) as defined above, we say that P is blocked on message passing in state σ iff Front_P(σ) is non-empty and, for every op ∈ Front_P(σ),

BoundaryReq(op, σ) holds and ChannelDisabled(op, σ) holds (equivalently, BoundaryReq(op, σ) ∧ ¬ChannelEnabled(op, σ)). Note (ALL disabled ⇒ stall). If any frontier op is channel-enabled at the chosen Σ-visible boundary, then P is not "blocked on message passing" for WFG purposes. Consequently, an implementation may not be physically stalled at the chosen Σ boundary by a cross-channel "join" predicate while some frontier Σ-boundary op remains channel-enabled; any such conjunctive/join dependency must be represented as an explicit staged/bundle/join boundary channel so that, when the process is physically stalled, the relevant frontier ops on that explicit boundary are all channel-disabled.

Definition (Drain-closed cutpoint for deadlock extraction). Let D be a non-empty set of processes. An ε-quiescent state σ is drain-closed for D iff no already-issued non-frontier blocking Push/Pop request of any process P ∈ D is channel-enabled at σ. Equivalently, after the frontier members Front_P(σ) are all disabled, there is no remaining already-issued downstream transfer of any process in D that can still commit as drain-only progress. If such a non-frontier drain transfer is enabled, σ is not yet a stable deadlock cutpoint for D; either that drain transfer must be allowed to commit finitely often until a drain-closed cutpoint is reached, or the resulting commit changes the WFG-relevant channel facts so that some frontier operation becomes enabled and the candidate deadlock disappears.

An internal message-passing deadlock is a reachable ε-quiescent state σ in which there exists a non-empty set of processes D such that σ is drain-closed for D and:

- Every process P in D is blocked on message passing, and in particular there exists at least one op ∈ Front_P(σ) on an internal message-passing channel c (i.e., a channel included in the WFG) such that BoundaryReq(op, σ) holds and ChannelDisabled(op, σ) holds.
- The vertices in D, together with the internal channels they access, form a strongly connected component in the WFG that has no outgoing edges (a closed cycle): processes in D can only wait on each other.

Intuitively, D is a set of processes that are waiting only on each other (via internal message-passing channels tracked by the WFG), with no remaining enabled drain-only transfer of any process in D that could change the WFG-relevant channel facts. Thus D can never be unblocked either by activity elsewhere in the modeled system or by already-issued downstream drain from within D. A process that is blocked solely on channels whose complementary endpoint is the external environment/testbench (excluded from the WFG) does not witness an internal message-passing deadlock in this appendix.
Our notion of liveness in this appendix is: Liveness = absence of internal message-passing deadlock as defined above. (Starvation of a single process in an otherwise live system is ruled out separately by weak fairness.)

K.2 Assumptions and Imported Results

K.2.1 Standing Assumptions We assume throughout Appendix K:
A1. R-rules and B-rules. The prescribed source reference model Sys_ref and the post-HLS implementation RTL_B satisfy the process- and channel-level semantic rules defined earlier (R-rules and B-rules).
A2. Finite Pipeline Depth (FPD). There is a global bound on the number of purely internal steps (ε-steps) that can occur between consecutive observable actions.
A3. Weak Fairness (WF) (B2). If an action remains continuously enabled (its guard remains true), the scheduler eventually selects it.
A4. Channel Progress (B3 (Additional Semantic Assumptions (B-rules)); Appendix N). For each channel c with capacity B(c), assume the progress invariants of Appendix N hold. This is an obligation on the

scheduler/environment (e.g., fairness of arbiters/back-pressure) and is not implied by the local R-rules alone.
A5. Source-Level Liveness Assumption. The prescribed source reference model Sys_ref (with capacities B(c) interpreted on its chosen abstract channels, including the explicit abstract channels of Sys_B^elab in the elaborated cases) is deadlock-free under assumptions A1–A4.
A6. Point-to-point channels (single-producer/single-consumer). For each message-passing channel c (including buffered channels with B(c)>0 and rendezvous channels with B(c)=0), exactly one process performs all Push_c operations on c (the producer) and exactly one process performs all Pop_c operations on c (the consumer). Hence the complementary endpoint for any blocked Push_c/Pop_c is unique, and WFG edges are well-defined even for mixed WFGs that include both buffered and rendezvous channels.
If a design has a shared resource that would violate this uniqueness, it must be modeled via an explicit arbiter process plus point-to-point channels (as noted in "Common Formal Definitions" B(c)).
A7 (Determinism / witness selection). Assume B1 holds for RTL_B (post-HLS), so the Σ-labeled execution under the given stimulus is unique up to finite ε-stutter. Do not require B1 for Sys_ref: there may be multiple Sys_ref executions that are ≡ to the same τ_post at the Σ/frontier level. Appendix K uses only the existence of such a Sys_ref witness (Corollary J.1). When a deterministic, reproducible Sys_ref witness is desired for simulation/debug, apply the Appendix L snooping wrapper to select a particular witness aligned with the latency-sensitive (arbiter-visible) ε-choices. We also rely on B4 (from the Additional Semantic Assumptions) and B5 (Appendix N) to justify ε-normalization / ε-quiescent endpoints, as required by Corollary J.1 (state correspondence).
A8. Barrier well-formedness (SyncChannels). For each SyncChannel instance, the set of designated participant processes reaches the barrier the same number of times under the fixed stimulus/initial state; equivalently, no participant can permanently bypass a barrier call while another participant is waiting at that barrier. (Barrier stalls are therefore outside the behaviors considered in Appendix K's internal message-passing deadlock analysis.)
A9. Single-transfer-per-cycle endpoint constraint (scalar channel interface). For every message-passing channel c, in any clock cycle t, at most one Push_c commit and at most one Pop_c commit may occur on c. If a design uses multi-lane/burst transfers on a logical resource, it must be modeled explicitly (e.g., k parallel point-to-point channels or a widened message) so that the Σ-level per-channel event model satisfies this constraint.
A10. Reset-empty FIFO assumption. After each reset boundary at which the compared execution begins or restarts, every buffered message-passing channel with B(c)>0 has logical occupancy 0 at the chosen Σ boundary. The Appendix J/K occupancy equations and the per-channel Push/Pop precedence lemma are interpreted relative to that reset-empty initial channel state.
A11. WFG-inert **ε** (K**ε**). The design satisfies Precondition Kε (WFG-inert ε / no hidden micro-timing control decisions) as stated in K.1.

No further standing assumption is introduced here beyond A1–A11.

The Front/NextFront bridge used later in Appendix K is derived in Lemma K.0 below from the Appendix B drain-only blocking discipline and the Appendix C Issue/Commit linkage. The rendezvous endpoint-request agreement used later in Appendix K is carried by the corresponding-cutpoint relation σ_ref ≡ σ_post (Corollary J.1 / Definition R.18), not by any claim that an asserted rendezvous request must belong to Front_P(σ).

K.2.2 Lemma K.0 — Front/NextFront Classification at **ε**-Quiescent Cutpoints

Purpose. Appendix K defines WFG edges using Front_P(σ) (minimal pending blocking Push/Pop operation instances), while Corollary J.1's state correspondence is phrased in terms of NextFront_P(σ) plus BoundaryReq agreement for message-operation items in that next frontier. The present lemma shows that, at ε-quiescent cutpoints, the asserted blocking message-operation slice BlockingMsgNextFront_P(σ) classifies the pending blocking frontier. No extra standing assumption is needed.

Clarification. Throughout this section, NextFront_P is interpreted in the witness-relative sense of Lemma S2: it is the ≤_P-minimal remaining source-level frontier-item set within the fixed matched witness universe Ω_P, not a claim of immediate enablement and not a stronger state-intrinsic frontier independent of that witness. The observable Σ-event contributed by a message-operation item exists only when that item commits; pending blocking items may therefore belong to NextFront_P without yet contributing a committed Σ-event.

Statement. Fix a compared execution / witness w (hence its associated per-process universes Q_exec,P, Ω_P, Q_Ω,P, Σ_P and order ≤_P), an ε-quiescent global state σ occurring as a cutpoint of that execution (of either Sys_ref or RTL_B), and a process P. Define the blocking message-operation slice of the next source-level frontier-item set, relative to this fixed witness, by:
BlockingMsgNextFront_P(σ) ≜ { x ∈ NextFront_P(σ) | x ∈ Q_Ω,P and x is a blocking source-level message-operation instance with kind(x) ∈ {Push_c, Pop_c} for some channel c }.

Then:
Front_P(σ) = { x ∈ BlockingMsgNextFront_P(σ) | BoundaryReq(x,σ) }.
Consequently, for any corresponding ε-quiescent states (σ_ref,σ_post) compared under the same matched witness pair and satisfying σ_ref ≡ σ_post (Corollary J.1),
Front_P(σ_ref)=Front_P(σ_post),
and for every op ∈ Front_P(σ_ref) the boundary request BoundaryReq(op,·) agrees in σ_ref and σ_post.

Non-blocking PushNB/PopNB call instances are intentionally excluded from BlockingMsgNextFront_P unless they are represented elsewhere as committed non-blocking transfer items in Q_Ω,P for non-WFG frontier/value reasoning. A successful non-blocking call may contribute the same committed Push_c/Pop_c Σ-label as a blocking call, and a failed non-blocking poll is ε, but neither case creates a pending blocking frontier member for WFG purposes. This exclusion is a predicate on blocking source-level operation frontier items in Q_Ω,P, not on committed Σ-event labels and not on failed polls in Q_exec,P \ Ω_P.

Corollary K.0a (Rendezvous endpoint-request agreement at corresponding cutpoints).
Let c be a point-to-point rendezvous channel (B(c)=0), and let P_prod(c) and P_cons(c) denote its unique producer and consumer processes. Define:
Req_prod(c,σ) :⇔ the producer-side boundary request on c is asserted at the ε-quiescent cutpoint σ;
Req_cons(c,σ) :⇔ the consumer-side boundary request on c is asserted at the ε-quiescent cutpoint σ.
Then, for any corresponding ε-quiescent cutpoints (σ_ref,σ_post) satisfying σ_ref ≡ σ_post,
Req_prod(c,σ_ref) = Req_prod(c,σ_post)
and
Req_cons(c,σ_ref) = Req_cons(c,σ_post).
No equivalence is claimed here between an asserted rendezvous endpoint request and membership in Front_P(σ); Appendix B explicitly allows already-asserted non-frontier blocking requests to persist as protocol state.

Proof. For the main equality:

(⊆) Let op ∈ Front_P(σ). Then op ∈ Pending_P(σ), so op is a blocking source-level message-operation instance whose Issue has occurred and whose Commit has not yet occurred in σ. By Appendix C's Issue/Commit linkage, BoundaryReq(op,σ)=true. Because op is ≤_P-minimal among the pending blocking message-operation instances in the current source interval, no earlier remaining blocking message-operation item of P in that interval can also be pending. Moreover, under the side-of-synchronization discipline, any synchronization or anchored signal frontier item that precedes the current message interval has already been realized at this cutpoint, and failed non-blocking polls are ε-steps rather than Ω_P frontier items. Thus op is the ≤_P-minimal remaining blocking message-operation item that can constitute the current blocking cause.

This is only a frontier-minimality statement. It does not assert that the implementation could not have already issued a later same-interval request. Appendix B explicitly allows such younger or downstream already-asserted requests to persist as non-frontier protocol state under the no-withdrawal and drain-only rules. Those requests simply do not enter Front_P(σ) while op remains the earlier frontier-minimal pending blocker. Therefore op belongs to the blocking message-operation slice of the next source-level frontier, i.e. op ∈ BlockingMsgNextFront_P(σ).

(⊇) Let x ∈ BlockingMsgNextFront_P(σ) and BoundaryReq(x,σ)=true. Since x is, by definition, a blocking source-level message-operation instance, Appendix C's stable-attribution rule for asserted blocking requests applies: the asserted request at the ε-quiescent cutpoint is attributed to a unique currently outstanding issued blocking message-operation instance q on the same endpoint direction; hence q ∈ Pending_P(σ). Because BoundaryReq(x,σ) is read operation-attributively for the specific source-level message-operation instance x, that unique outstanding operation must in fact be x itself: attribution cannot change from x to a distinct same-direction operation without an intervening commit on that endpoint direction, and no such commit has occurred while the request remains outstanding at the cutpoint. Thus q = x, so x ∈ Pending_P(σ). Since x lies in BlockingMsgNextFront_P(σ), there is no earlier remaining blocking message-operation item of P in the current source interval. Therefore x is ≤_P-minimal among the pending blocking message-operation instances in that interval, so x ∈ Front_P(σ). Appendix B's clarification on frontier-minimality vs. other asserted requests ensures that any younger already-asserted request does not enter Front_P(σ) while an earlier pending blocker remains frontier-minimal.

This proves
Front_P(σ) = { x ∈ BlockingMsgNextFront_P(σ) | BoundaryReq(x,σ) }.

Now let (σ_ref,σ_post) be corresponding ε-quiescent states with σ_ref ≡ σ_post. By Corollary J.1(2a), NextFront_P agrees between σ_ref and σ_post; hence the blocking message-operation slice BlockingMsgNextFront_P agrees as well. By Corollary J.1(2b), BoundaryReq agrees for every message-operation item x ∈ Q_Ω,P in that common NextFront_P; no BoundaryReq predicate is asserted for non-message frontier items. Therefore BoundaryReq also agrees for every x in the common BlockingMsgNextFront_P. Applying the displayed equality on both sides yields
Front_P(σ_ref)=Front_P(σ_post),
and BoundaryReq agrees for every op in that common Front set.

For the rendezvous corollary, direct frontier-classification is not sufficient: Appendix B explicitly allows an already-asserted blocking request that is not in Front_P(σ) to persist as protocol state while an earlier pending blocker remains frontier-minimal. Therefore we do not identify raw rendezvous endpoint-request assertion with frontier membership. Instead, for each point-to-point rendezvous channel c, the corresponding-cutpoint relation σ_ref ≡ σ_post carries equality of the producer-side and consumer-side endpoint-request predicates Req_prod(c,·) and Req_cons(c,·) across the compared ε-quiescent cutpoints because those raw endpoint-request predicates are included in the source-side state slice preserved by Corollary J.1 / Definition R.18, clause (4). This is exactly the predicate later used in the rendezvous enabling condition. □

K.2.2 Example K.0b — Front_P vs. MsgNextFront_P Example

The following example illustrates the distinction between Front_P and MsgNextFront_P, where MsgNextFront_P is the message-operation-item slice of the source-level NextFront_P frontier over Ω_P.

```
void P() {
  while (1) {
    wait();          // sync point
    int x = in.Pop();   // blocking Pop
    out.Push(x + 1);    // blocking Push
  }
}
```

Consider two ε-quiescent cutpoints for this same process:
Cutpoint **σ**0: just after wait() has committed, before in.Pop() has issued

- MsgNextFront_P(σ0) = { Pop_in }
- BoundaryReq(Pop_in, σ0) = false
- Front_P(σ0) = ∅

Cutpoint **σ**1: in.Pop() has issued, rdy is asserted, but no data has arrived yet

- MsgNextFront_P(σ1) = { Pop_in }
- BoundaryReq(Pop_in, σ1) = true
- Front_P(σ1) = { Pop_in }

So the difference is:

- MsgNextFront_P says which message-operation items are next candidates in the source-level frontier over Ω_P.
- Front_P says which of those next message-operation items are actually outstanding pending blockers right now.

K.3 Lemma K.1 — Characterization of Internal Message-Passing Deadlock (Buffered and Rendezvous Cases)

Statement: Let σ_post be a reachable ε-quiescent global state of the post-HLS system RTL_B (equivalently, the ε-normalization of some reachable state, which exists as a finite extension by B4/B5). Suppose there is a non-empty set of processes D that is internally message-passing deadlocked (as defined in K.1). Then:
Every process P in D is blocked on a blocking channel operation (Push or Pop), not on an internal step.
Buffered-channel case (B(c) > 0):
• If P is blocked on a frontier operation instance q ∈ Front_P(σ_post) with kind(q)=Push_c and B(c)>0, then occ_c(σ_post) = B(c) (channel is full at that cutpoint).
• If P is blocked on a frontier operation instance q ∈ Front_P(σ_post) with kind(q)=Pop_c and B(c)>0, then occ_c(σ_post) = 0 (channel is empty at that cutpoint).

Rendezvous case (B(c) = 0):
• If P is blocked on a frontier operation instance q ∈ Front_P(σ_post) with kind(q)=Push_c and B(c)=0, then the complementary endpoint is not simultaneously requesting the matching Pop_c in σ_post (i.e., the complementary Pop_c endpoint request is not true in that cycle, so rendezvous enabling for q is false).
• If P is blocked on a frontier operation instance q ∈ Front_P(σ_post) with kind(q)=Pop_c and B(c)=0, then the complementary endpoint is not simultaneously requesting the matching Push_c in σ_post.
The processes in D form a closed strongly connected component in the WFG.
Proof:
Internal Steps: Because σ_post is ε-quiescent (i.e., an ε-normalized endpoint that exists as a finite extension by B4/B5), no process has any enabled internal ε-step in σ_post. In particular, if some process had an internal step whose guard is true in σ_post, then σ_post would not be ε-quiescent: the ε-normalization could be extended by at least one additional ε-step. Therefore, no process in D is blocked on an internal step with a true guard; since each P ∈ D is blocked on message passing (K.1), the blocking operation must be a blocking channel operation (Push or Pop).
Blocked Push: By the definition of "blocked on message passing" in K.1, if P is blocked due to a frontier operation instance q ∈ Front_P(σ_post) with kind(q)=Push_c, then BoundaryReq(q,σ_post) is true and ChannelEnabled(q,σ_post) is false.
• If B(c)>0, the channel-side enabling condition for q is space availability, i.e., occ_c(σ_post) < B(c). Hence, if P is blocked on such a Push_c operation instance q in σ_post, necessarily occ_c(σ_post)=B(c) (channel is full).
• If B(c)=0 (rendezvous), the channel-side enabling condition for q is that the complementary Pop_c endpoint request is true in the same cycle. Since P is blocked, this rendezvous enabling condition is false; equivalently, the complementary endpoint is not simultaneously requesting Pop_c in σ_post.
Blocked Pop: Symmetric. By the definition of "blocked on message passing," if P is blocked due to a frontier operation instance q ∈ Front_P(σ_post) with kind(q)=Pop_c, then BoundaryReq(q,σ_post) is true and ChannelEnabled(q,σ_post) is false.
• If B(c)>0, the channel-side enabling condition for q is data availability, i.e., occ_c(σ_post) > 0. Hence occ_c(σ_post)=0 (channel is empty).
• If B(c)=0 (rendezvous), the channel-side enabling condition for q is that the complementary Push_c endpoint request is true in the same cycle. Since P is blocked, this rendezvous enabling condition is false; equivalently, the complementary endpoint is not simultaneously requesting Push_c in σ_post.
Closed Cycle: By definition of internal message-passing deadlock, processes in D cannot wait on processes outside D, or else an external action could unblock them.

K.4 Lemma K.2 — Dependency Refinement

We now relate the blocking dependencies in RTL_B to those in the prescribed source reference model Sys_ref.

Statement: For any pair of corresponding states (σ_ref,σ_post) where σ_ref ≡ σ_post (Corollary J.1), the wait-for edges agree in both directions: for any processes P,Q, there is a wait-for edge P→Q in WFG_post(σ_post) iff there is a wait-for edge P→Q in WFG_ref(σ_ref).
(In particular, WFG_post ⊆ WFG_ref, and the reverse inclusion holds as well.) This covers both the ordinary case Sys_ref = Sys_B and the elaborated case Sys_ref = Sys_B^elab. The statement holds for mixed WFGs containing edges via both buffered channels (B(c)>0) and rendezvous channels (B(c)=0), with B(c), occ_c(σ), BoundaryReq, ChannelEnabled, and ChannelDisabled interpreted at the chosen ε-quiescent cutpoints on the chosen abstract channels of Sys_ref.

Proof: Since σ_ref ≡ σ_post, clause (2a) gives NextFront alignment for every process P, and clause (2b) gives BoundaryReq agreement for message-operation items in the common NextFront_P at this ε-quiescent cutpoint. By Lemma K.0, this lifts directly to alignment of the pending blocking frontiers: for every P,
Front_P(σ_ref)=Front_P(σ_post),
and BoundaryReq(op,σ_ref)=BoundaryReq(op,σ_post) for every operation instance op ∈ Front_P. Moreover, for buffered channels with B(c)>0, the channel empty/full predicate agrees between σ_ref and σ_post (Corollary J.1, clause (2c)).

For rendezvous channels with B(c)=0, the complementary-endpoint request predicate needed for channel enablement is transferred directly by the corresponding-cutpoint relation. Concretely, for each point-to-point rendezvous channel c in the Appendix K internal message-passing boundary / WFG analysis, Corollary J.1 (equivalently Definition R.18, clause (4)) gives equality of the raw producer-side and consumer-side endpoint-request predicates across σ_ref ≡ σ_post, with that equality supplied by the preserved source-side cutpoint state used in the correspondence — not by any claim that matched observable history alone determines all already-asserted rendezvous requests. This is the exact predicate used by the rendezvous enabling condition, and it does not require any identification of asserted requests with frontier membership.

We show that each RTL edge transfers to the source-level WFG by case-splitting on B(c).

Case 1: Buffered channel (B(c)>0).

Consumer (Blocked Pop): Suppose P waits for Q via Pop_c in the RTL system on a channel c with B(c)>0. By Lemma K.1, occ_c(σ_post)=0 in σ_post (empty). By σ_ref ≡ σ_post and Corollary J.1(2c), we have the same logical occupancy in σ_ref; hence occ_c(σ_ref)=0 in σ_ref as well. Under the buffered-channel semantics of Sys_ref at that abstract channel boundary, completion of P's Pop_c requires a complementary Push_c by Q. Hence P→Q is also an edge in WFG_ref.

Producer (Blocked Push): Symmetric. If P waits for Q via Push_c on a channel c with B(c)>0, Lemma K.1 gives occ_c(σ_post)=B(c) (full) in σ_post. By σ_ref ≡ σ_post and Corollary J.1(2c), σ_ref has the same logical occupancy; hence occ_c(σ_ref)=B(c) in σ_ref as well. Completion of P's Push_c requires a complementary Pop_c by Q to make space. Hence P→Q is also an edge in WFG_ref.

Case 2: Rendezvous channel (B(c)=0).

Producer (Blocked Push): Suppose P waits for Q in the RTL system via a frontier operation instance q ∈ Front_P(σ_post) on channel c with kind(q)=Push_c and B(c)=0. By Lemma K.1, the complementary Pop_c endpoint request on c is not asserted in σ_post. By rendezvous endpoint-request agreement across corresponding cutpoints, the complementary Pop_c endpoint request on c is also not asserted in σ_ref. Since q ∈ Front_P(σ_post), Front alignment under σ_ref ≡ σ_post (Lemma K.0) gives q ∈ Front_P(σ_ref); and by BoundaryReq agreement for frontier operation instances, BoundaryReq(q,σ_ref)=true. Thus in σ_ref the producer is requesting the Push_c operation instance q while the complementary Pop_c endpoint is not requesting on c. Therefore q is channel-disabled in σ_ref, and P→Q is an edge in WFG_ref.

Consumer (Blocked Pop): Symmetric. Suppose P waits for Q in the RTL system via a frontier operation instance q ∈ Front_P(σ_post) on channel c with kind(q)=Pop_c and B(c)=0. By Lemma K.1, the

complementary Push_c endpoint request on c is not asserted in σ_post. By rendezvous endpoint-request agreement across corresponding cutpoints, the complementary Push_c endpoint request on c is also not asserted in σ_ref. Since q ∈ Front_P(σ_post), Front alignment under σ_ref ≡ σ_post (Lemma K.0) gives q ∈ Front_P(σ_ref); and by BoundaryReq agreement for frontier operation instances, BoundaryReq(q,σ_ref)=true. Thus in σ_ref the consumer is requesting the Pop_c operation instance q while the complementary Push_c endpoint is not requesting on c. Therefore q is channel-disabled in σ_ref, and P→Q is an edge in WFG_ref.

Therefore every RTL wait-for dependency is also a source-level wait-for dependency, so WFG_post ⊆ WFG_ref.

Conversely, suppose P→Q is an edge in WFG_ref. By the edge definition in K.1, there exists some operation op ∈ Front_P(σ_ref) on channel c such that op is channel-disabled in σ_ref and Q is the unique complementary endpoint for c. From the Front/BoundaryReq alignment established above (via σ_ref ≡ σ_post and Lemma K.0), we have op ∈ Front_P(σ_post); and from the buffered (empty/full via occ_c) and rendezvous (complementary BoundaryReq) enabling-predicate agreement established above, op is also channel-disabled in σ_post. Hence P→Q is an edge in WFG_post. Therefore WFG_post and WFG_ref have exactly the same wait-for edges (in both directions) at corresponding cutpoints. □

K.4a Lemma K.2a — Drain-Closure Transfer

Statement. Let σ_ref and σ_post be corresponding ε-quiescent cutpoints with σ_ref ≡ σ_post, and let D be a non-empty set of processes. Suppose σ_post is drain-closed for D in the sense of K.1. Then σ_ref is also drain-closed for D.

Proof. By K.1, drain-closure for D means that no already-issued non-frontier blocking Push/Pop request of any process P ∈ D is channel-enabled at the cutpoint. We show that if σ_ref were not drain-closed for D, then σ_post would not be drain-closed for D either.

Assume, for contradiction, that σ_ref is not drain-closed for D. Then there exists a process P ∈ D and a blocking message operation instance op of P such that:
- op is already issued and not yet committed in σ_ref, i.e. op ∈ Pending_P(σ_ref);
- op is not frontier-minimal, i.e. op ∉ Front_P(σ_ref); and
- ChannelEnabled(op,σ_ref) holds.

Since op is pending and blocking, Appendix C's Issue/Commit linkage gives the endpoint-owned request BoundaryReq(op,σ_ref), with the request attributed to this same dynamic source-level operation instance op. Since op is non-frontier only because an earlier pending blocking operation remains frontier-minimal, op is not a WFG source at this cutpoint, but it is still a concrete outstanding boundary request whose possible commit is part of the drain-only protocol state.

By Corollary J.1, clause (e), the corresponding-cutpoint relation σ_ref ≡ σ_post preserves the already-issued blocking request state needed for the automatic-flush / drain-only discipline. Therefore:
- op ∈ Pending_P(σ_post);
- the endpoint-owned request in σ_post is attributed to the same dynamic source-level operation instance op;
- BoundaryReq(op,σ_post) holds; and
- ChannelEnabled(op,σ_post) holds.

The last item is evaluated at the chosen abstract boundary of Sys_ref / RTL_B. For B(c)>0 channels it follows from the common B(c) and the shared logical occupancy/head-state facts of Corollary J.1, clause (c). For B(c)=0 rendezvous channels it follows from the raw complementary endpoint-request agreement of Corollary J.1, clause (d). In the elaborated case Sys_ref = Sys_B^elab, these facts are interpreted at the explicit abstract channel on which op is pending.

Moreover, because Front_P(σ_ref)=Front_P(σ_post) by Lemma K.0 / Lemma K.2, op remains non-frontier in σ_post whenever it is non-frontier in σ_ref. Thus σ_post has an already-issued non-frontier blocking request of a process in D that is channel-enabled, contradicting the assumption that σ_post is drain-closed for D.

Hence σ_ref is drain-closed for D. □

K.5 Theorem K.1 — Liveness Preservation

Statement:
If the prescribed source reference model Sys_ref is deadlock-free and the Appendix I/J correspondence prerequisites hold (in particular NB-CF, or else RTL-consistent witness-selected NB outcomes for latency-sensitive NB control flow), then RTL_B is also deadlock-free.
This theorem preserves absence of message-passing deadlock (WFG over blocking Push/Pop). SyncChannel/barrier deadlocks are treated as specification errors and are out of scope here.
Proof (by Contradiction):

1. Assume RTL_B reaches a deadlocked state **σ**_post.
2. Exec_post applicability at a deadlock cutpoint: By the time-divergence / idle-stutter convention (B6), the finite reachability prefix ending at σ_post can be extended (under the same stimulus) to an infinite execution. It remains to justify that there exists an extension that satisfies the Appendix J fairness/progress premises (B2/B3).
   At σ_post, we are at an ε-quiescent and drain-closed cutpoint and (by the definition of internal message-passing deadlock in K.1) there exists a non-empty set D of processes such that every P ∈ D is blocked on message passing, σ_post is drain-closed for D, and D is closed in the WFG (processes in D can only wait on each other). In particular, for every P ∈ D and every op ∈ Front_P(σ_post), BoundaryReq(op,σ_post) holds and ChannelDisabled(op,σ_post) holds. Since σ_post is drain-closed for D, no already-issued non-frontier blocking Push/Pop request of any process in D is channel-enabled at σ_post either.
   The use of Lemma K.1 here is case-split by channel kind. For a rendezvous channel B(c)=0, ChannelDisabled(op,σ_post) means that the required complementary endpoint request is not simultaneously asserted at σ_post. For a buffered channel B(c)>0, ChannelDisabled(op,σ_post) does not imply absence of a complementary endpoint request; it gives only the corresponding FIFO fact: a blocked Push_c has occ_c(σ_post)=B(c), and a blocked Pop_c has occ_c(σ_post)=0. Thus, in the buffered case, persistence of the deadlock is justified by the closed-SCC condition together with drain-closure and the Appendix N channel-progress premises: no enabled non-frontier drain transfer of a process in D exists, and any future creation of space/data capable of enabling a frontier operation of a process in D would have to arise through the same closed internal dependency set D rather than from a process outside D.
   Therefore no Σ-visible message transfer belonging to any P ∈ D is enabled at σ_post: frontier transfers are disabled by blockedness, and non-frontier drain transfers are disabled by drain-closure. By closure of D, actions of processes outside D cannot make any frontier op of a process

in D become channel-enabled without creating an outgoing WFG dependency, which is excluded by the closed-SCC hypothesis. Hence weak fairness (B2), which constrains only actions that are continuously enabled, imposes no obligation to schedule any transfer of a process in D on a continuation that preserves this deadlock. For processes outside D, choose an infinite continuation under the same stimulus that satisfies the fairness/progress premises in the usual way (i.e., whenever some observable Σ-action/transfer remains continuously enabled, it is eventually taken; and if the run reaches a B5 fixed point, the remainder may be idle-stutter). Moreover, the channel-progress invariants (B3) apply only when their antecedents hold. For rendezvous channels B(c)=0 incident to D, Lemma K.1's rendezvous-request characterization ensures that the complementary endpoint request needed to trigger the B3 antecedent is not simultaneously present at σ_post for any frontier-blocking transfer of a process in D. For buffered channels B(c)>0, Lemma K.1 provides only the full/empty occupancy fact for the blocked frontier operation; it does not imply absence of the complementary endpoint request. In the buffered case, the relevant B3 antecedent is prevented at σ_post by the combination of full/empty blockedness, drain-closure for D, and closed-SCC reasoning: no enabled drain transfer of a process in D exists, and no process outside D can create the required space/data for a frontier operation of D without contradicting closure of D in the WFG. Therefore there exists an infinite fair/progress-satisfying extension of the prefix reaching σ_post, so this prefix lies in Exec_post, and Corollary J.1 applies at this ε-quiescent drain-closed cutpoint.

3. Extract Cycle: Fix a non-empty set of processes D as guaranteed by the definition of internal message-passing deadlock in K.1: (i) every process $P \in D$ is blocked on message passing at σ_post, and (ii) D forms a strongly connected component in WFG_post with no outgoing edges (i.e., D is closed: processes in D can only wait on each other). In the remainder of this proof, "deadlocked process" means "a process in D". We invoke Lemma K.1 only after fixing this D, to characterize what each $P \in D$ is blocked on (and the associated full/empty / rendezvous-request facts) at σ_post, including when checking whether any B3 antecedent could hold for a frontier-blocking transfer of a process in D.
4. Map to Source-Level Reference Semantics: By Corollary J.1 (State Correspondence at the End of a Matched Observable Prefix), there exists a corresponding source-level state σ_ref of Sys_ref such that σ_ref ≡ σ_post.
5. Transfer Cycle and Drain-Closure: By Lemma K.2, for this corresponding pair (σ_ref,σ_post) the wait-for edges agree in both directions: for any processes P,Q, P→Q is an edge in WFG_post iff P→Q is an edge in WFG_ref. Since D is a closed SCC in WFG_post, every outgoing wait-for edge from any P∈D targets a process in D. Therefore every outgoing wait-for edge from any P∈D in WFG_ref also targets a process in D, so D has no outgoing edges in WFG_ref and forms a closed wait-for cycle there as well.
   It remains to transfer the drain-closed part of the K.1 deadlock definition. Since σ_post is drain-closed for D by the assumed RTL_B deadlock and σ_ref ≡ σ_post by Corollary J.1, Lemma K.2a gives that σ_ref is also drain-closed for D. Moreover, by Lemma K.0 / Lemma K.2, the pending blocking frontiers agree between σ_post and σ_ref, and the relevant BoundaryReq and ChannelDisabled facts for frontier operations agree. Hence every process $P \in D$ is blocked on message passing in σ_ref, D is closed in WFG_ref, and σ_ref is drain-closed for D.
6. Contradiction: Therefore Sys_ref is internally message-passing deadlocked in state σ_ref, in the full K.1 sense of deadlock, contradicting the Source-Level Liveness Assumption (A5).
7. Conclusion: RTL_B cannot deadlock. □

Appendix L – Snooping Introduction, Implementation and Concerns

Snooping Introduction

If designs are completely latency insensitive, verification of the pre-HLS versus post-HLS models is straightforward. However, if the design contains some components such as arbiters which have latency-sensitive behavior, and if that latency-sensitive behavior is in some cases externally visible to the DUT, then verification becomes somewhat more complex. Techniques can be applied to simplify verification.

Consider a design which has an arbiter like Matchlib toolkit example 09*. The arbiter uses non-blocking PopNB() on its inputs, and blocking Push() on its output to emit the winner of the arbitration. Since the latency between the pre-HLS and post-HLS designs will differ, the order of transactions presented to the arbiter in the two scenarios will differ, and thus the order of the winners will differ. This may result in verification mismatches between the two models if the order of the winners is externally visible to the DUT.

To force the pre-HLS and post-HLS simulations to match, we can run the two designs side-by-side. We can snoop the inputs to the arbiter in the post-HLS model, and only allow the inputs to the pre-HLS arbiter to proceed when the corresponding inputs are seen in the post-HLS model. This will force the order of the inputs to be equivalent between the pre-HLS and post-HLS models, and thus both arbiters will pick the same winners. When this technique is used, the pre-HLS simulation will be throttled by the post-HLS simulation, but the overall verification will still work properly.

Another related example involves interrupt request signals feeding into an interrupt controller within a CPU model. Typically, each interrupt request signal is a single bit sc_signal<>, indicating that an interrupt request is pending. If the requests originate from accelerator blocks that are being synthesized through HLS, then because of the latency differences between the pre-HLS and post-HLS models, the order of interrupt requests arriving at the interrupt controller will differ between the two models, and this will likely result in verification mismatches if the differences are externally visible to the DUT. To force the order of the requests to match, we snoop the requests arriving at the controller in the post-HLS model, and only then allow the requests to be seen in the pre-HLS model.

Simplified Snooping Approaches

In the snooping example directly above in which the arbiter is snooped, the entire post-HLS RTL DUT is simulated alongside the pre-HLS model to force the arbiter requests to be aligned in time. This is the most general case and assumes the input delays to the arbiter are difficult to determine via analysis.

Sometimes there are simpler cases where the input delays to the arbiter in the RTL DUT are easier to determine. For example, each input to the arbiter might arrive a fixed number of clock cycles after one of the primary inputs to the RTL DUT is pushed by the testbench. In this case, a much simpler model can be used to align the pre-HLS model with the post-HLS model. We can simply monitor the primary inputs to the pre-HLS model and then apply the fixed cycle delays to the pre-HLS arbiter inputs.

The two cases above illustrate two ends of a spectrum of possible approaches for extracting the delays from the RTL DUT. Between these two points there exist other possible approaches.

Snooping Implementation

To enable perfect matching between the pre-HLS and post-HLS system simulations, latency-sensitive global signals and latency-sensitive non-blocking message-passing operations need to be synchronized between the two simulations if their latency differences are externally visible to the DUT.

As explained earlier in this appendix, in the general case the entire post-HLS DUT RTL must be run alongside the pre-HLS system to achieve alignment. Examples of signals that need to be synchronized include global interrupt request signals (as discussed earlier in this appendix) and rdy/vld signal pairs for non-blocking operations on message-passing channels. All such synchronization signals and their corresponding handshake signals must be level-stable:

- A latency-sensitive signal s is level-stable if, once s becomes 1 in cycle t, it must remain 1 until the corresponding handshake signal h is 1 in some cycle $t' \geq t$.

Once the set of signals that need to be aligned between the two simulations is identified, the general rule to implement snooping is simple:

- For each of the pre-HLS and post-HLS simulations, make all readers of the signals see the logical AND of the values of each signal being driven in each separate simulation.

Often the post-HLS simulation will never run ahead of the pre-HLS simulation, because HLS typically only adds (rather than subtracts) latency within each process. This means that often the only signals that need to be delayed are on the pre-HLS side, and therefore the logical AND of the nets may not need to be driven on the post-HLS side. If this optimization technique is used, the logical AND of the nets should be compared with the actual value driven on the post-HLS side, and if they do not perfectly match then the optimization technique must not be used.

Snooping Concerns

The snooping technique is used to align pre-HLS and post-HLS latency-sensitive global signals and latency-sensitive non-blocking message-passing operations in cases where their latency differences are externally visible at the DUT boundary. When such a wrapper is applied, the pre-HLS model is throttled to follow the latency choices made by the post-HLS RTL, and the resulting traces at the observable interfaces are forced to agree.

Readers with a formal background may be concerned by this technique, because it appears to use the post-HLS RTL implementation to modify the behavior of the pre-HLS specification. From a formal point of view, the important observation is that the pre-HLS model is intentionally under-specified with respect to micro-timing/latency: subject to the R-rules, B-rules, weak fairness, and the happens-before relation on Σ (committed IO events; unsuccessful non-blocking polls are ε-steps), it can admit multiple ε-/latency-different executions that respect the same happens-before constraints. This is compatible with B1, which asserts that the post-HLS Σ-projected trace is unique under a fixed stimulus; the pre-HLS side may still have multiple admissible realizations that (when projected to Σ) match that unique RTL Σ-trace.

Snooping does not change what executions are allowed by the pre-HLS specification. Instead, it selects—purely for verification purposes—one admissible pre-HLS execution whose latency-sensitive

events align with those observed in the RTL. In other words, the implementation is used to pick a witness execution of the specification, not to redefine the specification.

Experienced SoC architects tend to view this slightly differently, in terms of design tradeoffs and verification cost.

First, it is usually possible at the architectural level to avoid latency-sensitive behaviors entirely, for example by insisting that all communication be latency-insensitive and fully synchronized. However, the quality-of-results (QOR) costs of such designs may be unacceptable. Introducing latency-sensitive behavior—non-blocking arbiters, latency-sensitive global interrupt signals, and so on—is a deliberate choice made by the designer to meet performance, power, or area goals.

Second, in many practical systems, latency-sensitive behavior is not functionally observable at the DUT boundary under the equivalence relation ≈ with Σ instantiated to include only the DUT-boundary observables (Appendix G; Common Formal Definitions above). As an example, consider a DUT that contains a single-port RAM used to exchange data between two internal processes. An arbiter is required to control access to that RAM port, and the arbiter uses latency-sensitive non-blocking Pop operations on its request channels. During HLS, internal latencies will change, so the order in which the arbiter sees pending requests and chooses winners may differ between the pre-HLS and post-HLS models.

In this example:

- The individual RAM read/write operations and the internal arbitration decisions are not directly visible at the DUT interface.
- Only the higher-level IO of the two processes (for example, their message-passing interfaces to the rest of the system) is externally visible.

One might initially conclude that it is necessary to snoop the arbiter's inputs to make the pre-HLS and post-HLS traces match. However, the modeling rules impose two additional constraints:

1. Weak fairness for the arbiter. The arbiter must satisfy the weak fairness assumptions, so that any request that remains pending is eventually granted in both the pre-HLS and post-HLS models.
2. Happens-before discipline on shared memory. The system must enforce a happens-before relation on shared-memory accesses, ensuring that sufficient synchronization exists between the two processes so that there are no read/write races through the RAM in either model.

Under these conditions, the different arbitration orders merely change the relative timing of internal operations and of causally independent external events. They do not change the functional ordering of causally dependent observable actions at the DUT boundary. In the terminology of Appendix G, the differences are incidental variations in latency rather than changes to the happens-before partial order, and therefore they are not considered observable differences in behavior at the DUT boundary. In such cases, snooping is not required. (See also Note 1 below).

(More precisely: the equivalence relation ≈ is parameterized by the chosen observable alphabet Σ. If internal functionality (such as the above RAM arbiter) uses channels/signals that are not designated observable in the chosen instantiation of Σ (see Common Formal Definitions), then mismatches on those internal actions are outside ≈ by construction: they are not in Σ. If those internal channels/signals are designated observable (e.g., by snooping for debug, or by expanding Σ for Appendix K deadlock analysis),

then they are Σ-events and must match under ≈. In addition, even when B1 holds (unique post-HLS Σ-trace under fixed stimulus), the pre-HLS model may still admit multiple ε-/latency-different executions consistent with the same happens-before constraints; Theorem J.1 captures the intended quantification ("existence of a matching witness execution prefix") rather than requiring a single pre-chosen trace.)

Experienced SoC architects therefore try to avoid designs in which latency-sensitive internal behaviors leak directly into the observable behavior of the system under test, since this both complicates verification and makes RTL behavior less predictable. When they do introduce such behaviors—examples include non-blocking arbiters whose winners affect externally visible ordering, global interrupt request signals at the DUT boundary—they typically know exactly where those latency-sensitive interfaces are and can isolate them.

A useful analogy is a subsystem whose clock frequency is adjusted dynamically based on on-die temperature. As temperature changes, the externally visible behavior of the SoC can change (for example, in terms of throughput or timing of events), and designers may choose such a temperature-dependent scheme to meet overall system requirements. To verify such a system, we may wish to compare a concrete RTL trace captured at a specific temperature profile with a higher-level reference model. To do so, the reference model must be driven with the same temperature evolution as the RTL saw. If that temperature profile is not otherwise under direct control, the simplest way to obtain it is to "snoop" the temperature as observed in the RTL trace and replay it into the reference model.
In this analogy, temperature is an external parameter of the environment rather than a fundamental part of the functional specification, but it still influences observable behavior. Snooping simply provides a mechanism to recover that parameter from the implementation when it cannot be easily prescribed a priori. Similarly, snooping of latency-sensitive IO provides a mechanism to supply implementation-specific latency choices—subject to the fairness and happens-before constraints—back to the pre-HLS specification so that the two models can be compared under a common environment.

Note 1 (Unobservable non-blocking micro-timing).
Safety / functional equivalence. The equivalence relation ≈ is parameterized by the chosen observable alphabet Σ (Appendix G). If, for safety-only checking, we instantiate Σ to include only DUT-boundary observables (Σ_DUT) and we assume that any latency-sensitive internal behavior is not visible at that boundary under ≈—i.e., it may change only internal micro-timing, but it does not change the values, ordering, or presence of Σ_DUT events—then snooping of those internal latency-sensitive interfaces is not required for proving Σ_DUT-safety. Any mismatches on actions outside Σ_DUT are outside ≈ by construction.
Liveness / deadlock analysis. For message-passing deadlock analysis (Appendix K WFG), any latency-sensitive interface whose ε-level choices can affect whether a blocking Push/Pop is enabled, or can create/remove a wait-for dependency without an intervening committed Σ-event, must be accounted for. Practically, the default is to snoop and replay every such latency-sensitive (typically non-blocking) arbitration/probe interface in the DUT. Snooping can be avoided only if the design is certified "WFG-inert" with respect to that interface: it cannot change WFG edges except via committed Push_c/Pop_c (or explicit synchronization) events.

Stress Testing Pre-HLS Models for Latency Robustness

The formal equivalence results in Appendices G–K establish that, under the scheduling rules and fairness assumptions, the post-HLS RTL is trace-equivalent to the pre-HLS model at all observable interfaces.

These results implicitly assume that the pre-HLS specification is *well-formed*: it must not rely on incidental, implementation-specific timing alignments for functional correctness. Designs that do rely on such incidental alignments fall outside the formal guarantees.

Designs that use non-blocking message-passing operations (e.g., PushNB/PopNB, ac_channel nb_read/nb_write) or latency-sensitive global signals are particularly vulnerable to this kind of fragility. For such designs, it is strongly recommended to subject the pre-HLS model to aggressive *latency stress tests* in which message and handshake latencies are varied across executions. The intent is to validate that the design behaves correctly across the range of weakly fair schedules permitted by the modeling rules, not just under one convenient execution order.

More concretely, verification should check that:

- Causal dependencies are explicit. All functionally significant "happens-before" relationships are enforced via message-passing, SyncChannel operations, or explicit handshake protocols, rather than by relying on the incidental execution order of concurrent processes. In the terminology of Appendix G, the design shall not depend on the ordering of causally independent events for correctness.
- Weak fairness does not hide latent deadlocks or starvation. The design continues to make progress under adversarial but *weakly fair* scheduling—i.e., enabled actions may be delayed arbitrarily long but not forever—consistent with the B2/B3 obligations on the scheduler and environment summarized in Appendix N.

If the pre-HLS model fails under such randomized-latency stress scenarios, then the specification itself is outside the formal model of this document: it violates the design rule that well-formed systems must not rely on the relative ordering of causally independent events. In that situation, the equivalence theorems still apply to well-formed traces, but they no longer guarantee that the "golden" specification is robust under the full range of latency variations allowed by the environment.

The Matchlib library provides practical mechanisms to automate these stress tests in pre-HLS simulation, including:

- Random stall injection on message-passing channels, to simulate variable communication delays while preserving rendezvous semantics.
- Latency and capacity back-annotation, to model specific per-channel buffer depths and delays, including the capacities B(c) chosen during HLS.

Worked examples of this methodology are provided in Catapult Matchlib examples 60* and 72*, which illustrate how to configure randomized stalls and back-annotated latencies when validating that a design remains well-formed and latency-robust.

Appendix M – Document Abstract

This document defines a precise user-level scheduling model for high-level synthesis (HLS) that enables system-level verification and debug to be carried out almost entirely on the pre-HLS SystemC model, while still permitting aggressive RTL optimizations in the synthesized design. It introduces a small set of uniform rules governing three classes of I/O operations—message-passing channels, signal I/O, and

explicit synchronization—and organizes them into a “basic conceptual model” that preserves source-order synchronization, anchors signal reads and writes to their nearest synchronization points, and constrains the reordering of message-passing operations so that HLS cannot introduce new deadlocks.

These rules are then extended to pipelined loops, shared and external memories (via an explicit array-access mapping layer and conflict-free reordering), and “direct input” pragmas that allow stable or periodically synchronized signals to bypass unnecessary internal storage while maintaining equivalence between pre- and post-HLS behavior.

The methodology is latency-insensitive by construction but accommodates latency-sensitive islands such as cycle-accurate transactors, non-blocking arbiters, and one-way handshake protocols through encapsulation and carefully specified synchronization schemes. The document also provides concrete modeling guidelines (e.g., rules for placing signal reads/writes around wait statements, coding of rolled loops with signal I/O, and use of Matchlib Connections and SyncChannel) that allow digital verification engineers to write a single testbench in SystemVerilog UVM or SystemC/C++ and reuse it across both pre-HLS and post-HLS models with “no surprises.”

A major contribution is the formalization, in Appendix G and subsequent appendices, of a trace-equivalence relation between the pre-HLS source model and the post-HLS RTL, expressed as partial-order constraints on observable I/O actions and encapsulated in equivalence rules E1–E5. For channels whose RTL implementations introduce finite buffering, the correspondence is stated against a source-level bounded-FIFO interpretation Sys_B whose capacities B(c) match RTL_B; the rendezvous case B(c)=0 is recovered as a special case.

These proofs are compositional (per process and system-level), incorporate weak fairness and bounded-FIFO assumptions, and cover key HLS transformations including loop pipelining, added FSM states, memory-access reordering, and configurable channel buffering. Collectively, the scheduling rules, coding guidelines, and formal guarantees provide a tool-agnostic, Catapult-compatible foundation for industrial HLS use and a concrete starting point for standardization efforts within bodies such as the Accellera Synthesis Working Group.

Keywords:
High-level synthesis (HLS); SystemC; scheduling rules; latency-insensitive design; message-passing channels; signal I/O; loop pipelining; direct input pragmas; shared memory; trace equivalence; formal verification; bounded FIFOs; Catapult HLS; Matchlib; Accellera Synthesis Working Group.

Appendix N - Scheduler and Environment Fairness Assumptions

The formal results in Appendices I–K rely on several semantic assumptions about the post-HLS scheduler and its environment:

- B2 (Weak fairness of the scheduler). If the enabling predicate for an action (Push, Pop, Sync) remains continuously true from some cycle onward, the scheduler must eventually select that action within a finite, but unspecified, number of cycles.

- B3 Channel Progress Invariants (ready/valid form).
  For every channel c with capacity B(c), interpret "continuously enabled" in terms of continuous request, not as an immediate commit.
  (No deassert-before-commit assumption.) For blocking Push_c/Pop_c transfers, these requests are non-withdrawable: once asserted for a transfer attempt, they are not deasserted prior to the corresponding commit (and Push_c payload remains stable while vld_c is asserted).
  Let vld_c denote the producer's persistent request to transfer (e.g., valid=1 with a stable payload for a Push_c), and let rdy_c denote the consumer's persistent request to transfer (e.g., ready=1 for a Pop_c).
  • P_push(c): If vld_c remains asserted continuously from some cycle t_o onward while the channel is full (occ_c = B(c)), and the complementary endpoint process continuously requests the matching Pop_c (i.e., rdy_c remains asserted continuously, with the Pop_c boundary request (i.e., BoundaryReq(Pop_c,t) remaining true), then some Pop_c commit must eventually occur (i.e., the full condition is eventually discharged). Equivalently: when both endpoints continuously request the transfer, the system cannot remain stuck forever in the full state without a Pop_c commit.
  • P_pop(c): If rdy_c remains asserted continuously from some cycle t_o onward while the channel is empty (occ_c = 0), and the complementary endpoint process continuously requests the matching Push_c (i.e., vld_c remains asserted continuously with stable payload, with the Push_c boundary request (i.e., BoundaryReq(Push_c,t)) remaining true), then some Push_c commit must eventually occur (i.e., the empty condition is eventually discharged). Equivalently: when both endpoints continuously request the transfer, the system cannot remain stuck forever in the empty state without a Push_c commit. (For B(c)=0 rendezvous channels: if both endpoints continuously request the transfer from some cycle t_o onward (both requests persistent; guards remain true), then the rendezvous completion eventually occurs.)
  These obligations rule out "permanent back-pressure loops" that are logically independent of the local R-rules.
- B5 (System quiescence closure). If at some cycle no observable actions are enabled in any process, then within a bounded number of cycles the system reaches a fixed point and executes no further ε-steps. This prevents a dead, all-disabled state from being hidden behind an infinite tail of internal activity.

These B-rules are assumptions on the execution environment, not guarantees automatically provided by the HLS tool. In practice they place concrete requirements on arbiters, back-pressure, and external masters that interact with the post-HLS RTL.

Priority arbiter example: fair vs unfair priority

Consider a simple two-input priority arbiter inside the post-HLS RTL. Each input is driven by a message-passing channel; the arbiter issues Pop operations to select which request to serve in each cycle.

- Input 0: high-priority channel c_high
- Input 1: low-priority channel c_low

Both channels may have pending requests at the same time.

1. Fair priority arbiter (satisfies B2 / B3).

A fair implementation might still give c_high strict priority in *individual* decisions, but it ensures that a continuously pending c_low request cannot starve. In RTL terms, this can be realized by, for example:

- A round-robin or rotating-priority arbiter that periodically moves the "highest" priority to the next requester, or
- A bounded-burst priority scheme in which c_high may win at most N consecutive cycles while c_low is requesting; after that, the next grant is forced to c_low.

Under these implementations:

- If Pop_low's enable remains true indefinitely (the low-priority channel has a pending request and downstream space is available), the scheduler must eventually grant Pop_low. This satisfies B2 for that action.
- If the low-priority FIFO is full and keeps requesting service, the system ensures that some Pop_low eventually fires, discharging data and freeing space. This is consistent with P_push/P_pop in B3.

Intuitively, the arbiter is still "priority-based," but its micro-architecture ensures that every continuously enabled requester is eventually served.

2. Unfair priority arbiter (violates B2 / B3).

A more naive design might implement:

```
if (req_high) grant_high();
else if (req_low) grant_low();
```

combined with an environment in which req_high can remain asserted indefinitely. In this case, even if req_low is also continuously asserted:

- Pop_low's enabling predicate is true forever, but it is never chosen by the scheduler.
- Low-priority requests can starve indefinitely, even though they are logically ready to fire.

This behavior violates B2 (weak fairness of the scheduler). If c_low's FIFO is full and the only way to make space is to service it, permanent starvation also violates B3's progress expectations for that channel.

In such a design, the safety properties E1–E5 may still hold (trace-equivalence on actions that *do* occur), but the liveness/results that depend on B2/B3—especially the starvation-vs-deadlock arguments in Appendix K—no longer apply.

Patterns that typically satisfy the fairness assumptions

The following design patterns normally satisfy B2 and are compatible with B3/B5, provided the rest of the system is well-behaved:

- Round-robin or rotating-priority arbiters. Any requester that remains enabled will eventually be at the front of the rotation and will be granted.
- Age-based or credit-based arbiters. Requesters accumulate age or credits while waiting; the arbiter prefers older or more-starved requests, guaranteeing that long-pending actions eventually win.
- Bounded-burst fixed priority. Fixed priority is combined with a counter that limits how long a higher-priority requester can dominate while lower-priority requesters are pending. After the burst limit is reached, lower-priority requests are forced to win until the system is "caught up."
- Back-pressure with bounded stall. When ready/valid or similar handshake signals are used, the design (or its environment) must enforce that ready cannot remain low forever while valid remains high, and vice versa. For example:
  - Downstream consumers are required (by specification) to service queues at least once every N cycles when data is present.
  - External bus masters are configured such that they cannot indefinitely defer reading from full DUT FIFOs that are logically part of the interface contract.
- Clock-gating and power-management schemes that respect B5. Once no observable actions are enabled, the design either:
  - Quietly stabilizes (no further internal toggling), or
  - Explicitly enters a low-power state from which it only wakes when some observable action again becomes enabled.

In each case, the intent is the same: continuous enable implies eventual service, and once nothing is enabled, the system converges rather than oscillating internally.

Patterns that tend to violate the fairness assumptions
Conversely, the following patterns are likely to break B2/B3/B5 and therefore fall outside the scope of the liveness arguments in this document:

- Pure fixed-priority arbitration with unbounded high-priority traffic. A static priority tree with no rotation or aging, combined with workloads in which a high-priority master can keep its request asserted indefinitely, can starve lower-priority channels forever.
- "Best-effort" external masters with no progress guarantee. For example:
    - A software driver that reads from a DUT output FIFO only when it happens to poll, and that may be pre-empted indefinitely by higher-priority threads.
    - An external bus or DMA engine that is architecturally allowed to ignore some requesters forever if higher-priority traffic remains heavy.
      Such environments can violate P_push/P_pop by allowing FIFOs to remain full or empty indefinitely while the DUT is continuously requesting progress.
- Circular back-pressure loops with no escape. Two or more processes form a cycle in which each is waiting for the other's channel to change state, and no other process can break the cycle. Without an explicit design-level guarantee that some process in the loop will eventually act (for example, by dropping priority or issuing a compensating Pop/Push), B3 is not satisfied.
- Internal oscillation without quiescence. A scheduler or control FSM that can toggle internal ε-level state forever, even after all observable actions are disabled, violates B5. While such designs are uncommon in practice, patterns involving mis-configured clock gating, asynchronous feedback, or "keep-alive" timers that never expire can have this effect.

In all these cases, the trace-equivalence theorems still describe what happens when actions do occur, but the liveness claims that depend on B2/B3/B5 (for example, "a continuously enabled channel will not starve") are no longer guaranteed. Designers and verification engineers should therefore either:

- Architect their arbiters and environments to satisfy these B-rules, or
- Treat the liveness guarantees in Appendix K as *out of scope* for those particular interfaces, and rely only on the safety-oriented parts of the model.

Appendix O - Support for Multiple Clock Domains (Informative Sketch)

This appendix is an informative sketch of how the single-clock formalism of Appendices G–K could be organized in a multi-clock design. The formal results of Appendices G–K are stated for a single global clock unless an explicit multi-clock semantic layer is added.

Within a single clock domain d, one may treat that domain as a synchronous island with a local cycle counter clk_d and apply the existing R- and E-rules to processes wholly inside that domain, interpreting clk(·) as clk_d(·) for local synchronization, signal I/O, and message-passing events. Inter-domain communication should be represented by explicit clock-domain-crossing components, such as CDC FIFOs, whose source-side and destination-side commit semantics are specified at their respective clock-domain boundaries.

A full formal lift requires additional definitions not supplied by Appendices G–K themselves: a global event order or partial order relating local-domain cycles, precise CDC commit semantics, occupancy

sampling rules across source and destination clocks, synchronizer fairness/metastability abstraction assumptions, and progress obligations for CDC components. Once those additional definitions are supplied, the intended proof strategy is to treat each synchronous island using the existing single-domain rules and to treat each CDC component as an explicit channel/refinement boundary with its own stated E4-style legality and progress obligations. Without those additional definitions, Appendix O should not be read as claiming that the single-clock proofs apply unchanged to arbitrary multi-clock systems.

Appendix P - AI Discussion of Alternative Formal Frameworks

The following links provide an AI discussion of alternative formal frameworks compared to the one presented in this document:

https://drive.google.com/file/d/1ccU-eRW7H2ggh-AZyKnCOzO-KLPejob_/view?usp=sharing

https://chatgpt.com/share/69457119-ec30-8006-8684-9a2a8b19f96f

---

Appendix Q – Multiple Input Pipelines Back-Annotation

Appendix J Remark 2 discusses back-annotation of pipelined loops with a single message passing input. In more complex cases such as HW pipelines with multiple message-passing inputs, the back-annotation mechanism may elaborate the Sys_B (source-level) simulation by introducing additional internal realization structure. In this document, we adopt the following simplified modeling discipline:

1. Terminology (elaboration and channel accounting). In an elaborated realization, the quantities B(·), occ_·(t), and the E4 availability predicate are interpreted over the explicit abstract channels present in the elaborated model (including any introduced staged/bundle channels). When we continue to write a source-level channel name c at the outer DUT/testbench observation boundary, we mean the chosen outer abstract boundary channel associated with c in that elaboration; downstream staging that is reachable only through a join (and therefore may be withheld when “join not met”) is represented by distinct internal channels with their own B(·) and occ_·(t), rather than as independent “slack” for c in isolation.
2. All buffering / capacity effects are represented explicitly as finite-capacity message channels (annotated with bounded capacity) that may appear upstream and/or downstream of any internal glue logic; and
3. Any internal “coupler” used to enforce mutual-stall between multiple inputs is purely combinational (stateless) handshake logic.

Observation-boundary convention for Appendix Q. In elaborated cases, distinguish the outer projected observation boundary from the proof-internal abstract boundaries. The outer Σ_DUT / testbench projection records only the chosen outer Push_c / Pop_c history after applying Π_elab. By contrast, Appendix K WFG reasoning is carried out over the explicit abstract channels of Sys_ref = Sys_B^elab, including introduced staged/bundle/join channels. Thus an introduced internal channel may be hidden by Π_elab from the outer DUT-visible trace while still being an explicit proof-internal boundary with its own B(·), occ_·(t), BoundaryReq, ChannelEnabled/ChannelDisabled, Front_P, and WFG facts.

Concretely, a combinational coupler is a stateless component that (a) reads the upstream channels' valid and data signals and computes/drives the downstream channels' valid and data signals, and (b) reads downstream ready signals and upstream valid signals and then computes/drives upstream ready signals. The coupler contains no internal state and has no separate credit/capacity state; any shared pipeline capacity, per-input staging, skid/elastic buffering, or credit-like effects are modeled only by the explicit finite-capacity channels inserted by the back-annotation realization.

Example (two inputs, staged Pops, II=1, latency=4):
Consider a 4-stage RTL pipeline that (i) performs Pop_c1 in stage 1, (ii) performs Pop_c2 in stage 2, and (iii) performs Push_cout in stage 4, with II=1. To model the pipeline's internal staging and the stage-2 *input coupling* in the source-level Sys_B simulation while keeping the same Σ boundary, the back-annotation elaboration may introduce the following internal realization structure:

- An explicit bounded channel F1 of capacity 1 on the c1 path to represent stage-1 in-flight storage of values that have been accepted from c1 but have not yet reached the stage-2 join point.
- A purely combinational coupler/join at the stage-2 boundary that enforces mutual stall: it allows a transfer to proceed only when (a) a token is available from F1, (b) a token is available from c2, and (c) downstream staging has space; it then presents the paired value(s) to the downstream staging.
- An explicit bounded channel F34 of capacity 2 after the coupler to represent the in-flight capacity of stages 3–4 for the joined/paired work.

This example illustrates why multi-input pipelines cannot always be captured by a single per-channel capacity number in isolation: c1 and c2 do not have independent "slack" because acceptance across the stage-2 join boundary (the paired transfer that consumes a token from c2 together with the staged token from F1) is coupled to availability of the corresponding stage-1 value from c1.
All buffering/capacity resides in the explicit bounded channels (F1, F34, and any additional bounded stages the realization uses); the coupler itself remains stateless combinational handshake logic.

This elaboration is an implementation/refinement of the abstract bounded-FIFO channels at the chosen observable boundary; it does not change the formal interface (Σ) or E4's per-channel FIFO meaning at that boundary. Concretely, any back-annotation elaboration (including extra FIFO stages and/or combinational couplers) must satisfy:
• No new DUT-boundary observables: It introduces no new Σ-observable actions at the chosen DUT boundary, and it does not change the labeling of existing Σ-events (Push_c(v), Pop_c(v), etc.).
• Boundary/E4 preservation: At the chosen observable boundary, each Σ-visible committed Push_c/Pop_c event still denotes a committed transfer on the same abstract channel c, with per-channel FIFO order and the same E4 capacity/enablement interpretation under the annotated B(c) and occ_c(t); any additional internal channels introduced by the elaboration (e.g., staged/bundle channels) are separate abstract channels in the elaborated model with their own B(·), occ_·(t), and E4 availability predicates.
• No double-counting of internal staging: internal transfers within the concrete realization that contributes to the effective capacity (extra FIFO stages, skid/elastic buffering, FIFO→pipeline-stage handoffs, movements across internal staged channels, and value propagation through combinational couplers) are ε-steps and do not create additional Σ-visible committed Push_c/Pop_c events beyond the single boundary Push/Pop events.
• Conservation / projection of internal staging (normative): For each Σ-visible abstract channel c at the chosen observable boundary, the elaborated realization shall admit a fixed projection/accounting map

from the occupancies of the explicit internal channels introduced by the elaboration (including staged/bundle/join channels, as applicable) to the boundary occupancy occ_c(t). This projection/accounting map is part of the realization and witnesses that the internal occupancy bookkeeping is only a refinement of the boundary model: (i) every ε-step internal transfer preserves the projected boundary occupancy for each Σ-visible channel, and (ii) each Σ-visible committed Push_c/Pop_c updates the projected boundary occupancy exactly as required by E4 (equivalently, exactly as the boundary occ_c(t) defined from Σ-visible boundary commits). For bundled/joined work items, the projection may assign one internal token to multiple Σ-visible channels as specified by the realization. Therefore, the internal capacities/occupancies B(·), occ_·(t) do not introduce an additional independent Σ-visible state; they only refine the same bounded-FIFO accounting already represented at the chosen boundary.
• Multi-input pipelines / combinational couplers (mutual stall + explicit capacity):
To preserve per-channel E4 at the Σ boundary, any downstream staging that is reachable only through a join (and therefore may be withheld when "join not met") shall be modeled as distinct internal finite-capacity channels with their own B(·) and occ_·(t), rather than being treated as additional independent "slack" for an upstream Σ-visible channel in isolation; the back-annotation elaboration may use combinational couplers to enforce the intended mutual-stall behavior when some required inputs are unavailable. Any "shared capacity" or "distributed staging" effects needed for such a pipeline shall be represented only by explicit finite-capacity channels in the realization (e.g., bounded staging FIFOs, bounded slot/credit channels, bounded bundle channels), not by coupler state. The coupler itself remains stateless combinational handshake logic and does not redefine the per-channel E4 capacity/availability predicates at the chosen observable boundary.

Projection/accounting invariant for Sys_B^elab (normative). Any elaborated reference model Sys_B^elab used for multi-input / coupler cases shall come with a projection map Π_elab from the explicit staged/bundle/join channels of the elaboration back to the chosen outer Σ-visible channel boundaries. This projection map has two roles:

• Observable-boundary preservation. When the internal staged/bundle/join events introduced by elaboration are hidden or projected away, the remaining outer-boundary Push_c(v) / Pop_c(v) history is the same boundary history that the non-elaborated model would expose at the chosen Σ-visible endpoints. Elaboration may add explicit internal abstract channels for proof and WFG purposes, but it may not create extra outer-boundary Push_c / Pop_c events, drop outer-boundary events, duplicate them, or change their payload labels.

• Capacity/occupancy accounting. For each outer channel c represented by an elaborated subnetwork, the outer-boundary availability facts used by E4 are preserved by the aggregate accounting of the explicit internal channels in Π_elab^{-1}(c). Internal staged/bundle/join occupancies may refine where storage and blocking occur, but after projection they must implement the same externally visible capacity/acceptance behavior at the chosen boundary. Conversely, any cross-channel dependency or join condition that can block progress must appear as ordinary BoundaryReq / ChannelEnabled / ChannelDisabled facts on some explicit internal abstract channel of Sys_B^elab, not as a hidden extra predicate on an otherwise E4-enabled outer-boundary operation.

All examples and constraints in Appendix Q are to be read relative to this projection/accounting invariant.

Explicit join boundary requirement (normative): The mutual-stall point shall be an explicit

staged/bundle/join boundary inside the elaborated realization (i.e., the endpoint of an explicit finite-capacity message channel). "Join not met" shall therefore be expressed as ordinary ChannelDisabled at that explicit internal boundary (because that boundary's own E4 availability predicate fails in the ε-quiescent handshake fixed point). The realization must not force a Σ-visible operation on an abstract channel c at the chosen boundary to be ChannelDisabled solely due to the state of some other channel; any cross-channel dependency shall be expressed only via the explicit staged/bundle channels introduced by the elaboration. Operationally, the coupler is invisible at the outer DUT/testbench observation boundary: it only propagates ready/valid between the explicit staged/bundle channels inside the elaborated realization, and it does not change the projected outer occ_c(t), which is defined solely by projected outer-boundary commits under Π_elab. This invisibility is only with respect to the outer projection; it does not hide any explicit staged/bundle/join channel from Appendix K's proof-internal WFG boundary.

• WFG compatibility (Appendix K): The realization must not introduce a new blocking point that is invisible to the proof-internal abstract boundary of Sys_B^elab. In particular, if the consuming process cannot advance the pending blocking frontier Front_P(σ) (Appendix K.1), then at least one blocking frontier operation (Push/Pop) at either the chosen outer boundary or an introduced explicit staged/bundle/join boundary is asserted with a true boundary request and is channel-disabled by the K.1 enablement test; the process is therefore "blocked on message passing" exactly as defined in K.1. Because couplers are purely combinational, they have no internal state transitions; any changes in readiness/validity induced by the coupler are purely a function of (i) downstream readiness and (ii) the same channel occupancies / Σ-visible committed transfers already accounted for by E4/K.1, with "downstream readiness" interpreted in the ε-quiescent (ε-normalized) state used for WFG evaluation. Pure internal data movements that do not change any proof-internal BoundaryReq, ChannelEnabled/ChannelDisabled, Pending_P, or Front_P fact are ε-steps and are WFG-inert under the same pending-blocking-frontier / Front_P(σ) discipline used elsewhere in Appendix K. By contrast, a staged/bundle/join channel boundary that can block progress is not hidden from Appendix K; it is part of the explicit Sys_B^elab proof boundary, even if it is projected away from the outer Σ_DUT trace by Π_elab.
• Combinational well-formedness: The network of combinational couplers and ready/valid connections shall be combinationally acyclic (no pure combinational ready/valid loops). Any required feedback that would otherwise create a combinational loop must pass through an explicit finite-capacity channel stage (and is therefore represented in the back-annotated realization and its effective capacities).

Accordingly, the formal development need not depend on the particular back-annotation construction, provided it yields an explicit source-side reference model Sys_ref = Sys_B^elab (Appendix J, Remark 2) equipped with a projection/accounting map Π_elab satisfying the invariant above. The elaborated model must respect E4, BoundaryReq, ChannelEnabled/ChannelDisabled, and the Appendix K blocking/WFG semantics at its explicit abstract boundaries, while its projection back to the chosen outer Σ-visible boundaries must preserve the externally visible Push_c / Pop_c history and aggregate capacity/occupancy behavior. In these cases the proofs do not target the raw outer-boundary Sys or the non-elaborated outer-boundary Sys_B directly; they target the explicit abstract channels of Sys_B^elab together with their Π_elab projection back to the outer observable boundary.

Appendix R – Compact Formal Core for Appendices B–K

R.1 Purpose and scope

This appendix packages the common formal core already used by Appendices B, C, J, and K into a compact definition/lemma/theorem form. Its purpose is only to factor out the shared semantic skeleton and finite-prefix cutpoint view of the formal story. All terminology, assumptions, and proof targets remain those of the main document. In particular, the prescribed source-side semantic target remains exactly the Appendix J, Remark 2 target: Sys_ref = Sys_B in the ordinary abstract back-annotated presentation using the case-split channel semantics of E4 (B(c)=0 as rendezvous, B(c)>0 as cycle-boundary non-fall-through bounded FIFO), and Sys_ref = Sys_B^elab in the elaborated/back-annotated / coupler cases. Thus the proofs here do not introduce any proof target separate from the Appendix J target family.

This appendix should therefore be read as a compact companion to the existing appendices, not as a replacement for the full modeling assumptions, witness constructions, fairness/progress premises, or elaboration conventions already stated there. Appendix B continues to supply the automatic-flush / drain-only blocking discipline, Appendix C the Issue/Commit linkage, Appendix I the witness construction machinery, Appendix J the full execution-level / cutpoint-correspondence results, Appendix L the witness-selection / snooping wrapper where needed, Appendix N the cited progress premises, and Appendix K the wait-for-graph and deadlock-preservation layer. In particular, whenever any abbreviated wording in this appendix about Issue, BoundaryReq attribution, or continuously asserted requests could be read more broadly than Appendix C, the full Appendix C Issue/Commit linkage governs.

R.2 Prescribed reference model

Definition R.1 (Prescribed source reference model Sys_ref).

For a fixed design and fixed external stimulus, let Sys_ref be exactly the prescribed source reference model of Appendix J, Remark 2:

- Sys_ref = Sys_B in the ordinary abstract back-annotated presentation using the case-split channel semantics of E4: B(c)=0 is rendezvous, and B(c)>0 is cycle-boundary non-fall-through bounded FIFO.
- Sys_ref = Sys_B^elab in the elaborated/back-annotated case.

All source-side semantic notions used below are interpreted with respect to this prescribed source reference model.

Remark R.1 (Interpretation at explicit abstract channels).

Whenever Sys_ref = Sys_B^elab, the quantities B(·), occ_·(t), E4, BoundaryReq, ChannelEnabled, ChannelDisabled, and the Appendix K blocking / wait-for-graph machinery are interpreted per explicit abstract channel, including any staged, bundle, or join channels introduced by elaboration. Any conjunctive/join dependency shall therefore be represented only via such explicit abstract channels, not as cross-channel disablement at the outer Σ-visible endpoints.

R.3 Observable alphabet and executions

Definition R.2 (Observable alphabet Σ).

Let Σ be the observable event alphabet consisting of:

(1) committed message-transfer events Push_c(v) and Pop_c(v),

(2) synchronization events, and

(3) anchored signal events Read(sig, v) and Write(sig, v).

Definition R.3 (ε-steps and compact trace notation).

Let ε denote internal micro-steps that are not externally observable at Σ, including internal staging/hand-off in back-annotated pipeline realizations, late direct-input sampling between anchors, and failed non-blocking polls.

A finite execution prefix τ is understood in the same sense as in Appendix J: it carries the inherited cycle

partition / per-cycle observable buckets of the execution, together with any ε-steps between successive Σ-visible cutpoints. For compact notation one may write a linearization over (Σ ∪ {ε}), but only modulo permutation of Σ-events that occur in the same cycle and are not additionally ordered by the document's explicit Σ-visible constraints. Accordingly, τ|Σ denotes the observable projection together with that inherited cycle partition; same-cycle observable events are not assigned any extra order merely by this compact notation.

Definition R.4 (ε-quiescent cutpoint).

A state σ is ε-quiescent if no further ε-step is enabled from σ. For any finite prefix τ, let $\bar{\tau}$ denote a maximal finite ε-extension to an ε-quiescent endpoint, preserving the same observable cycle partition / bucket structure on τ|Σ. The Appendix J / K state-comparison reasoning is always performed at such ε-quiescent cutpoints.

R.4 Source order and source-level frontier items

Definition R.5 (Per-process execution-call universe, frontier-item universe, observable projection, and source order).

For each process P, let Q_exec,P denote its execution-level set of dynamic source-level message-call instances for the particular execution / witness under the fixed external stimulus being analyzed. Q_exec,P includes blocking Push/Pop calls, successful non-blocking PushNB/PopNB calls, and failed non-blocking PushNB/PopNB polls.

Let Ω_P denote P's local dynamic source-level frontier-item universe for that same execution / witness. Thus Ω_P is stimulus-dependent: it contains exactly the local synchronization-call items, anchored signal-action items, and message-operation frontier items that belong to the realized execution being compared, and excludes items on untaken control-flow branches, unentered cases, and unexecuted loop iterations. Message-operation frontier items include reached blocking Push/Pop instances, whether pending or committed, and successful non-blocking PushNB/PopNB instances that contribute committed Push_c(v) / Pop_c(v) events. Failed non-blocking polls are ε-steps: they are members of Q_exec,P when executed, but they are not members of Ω_P merely because they executed.

Let Q_Ω,P := Q_exec,P ∩ Ω_P denote the message-operation slice of Ω_P. Let Σ_P denote the corresponding observable event/label projection: synchronization and signal items contribute their Σ labels when observed, and a message-operation item q ∈ Q_Ω,P contributes m(q) only if q commits. Pending blocking message-operation items may therefore be in Ω_P and Q_Ω,P without yet contributing any committed Σ-event. Let ≤_P denote the source-induced partial order on Ω_P, i.e. the order generated by the document's scheduling rules for synchronization calls, anchored signal actions, and message-operation frontier items. The execution-level order constraints E3/E5 use the corresponding source-induced order on Q_exec,P.

Definition R.6 (Done_P, Remain_P, and NextFront_P relative to the fixed witness).

Fix the particular execution / witness under analysis, hence the associated Q_exec,P, Ω_P, Q_Ω,P, Σ_P, and ≤_P from Definition R.5. For a state σ occurring as a cutpoint of that compared execution, let

Done_P(σ) ⊆ Ω_P

be the set of local frontier items already realized by σ. For synchronization and signal items, "realized" means that the corresponding Σ-event has occurred. For a message-operation frontier item q ∈ Q_Ω,P, "realized" means that q has committed and its committed Σ-event m(q) has occurred. A pending issued-but-uncommitted blocking message-operation item is therefore not in Done_P(σ), even though BoundaryReq(q,σ), ChannelEnabled(q,σ), and ChannelDisabled(q,σ) may be defined. A failed non-blocking poll q ∈ Q_exec,P \ Ω_P is not considered by Done_P, Remain_P, or NextFront_P, because it is an ε-step with no frontier item and no committed Σ-event.

Let
Remain_P(σ) := Ω_P \ Done_P(σ).
Then define the next source-level frontier-item set by
NextFront_P(σ) := min_≤P(Remain_P(σ)).
This is the Appendix J / Lemma S2 notion of the next source-level frontier-item set, interpreted relative to the fixed witness universe Ω_P. It may contain synchronization-call items, signal-anchored action items, and message-operation items. It is a frontier of source-level items, not a set of already-committed Σ-event labels.

Definition R.7 (Message-operation slice of the next frontier).
For the same fixed compared execution / witness, define
MsgNextFront_P(σ) := { x ∈ NextFront_P(σ) | x ∈ Q_Ω,P and kind(x) ∈ {Push_c, Pop_c} for some channel c }.
This is only the message-operation slice of the candidate next frontier; it is not yet the pending blocking frontier of Appendix B / K.

R.5 Commit, issue, and boundary requests

Definition R.8 (Commit).
For a message transfer at a chosen Σ-visible boundary endpoint, Commit(q) is the cycle in which an external clocked observer sees both valid and ready high simultaneously at that endpoint. The committed payload is the value seen in that same cycle.
Definition R.9 (Issue).
For a source-level message operation instance q at a chosen Σ-visible boundary endpoint, Issue(q) is the first cycle in which the calling process begins requesting the transfer corresponding to q at that endpoint. Issue(q) is an auxiliary scheduler/witness timestamp used for rules such as R4, E3, and E5.
Definition R.10 (Endpoint-owned boundary request).
Fix a Σ-visible endpoint direction. Let req(t) denote the endpoint-owned boundary request signal in cycle t, namely valid for a Push endpoint and ready for a Pop endpoint. The predicate BoundaryReq(q, σ) records that this endpoint-owned request is asserted for the operation q at state σ.
Axiom R.1 (Issue/Commit linkage).
Fix a Σ-visible endpoint direction. For source-level message operation instances on that endpoint direction, the following all hold:
(1) Boundary-request soundness / non-withdrawal.
If q is blocking, then once issued its endpoint-owned request remains asserted until q commits, with stable payload while asserted for Push.
(2) Stable attribution while outstanding.
If the request remains asserted across a cycle in which no transfer commits on that endpoint direction, then the continued assertion is attributed to the same outstanding q.
(3) Back-to-back issuance.
If q_o and q_1 are same-direction source-level message operation instances, q_o commits in cycle t, and q_1 is the next such operation issued back-to-back within the same source interval, then Issue(q_1) = t + 1, even if the request signal does not deassert between commits.
(4) Sync-boundary discipline.
If q_o and q_1 are same-direction source-level message operation instances and q_o ∈ pref_S(s) and q_1 ∈ suff_S(s) for a synchronization call s, then Issue(q_1) occurs strictly after the commit of s. Any request assertion that persists while the process is still pending at s is attributed to the outstanding operation(s) in pref_S(s), not to issuance of any operation in suff_S(s).
Remark R.2 (Non-blocking polls).

For non-blocking message passing, a failed poll produces no Σ event and is modeled as an ε-step, while a successful poll contributes the corresponding committed Push_c(v) or Pop_c(v) event. Each executed PushNB/PopNB call is nevertheless a distinct dynamic source-level instance q ∈ Q_exec,P for its owning process P; thus repeated failed polls in later loop iterations are distinct q's even if the endpoint request remains continuously asserted. The ε-modeling concerns observability in Σ, not the dynamic-instance identity of the underlying source-level calls.

R.6 Channel enablement

Definition R.11 (ChannelEnabled / ChannelDisabled).
Let q be a source-level message operation instance on channel c, and write kind(q) ∈ {Push_c, Pop_c} for its endpoint direction. At an ε-quiescent cutpoint σ, let t_σ denote the cycle of that cutpoint. Define:
ChannelEnabled(q,σ) to mean that BoundaryReq(q,σ) holds and that the channel-side condition required for q to commit is satisfied after combinational settling at that cutpoint, and
ChannelDisabled(q,σ) to mean that BoundaryReq(q,σ) holds but ChannelEnabled(q,σ) does not.

Definition R.12 (Buffered channels).
If B(c) > 0 and q is a source-level message operation instance on c, then the buffered-channel enabling condition is:
• ChannelEnabled(q,σ) ⇔ BoundaryReq(q,σ) ∧ occ_c(σ) < B(c), when kind(q) = Push_c
• ChannelEnabled(q,σ) ⇔ BoundaryReq(q,σ) ∧ occ_c(σ) > 0, when kind(q) = Pop_c
Thus, if q is a buffered Push_c operation that is channel-disabled, then BoundaryReq(q,σ) holds and occ_c(σ) = B(c); and if q is a buffered Pop_c operation that is channel-disabled, then BoundaryReq(q,σ) holds and occ_c(σ) = 0.

Definition R.13 (Rendezvous channels).
If B(c) = 0 and q is a source-level message operation instance on c, then the rendezvous enabling condition is:
ChannelEnabled(q,σ) ⇔ BoundaryReq(q,σ) ∧ (the complementary endpoint request on c is asserted at σ).
Equivalently:
• if kind(q) = Push_c, then
ChannelEnabled(q,σ) ⇔ BoundaryReq(q,σ) ∧ (the complementary Pop_c endpoint request is asserted at σ)
• if kind(q) = Pop_c, then
ChannelEnabled(q,σ) ⇔ BoundaryReq(q,σ) ∧ (the complementary Push_c endpoint request is asserted at σ)
Hence, if a rendezvous action q is channel-disabled, its own BoundaryReq holds but the complementary endpoint is not simultaneously requesting the matching partner action at that cutpoint.

R.7 Pending blocking frontier

Definition R.14 (Pending blocking operations).
Fix an ε-quiescent state σ of process P within the current interval I_P(σ) between the adjacent synchronization calls that bracket σ in P's current execution. Define
Pending_P(σ) := { q | q ∈ Q_Ω,P is a blocking source-level message-operation item in that same interval I_P(σ), Issue(q) has occurred, and Commit(q) has not yet occurred in σ }.
By Appendix C's Issue/Commit linkage together with the definition of BoundaryReq, each member of Pending_P(σ) has its endpoint-owned request asserted at the cutpoint.

Definition R.15 (Pending blocking frontier Front_P(σ)).
Define
Front_P(σ) := min_≤P(Pending_P(σ))
If several ≤_P-minimal pending blocking message-operation items are simultaneously outstanding, they form a frontier. This is the Appendix B / K notion of the pending blocking frontier (or minimal pending blocking frontier). It is distinct from NextFront_P(σ), which is the broader source-level frontier-item set over Ω_P.

R.8 Automatic flush / drain-only blocking discipline

Definition R.16 (Automatic flush).
A pipelined implementation satisfies the automatic-flush discipline if, whenever Front_P(σ) is non-empty and every member of Front_P(σ) is channel-disabled after ε-settling, the following all hold:
(1) Block point.
P is blocked on message passing at Front_P(σ).
(2) No new later work.
While Front_P(σ) remains non-empty and fully disabled, no new later blocking message operation is issued.
(3) No withdrawal.
Already-asserted blocking requests are not withdrawn before commit.
(4) Frontier-minimality.
An already-asserted blocking request that is not in Front_P(σ) is not treated as a wait-for source unless and until it later becomes frontier-minimal.
(5) Drain-only downstream progress.
Work already past the blocked frontier may drain and commit when channel-enabled, but no new work advances past the blocked frontier while that frontier remains fully disabled.
(6) Sync-boundary capacity withdrawal.
While a process is pending at an explicit synchronization call s, producer-facing availability for any back-annotated capacity belonging only to suff_S(s) is withheld until s commits; work already accepted before s may still drain.
Remark R.3 (ALL disabled ⇒ stall).
The document's intended policy is exactly the "ALL disabled ⇒ stall" interpretation: the process stalls only when none of the operations in Front_P(σ) is channel-enabled. Downstream drain is compatible with that stall provided no new later work is issued past the blocked frontier.

R.9 Observable equivalence and cutpoint correspondence

Definition R.17 (System-level observable equivalence).
For a witness prefix τ_ref of Sys_ref and a post-HLS prefix τ_post of RTL_B, define
τ_ref ≈ τ_post :⇔ E1 ∧ E2 ∧ E3 ∧ E4 ∧ E5
This appendix treats E1–E5 as primitive named conjuncts, exactly as used by Appendix J.

Definition R.18 (Cutpoint correspondence ≡).
Let σ_ref and σ_post be ε-quiescent terminal states of matched prefixes. Write
σ_ref ≡ σ_post
iff all of the following hold:
(1) For every process P,
NextFront_P(σ_ref) = NextFront_P(σ_post).
(2) For every process P, the Σ-relevant source-level facts for that common source-level frontier-item set agree across the two cutpoints, in the following typed sense; clause (5) separately records the additional pending-request facts needed for Appendix K drain-only reasoning:

(a) for every x in the common message-operation-item slice
MsgNextFront_P(σ_ref) = MsgNextFront_P(σ_post),
the boundary-request predicate BoundaryReq(x,·) agrees;
(b) for every x in the common set NextFront_P, every source-level control predicate Pred_x(·) used to determine whether x is the next selected source-level frontier item agrees; and
(c) for every x in the common set NextFront_P, every source-level value / payload expression needed to determine x's eventual Σ-label, if any, agrees.
Equivalently, σ_ref and the source-level projection of σ_post agree on all Σ-relevant source variables read by BoundaryReq for the common message-operation-item slice MsgNextFront_P, by the next-item selection predicates for the common source-level frontier-item set, and by the payload/value expressions for that same common frontier.
(3) For each buffered abstract channel, the E4-relevant abstract FIFO state agrees, equivalently the empty/full status and the logical head information determined by the matched Push/Pop history agree.
(4) For each point-to-point rendezvous abstract channel c used in the Appendix K internal message-passing boundary / WFG analysis, the raw endpoint-request predicates agree across the two cutpoints:
Req_prod(c,σ_ref) = Req_prod(c,σ_post)
and
Req_cons(c,σ_ref) = Req_cons(c,σ_post).

This rendezvous clause is an explicit component of the cutpoint-correspondence relation σ_ref ≡ σ_post. It is intentionally not derived from NextFront_P agreement, from clause (2)'s agreement on the common next-frontier slice, or from membership in Front_P(σ). The reason is that Appendix B permits already-asserted non-frontier blocking requests to persist as protocol state while an earlier pending blocker remains frontier-minimal. Such raw endpoint-request state may be needed directly by the rendezvous enabling condition even when the asserted request is not itself a current frontier member. Clause (4) therefore records the required raw producer-side and consumer-side endpoint-request equality as part of the preserved cutpoint state.

(5) For Appendix K drain-only reasoning, already-issued blocking request state agrees at the chosen abstract boundaries. More precisely, for every process P and every dynamic blocking Push/Pop operation instance q in P's current interval between adjacent synchronization calls:
q ∈ Pending_P(σ_ref) iff q ∈ Pending_P(σ_post).

For every such common pending q:
• the endpoint-owned boundary request is attributed to q on one side iff it is attributed to q on the other side;
• BoundaryReq(q,σ_ref) = BoundaryReq(q,σ_post); and
• ChannelEnabled(q,σ_ref) = ChannelEnabled(q,σ_post), equivalently ChannelDisabled(q,σ_ref) = ChannelDisabled(q,σ_post).

For B(c)>0 channels, this enablement agreement follows from the common B(c) and the logical occupancy/head-state agreement of clause (3). For B(c)=0 rendezvous channels, it follows from the raw producer/consumer endpoint-request agreement of clause (4). In the elaborated case Sys_ref = Sys_B^elab, all of these Pending, BoundaryReq, ChannelEnabled, and ChannelDisabled facts are interpreted at the explicit abstract channels of Sys_B^elab.

R.10 Prefix-matching theorem

Theorem R.1 (Finite-prefix witness consequence of Theorem J.1)
Fix a finite post-HLS execution prefix τ_post ∈ Exec_post of RTL_B under a fixed external stimulus, where Exec_post is exactly the Appendix J execution-set scope under the same witness / fairness / progress premises used there. Then there exists a finite execution prefix τ_ref of Sys_ref under that same stimulus such that
τ_ref ≈ τ_post
This theorem is a compact finite-prefix consequence of Appendix J's full execution-level result, not a replacement for that full execution-level statement. In particular, the controlling hypotheses are exactly Appendix J's execution-set scope conditions together with the relevant Appendix I / Appendix L witness machinery and the cited fairness/progress premises. Appendix K uses only the existence of such a matching source-side witness at the compared cutpoint, not uniqueness of that witness.

Corollary R.1 (Cutpoint correspondence; compact finite-prefix form of Corollary J.1).
Let τ_post ∈ Exec_post be as in Theorem R.1. Let σ_post be the ε-quiescent terminal state obtained from τ_post under the same ε-normalization convention used in Appendix J, and let σ_ref be the ε-quiescent terminal state of any witness prefix τ_ref supplied by Theorem R.1, under the same source-side state-correspondence / projection hypotheses used by Corollary J.1. Then
σ_ref ≡ σ_post
Equivalently, under exactly the same Appendix J execution-set scope, ε-normalization, and state-correspondence assumptions used to obtain the witness, matched prefixes yield the same next source-level frontier-item set over Ω_P, the same BoundaryReq values on the common message-operation-item slice of that frontier, the same source-level next-item selection facts and Σ-label-determining values, if any, for the common frontier, the same E4-visible abstract FIFO state at the compared ε-quiescent cutpoints, and — for point-to-point rendezvous channels included in the Appendix K internal message-passing boundary / WFG analysis — the same producer-side and consumer-side raw endpoint-request predicates at those compared ε-quiescent cutpoints.

R.11 Explicit frontier-classification bridge

Definition R.19 (Derived blocking-frontier slice at ε-quiescent cutpoints).
At an ε-quiescent cutpoint σ and for a process P, define the asserted blocking message-operation-item slice of the next source-level frontier-item set by
FrontFromNext_P(σ) := { x ∈ BlockingMsgNextFront_P(σ) | BoundaryReq(x, σ) },
where
BlockingMsgNextFront_P(σ) ≜ { x ∈ NextFront_P(σ) | x ∈ Q_Ω,P and x is a blocking source-level message-operation item with kind(x) ∈ {Push_c, Pop_c} for some channel c }.

Here BoundaryReq(x,σ) is read operation-attributively for the specific source-level blocking message-operation item x, exactly in the Appendix C sense: the asserted endpoint-owned request at the cutpoint is attributed to x only if x is the currently outstanding requesting source-level operation instance on that endpoint direction. For blocking Push/Pop, that attribution cannot change before commit. Non-blocking PushNB/PopNB call instances are excluded from BlockingMsgNextFront_P: successful non-blocking calls may contribute Push_c/Pop_c Σ-labels, and failed polls are ε, but neither creates a pending blocking frontier member for WFG purposes. This definition is over source-level operation items in Q_Ω,P ⊆ Ω_P, not over already-committed Σ-event labels.

Remark R.4 (derived bridge, not extra premise).

Corollary R.1 aligns NextFront_P and the relevant BoundaryReq values only on the common message-operation-item slice of that frontier. No BoundaryReq predicate is defined or required for non-message frontier items. Appendix K.2.2 (Lemma K.0) then derives, from the Appendix B drain-only blocking discipline and the Appendix C Issue/Commit linkage, that this asserted blocking message-operation-item slice is exactly the pending blocking frontier.

R.12 Front / NextFront classification lemma

Lemma R.1 (Front/NextFront classification; compact restatement of Lemma K.0).
Fix an ε-quiescent state σ of either Sys_ref or RTL_B, and a process P. Then
Front_P(σ) = FrontFromNext_P(σ) = { x ∈ BlockingMsgNextFront_P(σ) | BoundaryReq(x, σ) }.

Consequently, if σ_ref ≡ σ_post, then
Front_P(σ_ref) = Front_P(σ_post),
and BoundaryReq(op, ·) agrees for every blocking message-operation item
op ∈ Front_P(σ_ref) = Front_P(σ_post).

This is a classification of the current pending blocking frontier only. It does not say that later same-interval requests cannot already have issued under pipelined overlap or downstream drain. Such already-issued later requests may persist as non-frontier protocol state under Appendix B's no-withdrawal and drain-only rules; they contribute to Front_P only if and when they later become frontier-minimal pending blocking operations.

Corollary R.1a (Rendezvous endpoint-request agreement; compact restatement of Corollary K.0a).
For a point-to-point rendezvous channel c with B(c)=0, define Req_prod(c,σ) to mean that the producer-side boundary request on c is asserted at the ε-quiescent cutpoint σ, and Req_cons(c,σ) to mean that the consumer-side boundary request on c is asserted at σ. Then, for any corresponding ε-quiescent cutpoints σ_ref ≡ σ_post,
Req_prod(c,σ_ref) = Req_prod(c,σ_post)
and
Req_cons(c,σ_ref) = Req_cons(c,σ_post).
This is a raw endpoint-request agreement fact only; it does not identify an asserted rendezvous request with frontier membership.

R.13 Wait-for graph refinement

Definition R.20 (Wait-for graph).
At an ε-quiescent cutpoint σ, define the wait-for graph WFG(σ) exactly as in Appendix K: its vertices are the modeled processes, and its directed edges are induced from the frontier members Front_P(σ) by the corresponding channel-disabled dependencies on internal message-passing channels only. Equivalently, a directed edge P → Q is present when some op ∈ Front_P(σ) is channel-disabled at the chosen boundary and the unique complementary endpoint needed to enable that op belongs to process Q. Here "internal" means that both endpoints of the channel are modeled processes of Sys_ref / RTL_B. Channels whose complementary endpoint is the external environment/testbench (the DUT boundary) are excluded from the WFG and from the internal message-passing deadlock notion used in Appendix K. Buffered and rendezvous channels are both included, with B(c), occ_c(σ), BoundaryReq, ChannelEnabled, and ChannelDisabled interpreted at the chosen ε-quiescent cutpoint on the chosen abstract channels of Sys_ref.

Lemma R.2 (Dependency refinement; compact restatement of Lemma K.2).

Let σ_ref and σ_post be corresponding ε-quiescent cutpoints with
σ_ref ≡ σ_post.
Then
WFG_ref(σ_ref) = WFG_post(σ_post).
Equivalently, for any processes P, Q, there is a wait-for edge P → Q in the post-HLS wait-for graph iff there is a wait-for edge P → Q in the source-reference wait-for graph. The proof uses Corollary R.1 for NextFront_P agreement, BoundaryReq agreement on the common blocking message-operation-item slice BlockingMsgNextFront_P, abstract FIFO-state agreement for buffered channels, and raw rendezvous endpoint-request agreement for point-to-point rendezvous channels; Lemma R.1 then lifts the next-frontier blocking message-operation-item agreement to Front_P. Corollary R.1a supplies the rendezvous endpoint-request predicate directly, without identifying asserted complementary requests with frontier membership.

R.14 Deadlock preservation

Theorem R.2 (Cutpoint-level closed-cycle preservation and one-way drain-closure transfer; local consequence of Theorem K.1).
Let σ_ref and σ_post be corresponding ε-quiescent cutpoints satisfying the Appendix K assumptions — including the ε-normalization requirement and the broader witness / fairness / progress premises inherited from the Appendix I / J / L / N stack used to obtain the compared cutpoints. Then

WFG_post(σ_post) contains a closed internal message-passing wait-for cycle
iff
WFG_ref(σ_ref) contains a closed internal message-passing wait-for cycle.

Moreover, for any non-empty set D of processes forming such a closed internal wait-for cycle, the drain-closed property of K.1 transfers in the post-to-reference direction already established by Lemma K.2a:

if σ_post is drain-closed for D, then σ_ref is drain-closed for D.

Thus, for the deadlock-preservation use case, a post-HLS cutpoint satisfying the full K.1 closed-cycle-plus-drain-closure condition maps to a corresponding source-reference cutpoint satisfying the same K.1 condition. No converse drain-closure transfer is claimed here.

This is the local cutpoint-preservation conclusion used inside Appendix K's broader liveness/deadlock argument. The full global liveness/deadlock theorem remains Theorem K.1. The present appendix records the preserved closed-cycle consequence at corresponding ε-quiescent cutpoints and, when the full K.1 deadlock condition is being used for a post-HLS cutpoint, records the one-way transfer of drain-closure for the same process set D from σ_post to σ_ref. The result is stated relative to the prescribed source reference model Sys_ref and uses the derived frontier-classification result of Lemma R.1, the direct rendezvous endpoint-request agreement of Corollary R.1a, and the same outstanding-request/channel-enable correspondence used by Lemma K.2a, rather than any claim that an asserted rendezvous request must be frontier-minimal or any extra frontier-classification precondition. In the elaborated multi-input / coupler cases, that source-side target is Sys_B^elab, not merely the raw outer-boundary Sys_B.

R.15 Summary of dependency structure

For convenient reference, the compact dependency chain relevant to this appendix's condensed presentation is:
(1) Appendix B defines Front_P(σ) and the automatic-flush / drain-only blocking discipline.

(2) Appendix C defines Commit, Issue, and the Issue/Commit linkage that makes pending blocking operations well-defined at Σ-visible boundaries.
(3) Appendix G states the system-level observable-equivalence rules E1–E5 and the high-level functional-indistinguishability implication on which the later witness/correspondence stack relies.
(4) Appendix I supplies the witness-construction machinery used to relate post-HLS behavior to the prescribed source reference model.
(5) Appendix J proves the full execution-level theorem for Sys_ref and, from that theorem together with the required ε-normalization / state-correspondence assumptions, yields the cutpoint correspondence σ_ref ≡ σ_post used here in compact finite-prefix form.
(6) Appendix Q supplies the elaboration patterns for the multi-input / coupler cases, so that the relevant source-side proof target and all B(·) / occ_·(t) / BoundaryReq / ChannelEnabled / ChannelDisabled / WFG notions are interpreted on the explicit abstract channels of Sys_B^elab rather than merely on the raw outer-boundary Sys_B.
(7) Appendix L provides the witness-selection / snooping wrapper where a deterministic or latency-aligned source-side witness is needed.
(8) Appendix N supplies the cited progress premises relied upon by the surrounding liveness/deadlock argument.
(9) Appendix K.2.2 proves the structural bridge from MsgNextFront_P to Front_P. The rendezvous endpoint-request agreement used in wait-for refinement is supplied separately by the corresponding-cutpoint relation (Corollary J.1 / Definition R.18), rather than by any frontier-membership localization.
(10) Appendix K then refines wait-for edges and preserves closed internal message-passing cycles between RTL_B and Sys_ref.